



Digital Transformation

Manual de Operação

Smart Office

Organizador: Anderson Gadelha Fontoura

Julho/2026

Sumário

1	O que é o Smart Office	3
1.1	O problema: RPAs espalhados, sem visibilidade e sem controle	3
1.2	O conceito de <i>control tower</i> : um orquestrador de RPA	3
1.3	O que é o Smart Office	4
1.4	Agnóstico por natureza: ele orquestra, não automatiza	4
1.5	Posição no ecossistema RPA Core	5
1.6	O modelo de desenvolvimento esperado pela LG	5
1.7	Os módulos do Smart Office	6
1.8	Resumo: o que o Smart Office <i>não</i> é	6
2	Arquitetura e Conceitos-Chave	8
2.1	A cadeia de execução: do servidor à intranet	8
2.2	As quatro entidades operacionais	9
2.3	O ciclo de vida de uma execução	10
2.4	Agnosticismo e a pilha de tecnologias	12
3	Acesso, Home e Dashboard	14
3.1	Passo a passo de acesso	14
3.2	A Home do Orchestrator e o RPA Tracking	16
3.3	Os estados de uma task	17
3.4	Para que serve o dashboard no dia a dia	18
4	Automations e o Deploy do RPA	20
4.1	Criar uma automação: "New Automation +"	20
4.2	O coração do deploy: a estrutura obrigatória do ZIP	21
4.3	Owner e parâmetros de ambiente	24
4.4	A tabela de automações	25
4.5	Menu de opções (ícone de três pontos)	26
4.6	O fluxo completo de deploy	26
5	Tasks — Execução Sob Demanda	28
5.1	Task versus Schedule: escolha certa	28
5.2	Dois caminhos para criar uma Task	28
5.3	Preenchendo o popup New Task	29
5.4	O ciclo de vida de uma Task	29
5.5	A tabela de Tasks	30
5.6	Ações sobre uma Task	31
5.7	Busca de Tasks	32
6	Schedules — Execução Recorrente	33
6.1	Criar Schedule — "New Schedule +"	33
6.2	A tabela de Schedules	35
7	Logs e Monitoramento	38
7.1	A tabela de Logs	38
7.2	Os seis estados de execução (Status)	39
7.3	Ações disponíveis em cada log	39
7.4	Diagnosticando uma falha (Status = Failed)	40
7.5	Busca e atualização da tela	41
7.6	Logs só ajudam se o RPA os escrever bem	42

8	Runners e RPA Database	43
8.1	Runners Management	43
8.2	View Runner	44
8.3	RPA Database	47
9	O Runner Client	49
9.1	O que é o Runner e onde ele se encaixa	49
9.2	Instalando o Runner Client	50
9.3	Login na aplicação	50
9.4	Funcionalidades e menu lateral	51
9.5	Criar o Runner na máquina	52
9.6	Acompanhar e iniciar a execução	53
9.7	Configuração de hosts e validação	54
9.8	Encerrando a sessão (Logout)	55
10	Arquitetura de Projeto e Boas Práticas (The DX Way)	57
10.1	Fluxo de Desenvolvimento Padrão	57
10.2	Estrutura de Projeto	58
10.3	Princípios Arquiteturais	59
10.4	Boas Práticas de Codificação Python	59
10.5	Boas Práticas Específicas de RPA	60
10.6	Ciclo de Vida do Bot (BotCity)	62
10.7	Testes e Qualidade	63
11	GitLab: Versionamento e Merge Request	65
11.1	Nomenclatura de Repositórios	65
11.2	GitFlow: Fluxo de Versionamento	66
11.3	Commits Semânticos	67
11.4	Merge Request (MR)	68
12	Nomenclatura e Agendamento de Automações	70
12.1	As quatro entidades da orquestração	70
12.2	Convenção de nomes: departamento, processo e tipo	70
12.3	Nomenclatura de Runners	71
12.4	Deploy e versionamento operacional	72
12.5	Política de Schedulers (agendamentos)	73
12.6	Boas práticas de governança operacional	73
12.7	Monitoramento operacional	74
12.8	Papéis e permissões no orquestrador	74
13	Validação e Deploy em Produção	75
13.1	Etapa 1 — Gatilho: merge na branch main	75
13.2	Etapa 2 — Revisão final (Code Review)	75
13.3	Etapa 3 — Deploy em produção	77
13.4	Etapa 4 — Validação pós-deploy (Smoke Test)	78
13.5	Etapa 5 — Agendamento (Schedule)	80
13.6	Etapa 6 — Monitoramento contínuo	81
13.7	Fluxo de trabalho resumido	81
	Glossário	83

1 O que é o Smart Office

Este manual descreve a operação do **Smart Office**, a plataforma que a LG utiliza para orquestrar suas automações de RPA. Antes de abrir qualquer tela ou clicar em qualquer botão, é preciso entender com clareza o que o Smart Office é — e, tão importante quanto, o que ele *não* é. Muitos erros de operação nascem de uma confusão simples: tratar o Smart Office como se fosse a ferramenta que “faz” a automação. Ele não faz. Ele **comanda** quem faz. Este capítulo estabelece esse enquadramento, que vale para todos os demais.

1.1 O problema: RPAs espalhados, sem visibilidade e sem controle

À medida que uma operação de automação cresce, o número de robôs cresce junto. Sem uma camada central, cada RPA tende a viver isolado na máquina de quem o desenvolveu. Isso cria um cenário conhecido e doloroso:

- **Execução manual e frágil.** Alguém precisa lembrar de rodar o robô no horário certo, na máquina certa. Se essa pessoa está de férias ou o computador foi reiniciado, o processo simplesmente não roda.
- **Falta de visibilidade.** Quando um robô falha às 3 da manhã, ninguém fica sabendo até que a área de negócio reclame da planilha que não chegou. Não há um painel que diga “executou”, “falhou” ou “está rodando agora”.
- **Logs dispersos.** Cada robô grava (ou não) seus registros em um lugar diferente. Investigar um erro vira uma caça ao tesouro pelas máquinas.
- **Ambientes divergentes.** Um robô funciona na máquina do desenvolvedor e quebra em outra porque as versões de bibliotecas Python são diferentes. Não há garantia de que o ambiente de execução seja o mesmo em todo lugar.
- **Sem histórico de versões.** Não se sabe qual versão do código está de fato rodando em produção.

Esse modelo não escala. Dez robôs ainda são administráveis na base do improviso; cem, não. O Smart Office existe justamente para resolver esse problema estrutural.

1.2 O conceito de control tower: um orquestrador de RPA

Um **control tower** (torre de controle) é um **orquestrador de RPA**: um sistema central que comanda o ciclo de vida das automações, sem executar a lógica do robô por conta própria. A analogia é a torre de controle de um aeroporto — ela não pilota nenhum avião, mas sabe onde cada um está, autoriza decolagens, coordena pousos e mantém tudo sob observação.

Na prática, um orquestrador de RPA é responsável por:

- **Cadastrar** as automações — registrar qual robô existe, qual sua versão e quem é o responsável.
- **Provisionar o ambiente** de execução — garantir que as dependências corretas estejam instaladas antes de rodar.
- **Acionar a execução** em uma máquina dedicada, sob demanda ou por agendamento.
- **Coletar logs e status** de cada execução, em tempo real.
- **Monitorar** a saúde da operação como um todo — quantos robôs rodaram, quantos falharam, quantos estão na fila.

O ponto central — e a fonte de metade da confusão de quem começa — é este: **o orquestrador não é o robô**. O orquestrador organiza, agenda e observa; o robô é um código separado, que faz o trabalho de fato. São duas camadas distintas, e o resto deste manual só faz sentido quando essa separação está clara.

1.3 O que é o Smart Office

O **Smart Office** é o control tower da LG. Ele é uma plataforma de orquestração e gerenciamento das operações dos RPAs, que **centraliza toda a operação em um só lugar**, dando controle sobre os processos que antes viviam espalhados.

Seu desenho é inspirado no *BotCity Orchestrator*: o desenvolvedor empacota o robô em um arquivo `.zip` contendo `bot.py` e `requirements.txt`, e sobe esse pacote para a plataforma. A diferença é que o Smart Office é uma solução **enxuta, interna e sem custo adicional de licença** — construída para o uso da LG, com o essencial e sem o peso (nem o preço) de uma ferramenta comercial. É por meio dessa combinação de **maior flexibilidade** e **maior visibilidade** que a plataforma consegue superar ferramentas já consolidadas no mercado, sem que o crescimento da operação implique aumento de custo de licenciamento.

O acesso é feito pelo endereço <http://smartoffice.lge.com>, com login usando o mesmo usuário e senha do computador LG. Dentro do Smart Office, a área de orquestração é aberta pela opção **Orquestrator** no menu lateral. A partir dela, todos os módulos descritos neste manual ficam disponíveis, e a operação passa a trabalhar com metas mensuráveis — por exemplo, uma taxa de execução bem-sucedida superior a 85%, acompanhada por um painel central em vez de por relatos avulsos.

Informação

Ao longo desta documentação, sempre que uma fonte externa mencionar “BotCity Orchestrator” como destino de *deploy*, entenda que, na LG, o orquestrador é o **Smart Office**. O modelo de empacotamento (`.zip` com `bot.py` na raiz) é o mesmo; a plataforma que recebe o pacote é a nossa.

1.4 Agnóstico por natureza: ele orquestra, não automatiza

Este é o enquadramento mais importante do manual inteiro. O Smart Office é **agnóstico**: ele *não é um framework de automação*. Ele não abre navegadores, não clica em botões de sistemas, não lê planilhas. Nada disso é trabalho dele.

O que o Smart Office faz é orquestrar. Ao acionar uma automação, a plataforma provisiona um **ambiente virtual** (`venv`) em um **servidor** e, a partir dele, aciona um **Runner** — uma **máquina real** — para **executar o RPA**. É esse RPA, rodando no Runner, que de fato automatiza algum sistema ou planilha dentro da **intranet da LG**.

Como o Smart Office é agnóstico, o RPA em si é escrito com os frameworks comuns do mercado. O robô pode usar:

Framework	Papel na automação
BotCity	Framework de RPA (web e desktop) com ciclo de vida de bot (<code>on_start</code> , <code>action</code> , <code>on_finish</code>) e cofre de credenciais.
Selenium	Automação de navegador via WebDriver, para interação com sistemas web.
Playwright	Automação de navegador moderna, com esperas inteligentes e suporte multi-browser.

O Smart Office orquestra **qualquer um deles** da mesma forma — para a plataforma, todo robô é apenas um `bot.py` a ser executado dentro de um `venv`. A escolha do framework é uma decisão de desenvolvimento, não de orquestração.

Dica

Guarde esta distinção: **framework** é *como* o robô automatiza; **orquestrador** é *quando, onde e com qual visibilidade* o robô roda. O Smart Office é a segunda coisa. Se você está escrevendo o código que clica na tela, está usando um framework; se está agendando, publicando e monitorando esse código, está usando o Smart Office.

1.5 Posição no ecossistema RPA Core

O Smart Office não vive sozinho. Ele é a camada de **orquestração e monitoramento** dentro do RPA Core, o núcleo responsável pela governança, desenvolvimento e operação das automações corporativas da LG. O desenvolvimento do código acontece com apoio do GitLab; a publicação e a operação acontecem no Smart Office. O caminho completo, do código ao resultado, é o seguinte:

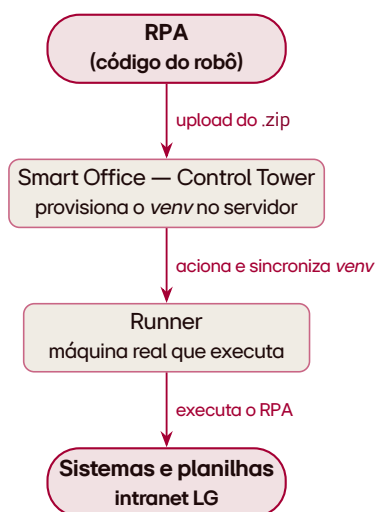


Figura 1: Fluxo de funcionamento: do RPA ao sistema-alvo, orquestrado pelo Smart Office

Lendo de cima para baixo: o **RPA** é o código desenvolvido pelo time; o **Smart Office** recebe esse código, prepara o ambiente virtual e comanda a execução; o **Runner** é a máquina real onde o robô roda; e o alvo final são os **sistemas e planilhas** da intranet LG que serão automatizados. Cada seta é uma responsabilidade distinta, e nenhuma delas se sobrepõe às outras.

1.6 O modelo de desenvolvimento esperado pela LG

Como o Smart Office é agnóstico, ele impõe pouquíssimas exigências ao código — mas essas poucas são **obrigatórias**. O modelo esperado tem dois pilares.

1. Empacotamento. O robô é entregue ao orquestrador como um **.zip**. Para o Smart Office localizar o ponto de entrada, o arquivo **bot.py** e o **requirements.txt** precisam estar na **raiz** do pacote — nunca dentro de uma subpasta:

Estrutura do pacote .zip do RPA

```

meu_rpa.zip |——
bot.py      <- entrypoint, na RAIZ do zip |——
requirements.txt <- dependencias do venv |——
utils.py |——
config.json |——
modules/
  |—— ...
  
```

O `requirements.txt` é o que permite ao Smart Office provisionar o `venv` correto no Runner: é a lista de dependências Python que a plataforma instala antes de executar. Um `requirements.txt` incompleto é a causa número um de robôs que “rodavam na minha máquina” e falham no Runner.

2. Versionamento e revisão. O Smart Office é compatível com o **GitLab**, seguindo o padrão interno conhecido como **The DX Way**: convenções de nomenclatura de repositórios, fluxo de *branches* e revisão via *Merge Request*. O GitLab cobre o *versionamento* e a *revisão de código* — não a publicação no orquestrador.

Atenção

O caminho de publicação para o Smart Office é o **upload manual do .zip**. O Smart Office *não* é compatível com GitHub Actions nem com *pipelines* genéricos de CI/CD para subir o bot ao orquestrador. O GitLab organiza o código e a revisão; quem leva o robô para produção é uma pessoa fazendo o upload do pacote na plataforma.

Erro comum

“Configurei um *pipeline* de CI/CD para dar *deploy* automático no orquestrador quando eu faço *merge* na *main*, mas o robô nunca aparece no Smart Office.”

Solução

No Smart Office não existe *deploy* automático. Faça o *merge* normalmente no GitLab para versionar e registrar a revisão; em seguida, gere o `.zip` (com `bot.py` e `requirements.txt` na raiz) e faça o **upload manual** na tela de criação/atualização de automação. A publicação é sempre um passo humano e explícito.

1.7 Os módulos do Smart Office

A operação do Smart Office é organizada em módulos, cada um responsável por uma parte do ciclo de vida da automação. A tela inicial (**Home / Dashboard**) reúne os principais indicadores do orquestrador; a partir dela, os módulos abaixo dão acesso a cada função. Cada um deles é detalhado em seu próprio capítulo neste manual.

Módulo	Para que serve
Automations	Criação, edição e remoção de automações — os RPAs cadastrados na plataforma. É aqui que se faz o <i>deploy</i> do <code>.zip</code> .
Tasks	Gerencia as execuções de uma automação — cada <i>task</i> é uma instância de execução de um RPA.
Schedules	Cria execuções recorrentes, permitindo que um RPA rode automaticamente após um intervalo definido.
Logs	Acompanha os detalhes de cada execução, incluindo os erros quando um RPA falha.
Runners	Lista os Runners cadastrados e seus status (ativo ou inativo).
View Runner (Runner Details)	Mostra informações e indicadores relevantes de cada Runner individualmente.
RPA Database	Permite visualizar os arquivos das automações, graças à integração com o DX Lake, que armazena os códigos-fonte dos RPAs.

1.8 Resumo: o que o Smart Office não é

Como o enquadramento agnóstico é a fonte da maioria das confusões, vale fixá-lo em negativo. Se um destes pontos soar estranho, releia este capítulo antes de seguir adiante.

Informação**O Smart Office NÃO é:**

- **Não é um framework de automação.** Ele não automatiza sistemas por conta própria — quem faz isso é o RPA (escrito com BotCity, Selenium ou Playwright) rodando no Runner. O Smart Office apenas orquestra.
- **Não tem imagem Docker nem plugin próprios.** Não existe um recurso de infraestrutura “de marca” Smart Office. Ele pode conviver com Docker e outros recursos porque as tecnologias de automação já os utilizam — mas não há um artefato Docker atribuído ao orquestrador em si.
- **Não publica robôs por GitHub Actions nem por CI/CD.** O *deploy* para o orquestrador é sempre o **upload manual do .zip**. O GitLab cuida de versionamento e revisão (o *The DX Way*), não da publicação automática.
- **Não é uma ferramenta paga de mercado.** É uma solução interna, enxuta e sem custo adicional de licença, feita sob medida para a operação da LG.

Com esse alicerce estabelecido — orquestrador, não framework; upload manual, não CI/CD; código no Runner, controle no Smart Office — os próximos capítulos entram em cada módulo, começando pela tela inicial e pela criação da primeira automação.

fim do capítulo ■



Digital Transformation

2 Arquitetura e Conceitos-Chave

Antes de criar a primeira automação, é preciso entender **o que o Smart Office realmente é** e como as peças se encaixam. Este é o capítulo mais importante da Parte I: sem esse modelo mental, cada botão da interface parece arbitrário; com ele, cada tela passa a fazer sentido imediato.

Em uma frase: o Smart Office é um **control tower** — um **orquestrador** de RPA, nos moldes de ferramentas consolidadas de mercado, porém enxuto e voltado ao uso interno da LG, sem custo adicional de licença. Ele **não executa** o robô diretamente e **não é um framework de automação**. Seu papel é **coordenar**: registrar o RPA, provisionar o ambiente onde ele roda, acionar a máquina que o executa e centralizar os logs do que aconteceu.

2.1 A cadeia de execução: do servidor à intranet

Toda execução no Smart Office percorre a mesma cadeia física. O orquestrador vive em um **servidor**, e é nele que se provisiona um **ambiente virtual** (venv) — um Python isolado, com as dependências declaradas em `requirements.txt`, separado de qualquer outra automação. A partir desse ambiente, o orquestrador **aciona um Runner**, que é uma **máquina real** onde o RPA de fato roda. O Runner, por sua vez, executa o robô, e o robô automatiza algum **sistema ou planilha dentro da intranet da LG**.

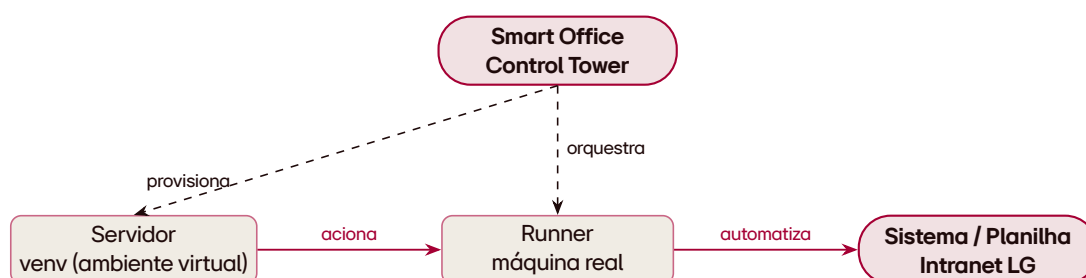


Figura 2: Cadeia de execução: do servidor, passando pelo Runner, até o sistema na intranet

Por que essa separação em camadas? Cada elo resolve um problema específico:

- **Servidor + venv** — garante **isolamento e reprodutibilidade**. Duas automações com versões conflitantes da mesma biblioteca não se atrapalham, porque cada uma vive em seu próprio ambiente virtual. É também o que torna a plataforma **agnóstica**: o venv instala o que o robô pedir, seja BotCity, Selenium ou Playwright.
- **Runner (máquina real)** — é onde existe uma **sessão gráfica de verdade**: um navegador visível, um desktop, acesso de rede à intranet. RPA baseado em interface precisa *ver* a tela para clicar, digitar e ler — por isso quem executa é uma máquina real, e não o servidor.
- **Sistema-alvo na intranet** — é o objetivo final: o ERP, o portal interno, a planilha compartilhada. O robô só o alcança porque o Runner está *dentro* da rede LG.

Atenção

O **Runner é uma máquina real**, não um contêiner nem uma sandbox descartável. É preciso **instalar o Runner Client** na máquina desejada e **iniciá-lo** (*Start Runner*) para que ela passe a receber trabalho. Enquanto o Runner não estiver ligado, as execuções destinadas a ele ficam paradas na fila — e não é um erro do robô, é simplesmente a máquina esperando alguém dar o start.

Informação

O código-fonte de cada RPA fica armazenado por meio da integração com o **DX Lake**, e pode ser baixado a qualquer momento pelo módulo **RPA Databases**. Ou seja: o orquestrador guarda o artefato publicado, além de coordenar sua execução.

2.2 As quatro entidades operacionais

O Smart Office organiza tudo em torno de **quatro entidades**. Elas representam a mesma automação em estágios diferentes do ciclo de execução, e entender a fronteira entre uma e outra evita a maior parte da confusão inicial.

Entidade	O que é	Papel no ciclo
Bot	O código da automação empacotado (o .zip do RPA, escrito em Python).	É o <i>o quê</i> executa. Contém a lógica do robô.
Automation	Registro lógico no orquestrador que <i>vincula</i> um Bot a um ou mais Runners.	Controla <i>deploy</i> , <i>versão</i> e <i>status</i> . É a identidade do RPA na plataforma.
Runner	A máquina real que executa as automações.	Executa os <i>jobs</i> : fica aguardando a fila e roda o que lhe é alocado.
Scheduler	Módulo de agendamento (Task e Schedule) que define <i>quando</i> rodar.	Dispara execuções — únicas (Task) ou recorrentes (Schedule).

A relação entre elas segue sempre o mesmo desenho: um **Bot** é registrado como uma **Automation**, a Automation é alocada a um **Runner**, e o **Scheduler** é quem dispara a execução daquela Automation no momento certo.

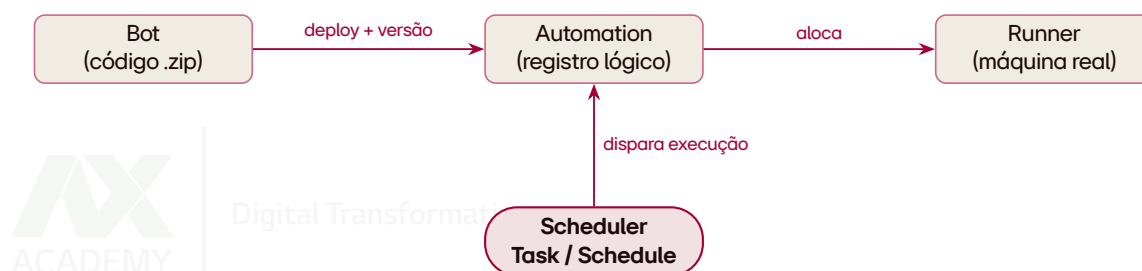


Figura 3: As quatro entidades e como se relacionam: Bot, Automation, Runner e Scheduler

2.2.1 Bot

O **Bot** é puro código — o artefato empacotado que contém a lógica do robô. Sozinho, ele não sabe *quando* rodar nem *em qual máquina*: é apenas o *payload*. No Smart Office, esse artefato chega em um .zip que precisa ter `bot.py` e `requirements.txt` na **raiz** (o detalhe de empacotamento fica no capítulo de Automations).

2.2.2 Automation

A **Automation** é o registro que dá **identidade** ao RPA dentro do orquestrador. Sem ela, o Smart Office não conhece o robô e não há o que agendar ou monitorar. Ela é o ponto que **vincula** o Bot (código) aos Runners (máquinas) e que **controla a versão**: a cada novo upload do .zip, a versão é incrementada automaticamente, com base na quantidade de uploads já feitos. É por isso que atualizar um robô *não* significa recriar a automação — basta subir uma nova versão do código na Automation existente (opção **Select RPA**). Cada Automation também registra o **Owner**, o responsável técnico daquele RPA.

2.2.3 Runner

O **Runner** é a máquina que efetivamente roda o robô. Ele fica **aguardando a fila**: assim que é iniciado (**Start Runner**), recebe todas as automações alocadas para ele e as executa **por ordem de prioridade e de chegada**. Runners seguem convenções de nome como `dx-virtual-machine1` (padrão numérico) ou `scm-runner` (por departamento), e cada um deve ter um *owner* técnico documentado. Na criação de uma Task ou Schedule você escolhe *qual* Runner executará — exceto no modo **Smart** de agendamento, em que o próprio orquestrador seleciona um Runner disponível.

2.2.4 Scheduler: Task e Schedule

O **Scheduler** é o agendamento, e ele tem duas formas que é fundamental não confundir:

Aspecto	Task	Schedule
Natureza	Execução <i>única</i> .	Execução <i>recorrente</i> .
Quando roda	Uma vez, na data/hora marcada, e encerra.	De tempos em tempos, conforme uma Cron Scheduling String .
Campos-chave	Automation, Date, Runner, Priority (1 a 5).	Name, Automation, Type (Manual / Smart), Cron, Runner, Priority.
Uso típico	Rodar agora, testar, reprocessar um dia específico.	Rotina diária/semanal/mensal do processo em produção.

A **prioridade** vai de 1 (mais urgente) a 5 (menos urgente) e define a ordem quando várias execuções disputam o mesmo Runner. No Schedule, o campo **Type** decide se você informa o Runner (**Manual**) ou se o orquestrador escolhe um automaticamente (**Smart**). A recorrência usa o formato **Cron**: por exemplo, `0 8 * * *` roda todo dia às 8h, `0 8 * * 1` roda toda segunda-feira às 8h, e `0 8 1 * *` roda no dia 1 de cada mês.

Dica

Bot, Automation e Schedule de um mesmo processo devem **compartilhar o mesmo identificador base**, mudando só o sufixo do tipo: `SCM_InquiryEndingOnhand_BOT`, `SCM_InquiryEndingOnhand_AUTO` e `SCM_InquiryEndingOnhand_SCHED`. Isso mantém as três faces da mesma automação rastreáveis lado a lado nas telas de busca.

2.3 O ciclo de vida de uma execução

Com as entidades no lugar, uma execução individual sempre percorre o mesmo caminho. Você cria uma **Task** (ou o Schedule cria uma por você); ela entra na **fila** do Runner; quando o Runner está iniciado, ela passa a **Running**; ao terminar, gera um **Log** com o resultado.

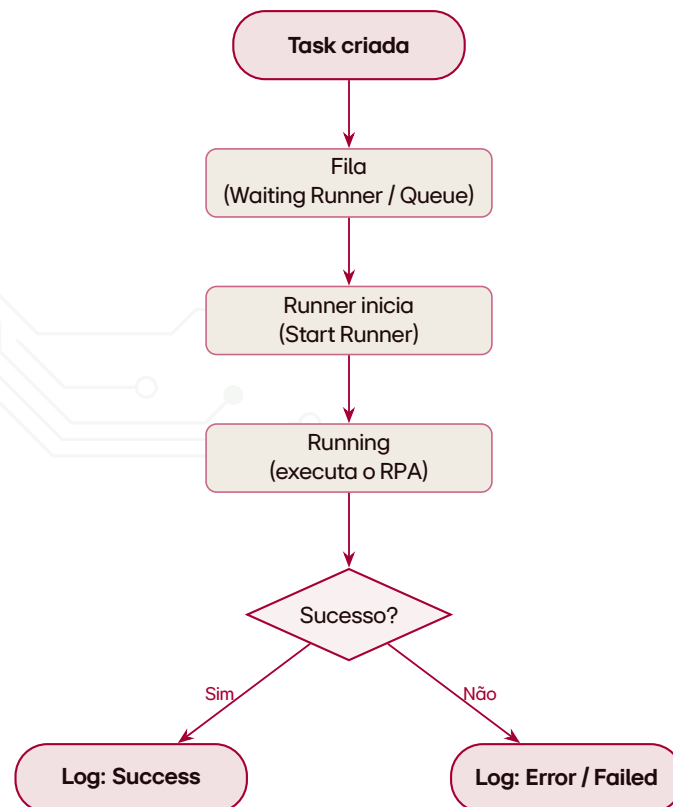


Figura 4: Ciclo de vida de uma execução no Smart Office

Os **estados** que aparecem no Dashboard e nos Logs correspondem exatamente a essas etapas:

Estado	Significado
Waiting Runner	A task foi criada e aguarda o Runner especificado ser iniciado. Enquanto o Runner não ligar, ela permanece aqui.
Queue / In Queue	A execução aguarda na fila do Runner — porque ele ainda não iniciou ou já está rodando outra execução. Comum em tasks recorrentes.
Running	Está executando naquele momento na máquina.
Completed / Success	Executou até o fim com sucesso.
Error / Failed	A execução falhou; os detalhes ficam no Log para diagnóstico.
Canceled	Foi cancelada pelo usuário — inclusive em tempo real, mesmo durante a execução, pela ação Cancel Task .

Erro comum

Uma task fica presa em **Waiting Runner** e “nada acontece”. O reflexo é achar que o robô travou ou que o agendamento falhou.

Solução

Quase sempre o **Runner de destino não foi iniciado**. Abra a tela de acompanhamento do Runner e clique em **Start Runner**. Ao iniciar, todas as execuções alocadas para aquela máquina são enviadas e processadas por ordem de prioridade. Confirme também que a task foi criada apontando para o Runner certo.

Informação

Todo o histórico fica no módulo **Logs**, com **Id**, **Status**, **Runner**, datas e o **Event** de cada execução. A tela se atualiza sozinha a cada 10 segundos, então dá para acompanhar uma execução em tempo real sem recarregar a página.

2.4 Agnosticismo e a pilha de tecnologias

O ponto que mais diferencia o Smart Office de um framework de automação é o **agnosticismo**. O orquestrador não impõe *como* o robô é escrito — ele só exige o contrato mínimo de empacotamento (**bot.py** e **requirements.txt** na raiz do **.zip**). Tudo o que estiver dentro do **venv** é responsabilidade do RPA. Por isso, o robô pode usar os **frameworks comuns do mercado**: **BotCity**, **Selenium** ou **Playwright**. O Smart Office orquestra qualquer um deles da mesma forma.

Categoria	Ferramentas / Frameworks	Finalidade
RPA	Python, BotCity Framework, Selenium, Playwright	Criação e execução de automações baseadas em interface (UI).
Versionamento	GitLab	Versionamento, nomenclatura e revisão (Merge Request) — o <i>The DX Way</i> .

O Smart Office **não possui plugin próprio nem imagem Docker atribuída a ele**. Isso não é limitação: como as tecnologias de RPA já trazem esses recursos, eles podem ser incorporados normalmente pelo robô. O orquestrador se mantém enxuto justamente por não reinventar o que o ecossistema já oferece.

2.4.1 Deploy é upload de .zip

O caminho para colocar um robô no orquestrador é o **upload manual do arquivo .zip** na criação (ou atualização) de uma **Automation**. Não existe publicação automática. O empacotamento correto e o passo a passo do upload são detalhados no capítulo de Automations; aqui basta fixar o conceito: **deploy = subir o zip**.

Atenção

O Smart Office **não publica o bot via CI/CD**. Ele é compatível com **GitLab** para **versionamento, nomenclatura e Merge Request** (o *The DX Way*) — mas o GitLab cobre revisão e controle de versão do código, **não** a publicação no orquestrador. Não existe integração com GitHub Actions nem pipeline genérico que suba o robô sozinho. Sempre que a documentação de origem citar “deploy no orquestrador via CI/CD”, entenda no Smart Office como **upload manual do .zip**.

Erro comum

O deploy do **.zip** falha e o orquestrador “não reconhece” o RPA. Costuma acontecer quando o projeto foi compactado com o conteúdo dentro de uma pasta interna (por exemplo, **meu_rpa/bot.py**).

Solução

O orquestrador procura o **entrypoint** na **raiz** do **.zip**. Selecione os **arquivos** do projeto — e não a pasta que os contém — antes de compactar, de modo que **bot.py** e **requirements.txt** fiquem no nível principal do zip. Se o **bot.py** estiver aninhado, o Smart Office não localiza o ponto de entrada e o deploy quebra.

Com esse modelo em mente — a cadeia servidor/venv → Runner → intranet, as quatro entidades e

o ciclo de vida de uma execução — os próximos capítulos, que percorrem tela a tela cada módulo do orquestrador, deixam de ser uma lista de botões e passam a ser a operação concreta dos conceitos vistos aqui.

fim do capítulo ■



Digital Transformation

3 Acesso, Home e Dashboard

O primeiro contato de qualquer operador com o **Smart Office** acontece pela tela de *Home*. É a partir dela que se enxerga, em um único painel, a saúde de toda a operação de automação: quantos RPAs estão rodando agora, quantos concluíram com sucesso, quantos falharam e quantos aguardam execução. Antes de criar automações, agendar tarefas ou investigar erros, é essencial dominar o caminho de acesso e aprender a ler o dashboard — é essa leitura que orienta todas as decisões do dia a dia.

Lembre-se do enquadramento: o Smart Office é o **control tower** (torre de controle) da LG, um orquestrador de RPA de uso interno. Ele não executa a automação diretamente; ele provisiona o ambiente e aciona os *runners* (máquinas reais) que rodam os robôs dentro da intranet. A Home é a janela por onde você observa esse orquestrador trabalhando.

3.1 Passo a passo de acesso

O acesso ao orquestrador segue quatro passos simples. O fluxo abaixo resume o caminho da URL até a tela inicial do *Orchestrator*.

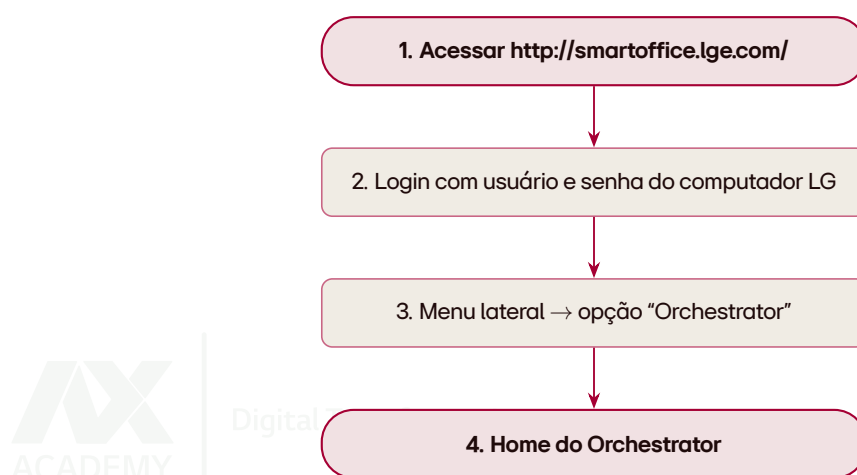


Figura 5: Fluxo de acesso ao Orchestrator do Smart Office

3.1.1 Etapa 1 — Abrir o Smart Office

Abra o navegador e acesse <http://smartoffice.lge.com/>. Como o Smart Office é um sistema interno, hospedado dentro da rede da LG, esse endereço só resolve a partir da **intranet corporativa**. Fora dela — em uma rede doméstica sem VPN, por exemplo — o navegador não encontrará o servidor.

Atenção

O Smart Office vive na intranet da LG. Se a página não carregar, o problema quase nunca é o sistema: confirme primeiro que você está conectado à rede corporativa (cabo, Wi-Fi interno ou VPN). O orquestrador aciona runners que automatizam sistemas e planilhas internas — por isso todo o ecossistema é fechado à rede da empresa.

3.1.2 Etapa 2 — Fazer login

Efetue o login com o **mesmo usuário e senha do seu computador LG**. Não há cadastro separado nem senha própria do Smart Office: a autenticação reaproveita a sua identidade corporativa.

Há uma razão prática para isso. Ao identificar você pela conta corporativa, o orquestrador conse-

que registrar quem é o *owner* de cada automação e manter a rastreabilidade das ações — quem criou, quem editou, quem executou. Essa atribuição de responsável é o que sustenta a auditoria da operação.

Dica

Se a sua senha do computador LG mudou recentemente (troca periódica de senha corporativa), use a senha *nova* também aqui. Como a credencial é a mesma, uma senha desatualizada é a causa mais comum de falha de login no primeiro acesso do dia.

3.1.3 Etapa 3 — Entrar no Orchestrator

Após o login, o Smart Office reapresenta a sua **tela inicial**, agora com *mais opções no menu lateral* do que apareciam antes de você se autenticar. Essa tela inicial é a visão macro do Control Tower (Figura 6): o painel estratégico da operação.



Figura 6: Tela inicial do Smart Office (Control Tower), com o FTE acumulado por departamento, o medidor de KPI Progress e o painel de Office Automations.

Nessa visão macro você encontra indicadores de alto nível: o gráfico *FTE Accumulated (Hours by Department)*, que soma as horas de trabalho poupadas pela automação em cada área; o medidor *KPI Progress*, comparando a meta anual com o realizado; e o bloco *Office Automations*, que resume o FTE total acumulado, a taxa de execução (*Running*) e a quantidade de automações cadastradas. É a leitura gerencial do parque de robôs — útil para gestão, mas não é ainda o painel operacional do orquestrador.

Para chegar ao painel de operação, clique na opção **“Orchestrator”** no menu lateral. O sistema então abre a *Home do Orchestrator*, que é o foco deste capítulo.

Erro comum

Ao entrar no Smart Office, a opção **“Orchestrator”** não aparece no menu lateral.

Solução

O menu lateral é montado de acordo com as permissões do seu usuário. Se a opção “Orchestrator” não está visível, provavelmente o seu perfil ainda não recebeu acesso ao módulo de orquestração. Solicite a liberação ao time responsável informando o seu usuário corporativo — não é um erro do sistema, e sim uma questão de permissão.

3.2 A Home do Orchestrator e o RPA Tracking

A Home do *Orchestrator* concentra os principais indicadores do dia. No topo há uma barra de navegação com as abas **Home**, **Logs**, **Runners** e **RPA Database**; logo abaixo, à esquerda, fica a seção que interessa ao acompanhamento operacional: o **RPA Tracking** (Figura 7).

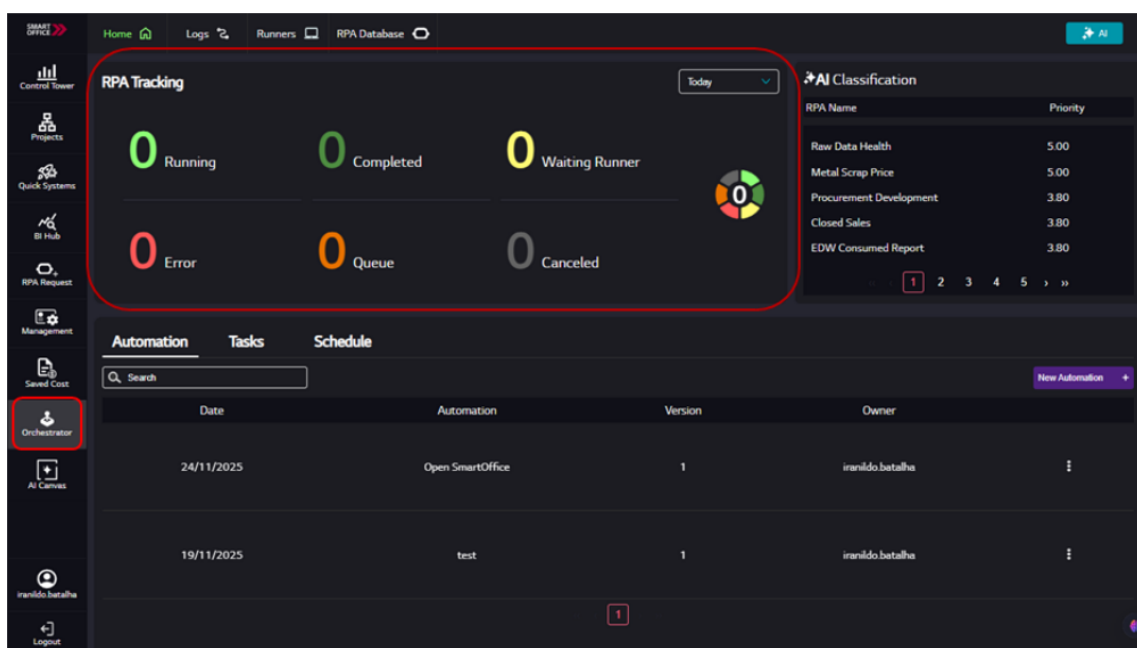


Figura 7: Home do Orchestrator. Em destaque, a seção *RPA Tracking* com os indicadores de estado das tasks e o seletor de período (Today/Monthly). Abaixo, as abas Automation, Tasks e Schedule.

O RPA Tracking apresenta **estatísticas dos status de cada tarefa** — ou seja, quantas *tasks* (instâncias de execução de um RPA) estão em cada estado. No canto superior direito dessa seção há um seletor de período que recalcula todos os indicadores. As duas opções são:

Filtro	O que apresenta
Today	As métricas de execução dos RPAs no <i>dia atual</i> . É a visão de operação — “o que aconteceu hoje”.
Monthly	As métricas de execução referentes ao <i>mês atual</i> . É a visão de acompanhamento — “como está o mês”.

Informação

Ao alternar entre *Today* e *Monthly*, os indicadores de estado são recalculados de uma só vez para o novo período. Um número que parece baixo em “Today” pode ser expressivo em “Monthly”: são recortes de tempo diferentes sobre os mesmos dados de execução.

3.3 Os estados de uma task

Cada task executada pelo orquestrador passa por uma sequência de estados, e o RPA Tracking mostra a **quantidade de tasks em cada um deles**, com base nos *Logs* dos RPAs. Entender o significado exato de cada estado é o que permite diagnosticar a operação em segundos — um número fora do esperado em determinado estado aponta diretamente para o que precisa de atenção. Os estados aparecem lado a lado, cada um com sua cor no painel (Figura 8).

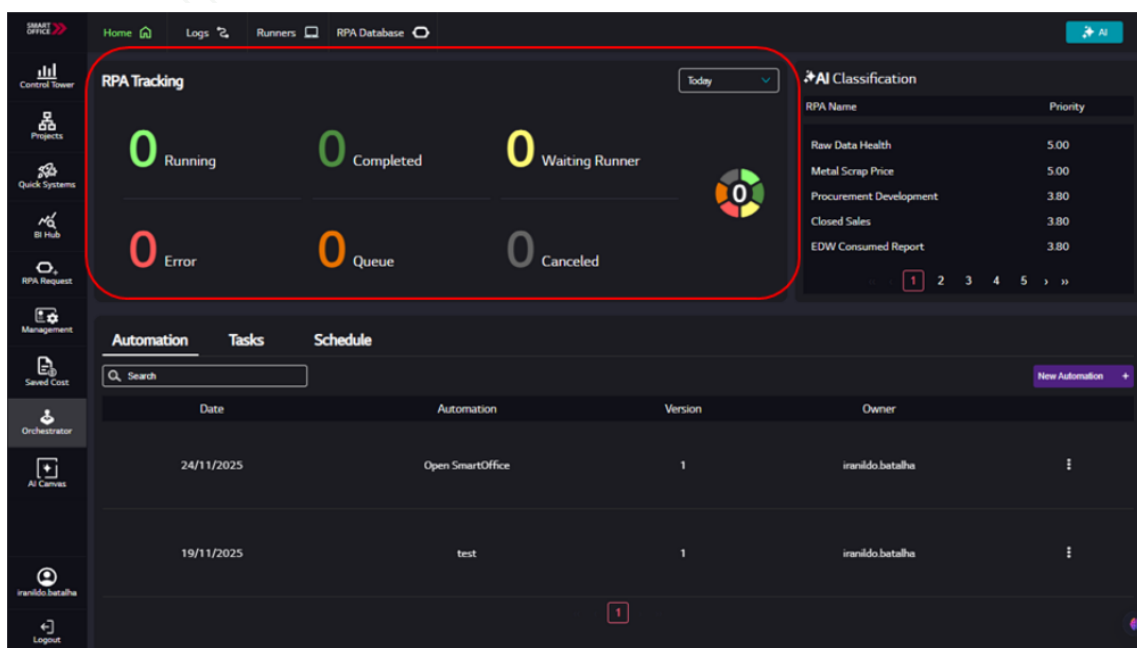


Figura 8: Indicadores de estado no RPA Tracking: Running, Completed, Waiting Runner, Error, Queue e Canceled, cada um com sua contagem e cor.

Estado	Cor	O que significa
Running	Verde	Quantidade de RPAs <i>executando naquele momento</i> . É a carga em tempo real sobre os runners: quanto está de fato em andamento agora.
Completed	Verde	Quantidade de RPAs que <i>executaram com sucesso</i> . Respeita o filtro: em <i>Today</i> , os que concluíram hoje; em <i>Monthly</i> , os que concluíram no mês. É o indicador de produtividade da operação.
Waiting Runner	Amarelo	Tasks que foram <i>criadas e aguardam execução</i> no runner especificado. Enquanto o runner escolhido para a task não for iniciado, ela permanece nesse estado.
Error	Vermelho	Tasks que <i>apresentaram algum erro</i> durante a execução. É o indicador que dispara a investigação — toda contagem aqui merece um olhar nos Logs.
Queue	Laranja	Tasks recorrentes (executadas em intervalos de tempo) que <i>aguardam na fila</i> do runner. A task entra nesse estado quando o runner ainda não foi iniciado ou quando já há outra task executando e esta precisa esperar a vez.
Canceled	Cinza	Tasks que foram <i>canceladas pelo usuário</i> por algum motivo. Conforme o período selecionado, mostra os cancelamentos do dia ou do mês.

Atenção

Há uma diferença sutil e importante entre **Waiting Runner** e **Queue**. Ambos indicam tasks aguardando, mas a causa é distinta: *Waiting Runner* é uma task pontual esperando que o seu runner seja *ligado*; *Queue* é uma task *recorrente* esperando a vez na fila (porque o runner não

está iniciado ou porque outra task já ocupa o runner). Confundir os dois leva a diagnósticos errados — um pede “ligar o runner”; o outro pode ser simplesmente a fila trabalhando normalmente.

Erro comum

Todos os indicadores do RPA Tracking estão em 0.

Solução

Zero não é necessariamente falha. Verifique primeiro o filtro de período: com *Today* selecionado, se nenhuma task foi disparada ainda no dia, todos os contadores ficam em 0 — o que é o esperado no início do expediente. Troque para *Monthly* para confirmar que há atividade no mês. Se, mesmo em *Monthly*, tudo permanece zerado e você sabe que há automações ativas, aí sim vale investigar os runners e os Logs.

3.4 Para que serve o dashboard no dia a dia

Na rotina de operação, a Home do Orchestrator é o **primeiro lugar a ser aberto** a cada dia. Ela funciona como um painel de instrumentos: com um único olhar, o operador sabe se a operação está saudável ou se algo exige ação imediata. A leitura recomendada segue uma ordem de prioridade:

1. **Comece com o filtro em Today** e olhe primeiro o *Error*. Qualquer número maior que zero aqui é o sinal mais urgente — é o gatilho para abrir a aba *Logs* e investigar a falha (mensagem de erro, runner, horário e etapa em que o RPA parou).
2. **Observe Waiting Runner e Queue**. Contagens persistentes nesses estados costumam indicar um *runner que não foi iniciado*. Se as tasks não saem do lugar, verifique na aba *Runners* se a máquina responsável está ativa.
3. **Confira Running e Completed**. Juntos, mostram o ritmo do dia: o que está em andamento e o que já concluiu com sucesso. É a confirmação de que a operação está produzindo.
4. **Troque para Monthly** quando quiser a visão de acompanhamento do mês — útil para relatórios e para perceber tendências que não aparecem no recorte diário.

Dica

Adote a Home como ritual de abertura do dia: filtro em *Today*, olhar no *Error* primeiro, depois *Waiting Runner/Queue*, depois *Running/Completed*. Essa varredura de poucos segundos evita que uma automação falhe silenciosamente e só seja notada quando a área de negócio cobra o resultado.

Essa disciplina de monitoramento é o que sustenta a meta operacional da LG de manter uma taxa de execução alta (superior a 85%) e a classificação de saúde da operação. O dashboard não é decorativo: cada indicador é um ponto de decisão. *Error* manda investigar; *Waiting Runner* e *Queue* mandam checar runners; *Completed* confirma entrega. Ler o painel corretamente é a diferença entre reagir às falhas em minutos e descobrir o problema tarde demais.

Informação

Além do RPA Tracking, a Home exibe, à direita, um painel de *AI Classification* que lista os RPAs por prioridade (visível na Figura 7) e, logo abaixo, as abas *Automation*, *Tasks* e *Schedule*, que dão acesso ao cadastro de automações, às execuções e aos agendamentos. Essas áreas são de-

talhadas nos capítulos seguintes — por ora, basta saber que a Home é o ponto de partida a partir do qual toda a navegação do orquestrador se ramifica.

fim do capítulo ■



Digital Transformation

4 Automations e o Deploy do RPA

Este é o capítulo central da operação. Tudo o que você desenvolveu — o código do robô, as dependências, os módulos de apoio — só passa a existir para o orquestrador quando vira uma **Automation**. É aqui que o Smart Office deixa de ser um painel de leitura e passa a ser o control tower que provisiona ambiente, aciona o Runner e coloca o RPA em produção dentro da intranet da LG.

Antes de qualquer coisa, um alinhamento de vocabulário. No Smart Office, uma **Automation** é o *registro lógico* que identifica um RPA para o orquestrador: é ela que vincula um código (o robô) a um ou mais Runners e controla deploy, versão e status. O código em si é agnóstico — pode usar **BotCity**, **Selenium** ou **Playwright** indistintamente —, porque o Smart Office não é um framework de automação: ele orquestra qualquer um deles. Cadastrar a Automation é, portanto, o pré-requisito para monitorar, agendar e executar um RPA na plataforma.

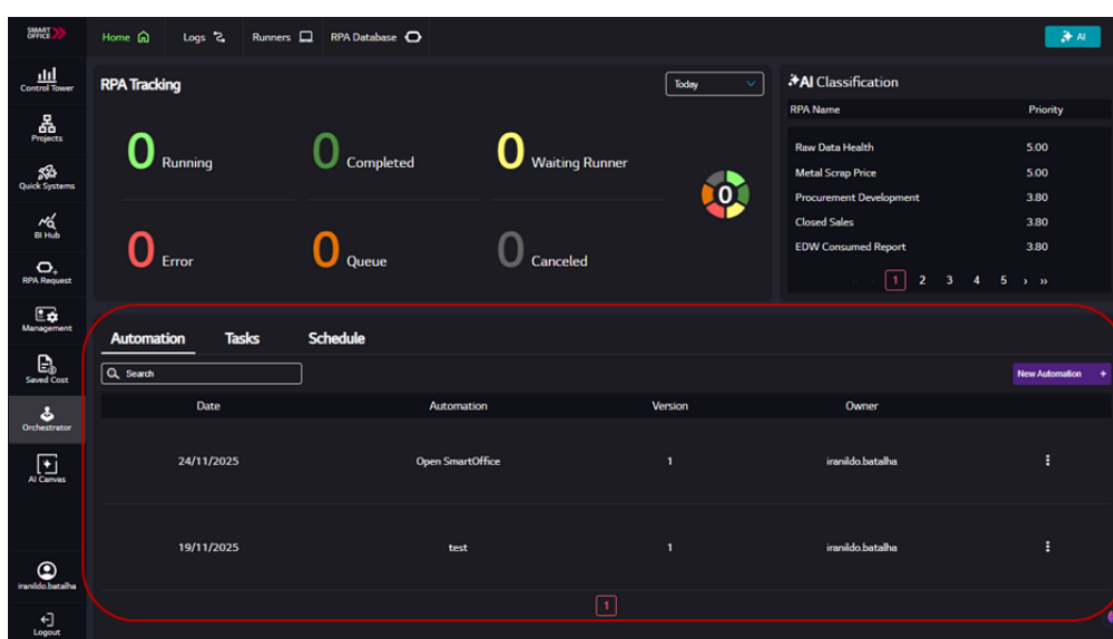


Figura 9: Aba Automations: todas as automações cadastradas na plataforma. Enquanto o RPA não estiver aqui, ele não existe para o orquestrador.

4.1 Criar uma automação: “New Automation +”

Na aba Automations há um botão roxo no canto inferior direito com o texto **New Automation +**. Ao clicá-lo, abre-se um popup com os campos que identificam o RPA e a seção onde o pacote de código será enviado.

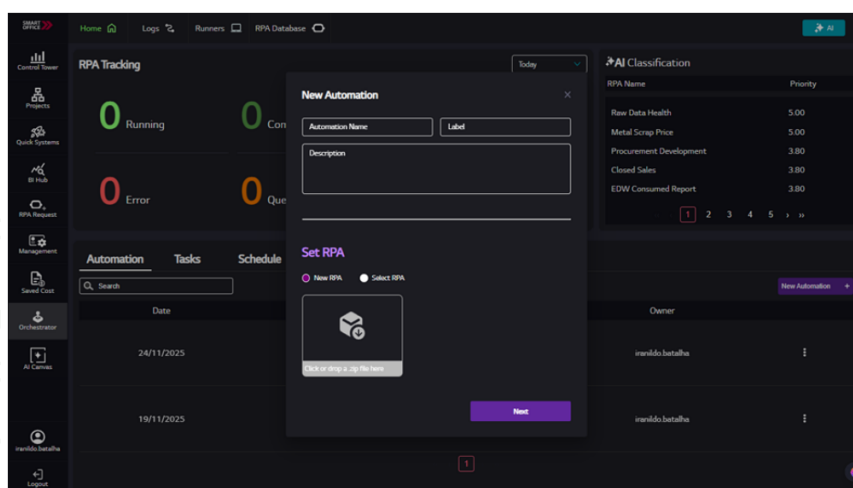


Figura 10: Popup de criação da automação: os campos de identificação (topo) e a seção **Set RPA** para o deploy (base).

Os três campos de identificação são:

Campo	O que preencher
Automation Name	O nome que identifica a automação dentro do módulo do orquestrador. É o rótulo pelo qual você vai encontrar e reconhecer o RPA nas telas de Tasks, Schedules e Logs.
Label	Um rótulo auxiliar de busca. Use texto em minúsculo e troque espaços por <i>underline</i> , por exemplo <i>my_automation</i> . Serve para filtrar rapidamente na barra de pesquisa.
Description	Descreva o RPA: <i>o que ele faz, por que foi criado e qual o ROI</i> obtido. Não é campo decorativo — é o que permite a quem opera o orquestrador entender o processo sem abrir o código.

Dica

O modelo de desenvolvimento da LG adota uma convenção de nomes no formato `DEPARTAMENTO_NOME_DO_PROCESSO_TIPO`. Para o registro de automação, o sufixo é **AUTO** — por exemplo, `SCM_InquiryEndingOnhand_AUTO`. Manter o mesmo identificador base entre o bot, a automação e o schedule facilita rastrear a mesma automação nos diferentes estágios do ciclo de execução.

4.1.1 Set RPA: New RPA vs. Select RPA

Na parte inferior do popup há a seção **Set RPA**, onde o pacote de código é anexado. Ela oferece duas opções, e escolher a errada é uma fonte comum de confusão:

Opção	Quando usar
New RPA	Quando o RPA é <i>novo</i> no sistema — primeiro deploy, ainda não existe código associado.
Select RPA	Quando já existe um RPA cadastrado e você quer apenas <i>atualizar</i> o código/versão dele. Cada novo envio por aqui incrementa o número de versão da automação.

4.2 O coração do deploy: a estrutura obrigatória do ZIP

Aqui está a regra que faz ou quebra o deploy. Para publicar o RPA no orquestrador, você envia um arquivo **.zip** com o código-fonte do robô. O Smart Office só reconhece o RPA se, *no nível principal (raiz) do zip*, existirem os arquivos `bot.py` e `requirements.txt`.

Atenção

bot.py e **requirements.txt** **devem estar na raiz do zip**, nunca dentro de uma pasta interna. Se o **bot.py** não estiver na raiz, o orquestrador não consegue localizar o **entrypoint** e o deploy falha.

Informação

Por que a raiz importa tanto? O Smart Office é um control tower agnóstico: ele não roda o robô diretamente. Ao receber o zip, o orquestrador **provisiona um ambiente virtual (venv)** em um servidor a partir do **requirements.txt** — é ali que ele instala BotCity, Selenium, Playwright ou o que quer que suas dependências declarem. Em seguida, aciona um **Runner** (uma máquina real) que executa o **bot.py** como ponto de entrada do RPA. Se esses dois arquivos estiverem afundados em **meu_rpa/** ou **rpa_v1/src/**, o orquestrador olha para a raiz, encontra uma pasta em vez do **bot.py**, e não tem como montar o venv nem chamar o **entrypoint**.

Estrutura correta (arquivos na raiz).

```
# CORRETO: bot.py e requirements.txt na RAIZ do .zip
meu_rpa.zip
  bot.py           # entrypoint - OBRIGATORIO na raiz
  requirements.txt # dependencias - OBRIGATORIO na raiz
  utils.py
  config.json
  modules/
    processa.py
```

Estrutura incorreta (não funciona).

```
# ERRADO: tudo dentro de uma pasta interna
meu_rpa.zip
  meu_rpa/
    bot.py           # o orquestrador NAO enxerga aqui dentro
    requirements.txt
    modules/

# ERRADO: aninhado em src/
rpa_v1.zip
  rpa_v1/
    src/
      bot.py         # nivel abaixo da raiz -> deploy falha
      requirements.txt
```

Erro comum

Deploy recusado ou automação que nunca inicia, com o **entrypoint** “não encontrado”. Quase sempre a causa é a mesma: o zip foi gerado compactando *a pasta do projeto*, e não os arquivos de dentro dela. O resultado é o **bot.py** um nível abaixo da raiz.

Solução

Compacte os **arquivos**, não a pasta. Abra o diretório do projeto, selecione **bot.py**, **requirements.txt** e os demais itens necessários, e só então gere o zip. O **bot.py** tem que aparecer no primeiro nível ao abrir o arquivo compactado.

4.2.1 Como preparar o ZIP corretamente

O passo a passo, no Windows, é direto:

1. Organize o diretório do RPA com o `bot.py` e o `requirements.txt` no nível de cima, junto dos módulos de apoio (`utils.py`, `config.json`, pasta `modules/` etc.).
2. **Selecione todos os arquivos dentro da pasta** — e não a pasta em si. Este é o passo que a maioria erra.
3. Com os arquivos selecionados, use o menu de contexto do Windows: “Enviar para” e então “Pasta compactada (.zip)”.

A estrutura final, ao abrir o zip gerado, deve mostrar diretamente:

Conteúdo final do zip

```
bot.py
requirements.txt
utils.py
config.json
modules/
```

Com o zip pronto, na seção **Set RPA** escolha **New RPA** (ou **Select RPA**, se for atualização) e clique no ícone de **upload** para carregar o arquivo. O arquivo enviado sempre deve ser um **.zip**.

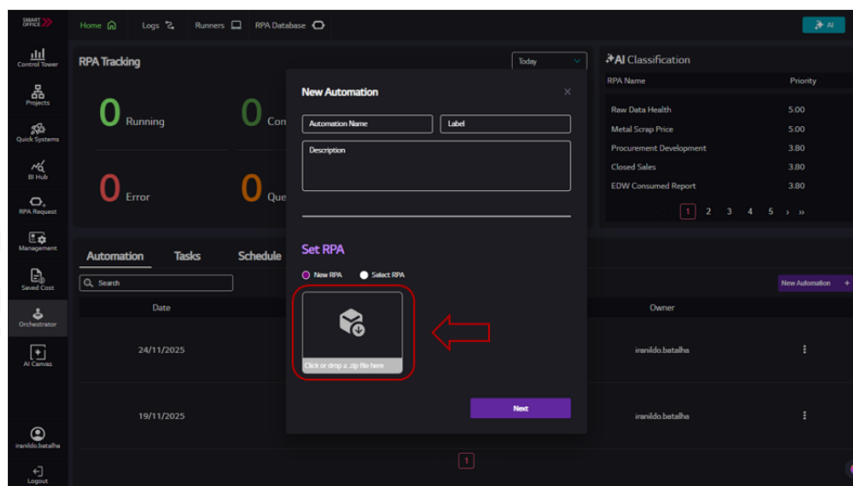


Figura 11: Upload do pacote na seção **Set RPA**. O único formato aceito é **.zip**.

Dica

O `bot.py` é apenas o *ponto de entrada*; ele não precisa conter toda a lógica. Um esqueleto agnóstico típico só orquestra o que mora nos módulos de apoio:

Esqueleto de um `bot.py` agnóstico

```
1 # bot.py - entrypoint que o Runner executa
2 from modules import processa
3
4 def main():
5     # aqui entra a automacao: BotCity, Selenium ou Playwright
6     processa.run()
7
```

```
8 if __name__ == "__main__":  
9     main()
```

4.2.2 Ligando ao modelo de projeto

Vale distinguir duas coisas que costumam se confundir. No capítulo sobre o modelo de desenvolvimento você viu a **estrutura modular do repositório** — `modules/`, `utils/`, `resources/`, `tests/` —, versionada no GitLab e organizada para manutenção, testes e revisão. O que sobe para o orquestrador é outra coisa: é o **artefato de execução**, um recorte enxuto desse repositório contendo só o que o robô precisa para rodar, com `bot.py` e `requirements.txt` na raiz e os módulos de apoio ao lado.

Ou seja: você *não* compacta o repositório inteiro (com `tests/`, `docs/`, artefatos de CI). Você monta o pacote de produção. É a mesma automação, em uma forma preparada para o control tower montar o venv e executar.

Atenção

O deploy no Smart Office é sempre um **upload manual do zip**. Não existe pipeline de CI/CD que publique o robô no orquestrador automaticamente, e o Smart Office *não* usa GitHub Actions. O papel do **GitLab** é o versionamento e a revisão — nomenclatura, branches, Merge Request, o “The DX Way” —, mas o passo final de subir o pacote é feito aqui, à mão, nesta tela de **Automations**. Sempre que uma documentação antiga citar “BotCity Orchestrator” como destino do deploy ou um pipeline de CI/CD subindo o bot, entenda: aqui o orquestrador é o Smart Office e a publicação é o upload do zip.

4.3 Owner e parâmetros de ambiente

Preenchidos os campos e anexado o zip, clique em **Next**. A próxima tela pede a pessoa **dona (owner)** do RPA: localize o seu nome e selecione-o. O owner é o responsável técnico pela automação — quem responde por ela quando algo precisa de manutenção.

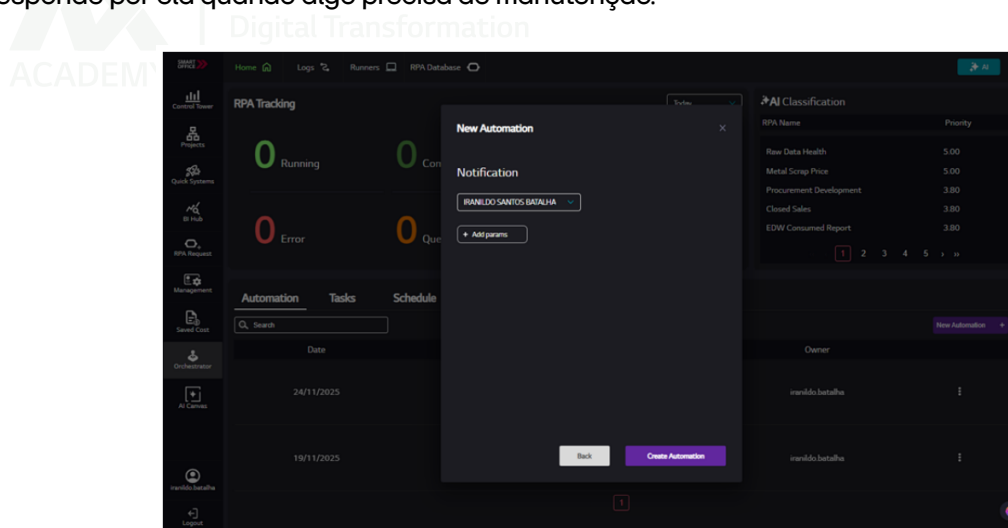


Figura 12: Seleção do owner e o botão + Add params para declarar as variáveis de ambiente do RPA.

Abaixo da seleção de nome há o botão **+ Add params**. Ele serve para informar que o RPA precisa de alguns parâmetros para executar — tipicamente **variáveis de ambiente** que o robô consome em tempo de execução (URLs, timeouts, identificadores). Para usar:

1. Clique em **+ Add params** e preencha os campos de acordo com a necessidade do seu RPA.
2. Confira tudo e clique em **Create Automation**.

Dica

Os params são o mecanismo correto para *não* deixar configuração nem credenciais escritas no código. O código lê esses valores do ambiente que o orquestrador injeta ao provisionar o venv; assim o mesmo `bot.py` roda em máquinas e Runners diferentes sem alteração. Nunca hardcode URL, usuário ou senha no fonte — declare como parâmetro aqui.

Feito isso, a automação é criada e passa a aparecer na tabela de automações.

4.4 A tabela de automações

Cada automação criada vira uma linha na tabela principal da aba **Automations**.

Date	Automation	Version	Owner
24/11/2025	Open SmartOffice	1	iranielo.batalha
19/11/2025	test	1	iranielo.batalha

Figura 13: Tabela de automações com Date, Automation, Version e Owner.

As colunas são:

Coluna	Significado
Date	Data de criação da automação.
Automation	Nome da automação criada (o Automation Name do popup).
Version	Número da versão da automação, calculado com base no número de uploads do arquivo. Cada vez que você usa Select RPA e reenvia o zip, a versão sobe.
Owner	Nome da pessoa dona/responsável pelo RPA.

Informação

O **Version** do Smart Office conta *uploads* no orquestrador, não é a *tag vXYZ* do GitLab. São dois relógios diferentes: o do GitLab marca a evolução do código-fonte revisado; o do Smart Office marca quantas vezes aquele pacote foi publicado. Manter os dois coerentes — só publicar o que já passou por Merge Request — é parte da disciplina esperada.

4.5 Menu de opções (ícone de três pontos)

Ao final de cada linha há um ícone de **três pontos** que abre o menu de ações da automação.

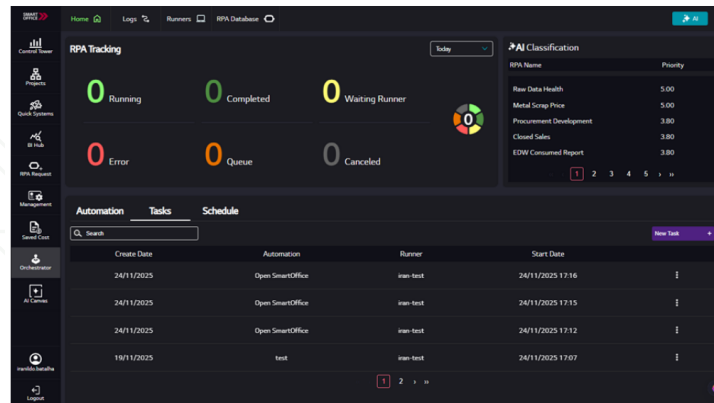


Figura 14: Menu de opções por automação: New Task, Edit, Delete e Download.

Opção	O que faz
New Task	Cria uma <i>task</i> para esta automação — um agendamento para executar o RPA em um momento definido.
Edit	Edita as informações do RPA ou <i>atualiza o código</i> em caso de nova versão (equivalente a um novo upload via Select RPA).
Delete	Remove a automação quando ela não é mais necessária.
Download	Baixa a <i>última versão</i> do código-fonte do RPA associado, como arquivo .zip .

Acima da tabela há ainda um campo de busca. A pesquisa usa como critério o **nome da automação** — basta digitar parte do nome para filtrar a lista.

4.6 O fluxo completo de deploy

Reunindo tudo, o caminho do código pronto até a automação ativa na tabela é o seguinte. Note que o único ponto de falha do deploy propriamente dito é a estrutura do zip:

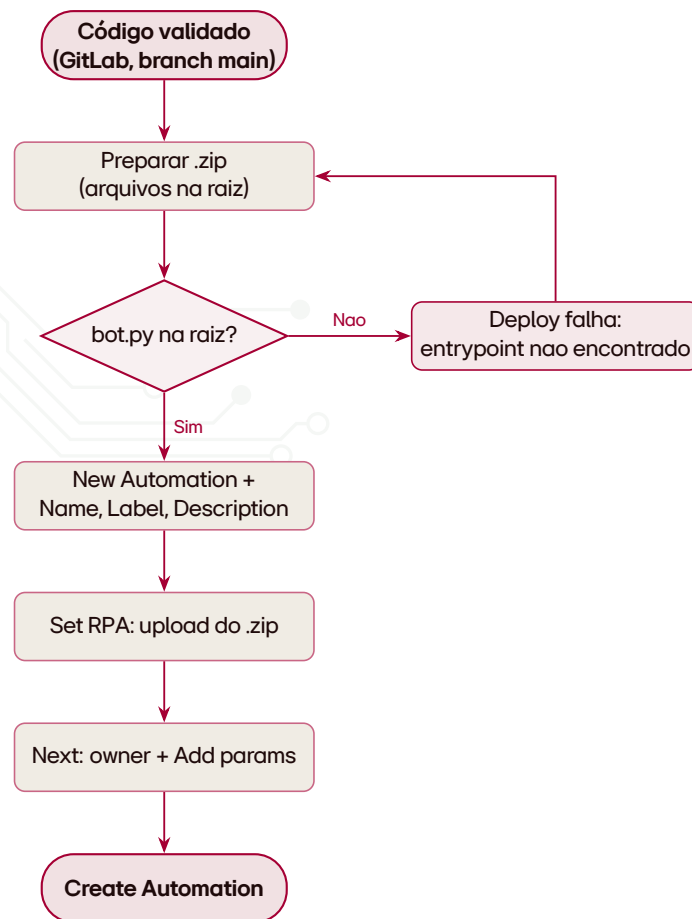


Figura 15: Fluxo completo de deploy de um RPA, do código ao upload no orquestrador

Com a automação criada e listada na tabela, o RPA está *cadastrado* — mas ainda não roda sozinho. Para colocá-lo em execução, você cria uma *Task* (execução única) ou uma *Schedule* (execução recorrente), apontando para um Runner que fará o trabalho na máquina real. Esses são os próximos passos da operação.

fim do capítulo ■

5 Tasks — Execução Sob Demanda

Depois que uma automação já existe no orquestrador (cadastrada no módulo **Automations**, com o `bot.py` e o `requirements.txt` enviados dentro do `.zip`), ela ainda não roda sozinha: ela apenas fica *disponível*. Para que o código realmente seja executado em uma máquina, é preciso emitir uma **ordem de execução**. No Smart Office, essa ordem tem nome: **Task**.

Uma **Task** é uma **ordem de execução única** de uma automação para um determinado momento. Ela dispara uma vez, o Runner processa o RPA e, ao terminar, a task se encerra — não volta a rodar. É o oposto de uma **Schedule** (execução recorrente), que veremos no capítulo seguinte. Sempre que você quiser rodar um RPA “agora” ou “em tal dia e hora”, de forma pontual, o instrumento correto é a Task.

Informação

Por que a Task é tão importante no modelo da LG? Porque ela é o mecanismo do **smoke test** de pós-deploy. Assim que um RPA novo é enviado ao orquestrador (upload do `.zip`), a validação obrigatória é feita disparando uma *task manual* não crítica: valida-se conectividade, acesso às credenciais e a execução do fluxo principal antes de qualquer agendamento recorrente. Task também é o caminho para reprocessar uma execução que falhou e para rodadas avulsas fora da rotina.

5.1 Task versus Schedule: escolha certa

Antes de criar qualquer coisa, decida o que você precisa. Confundir os dois gera task duplicada ou rotina que não repete.

Aspecto	Task	Schedule
Frequência	Executa uma vez e encerra	Executa de tempos em tempos (recorrente)
Gatilho	Campo Date (um momento)	String CRON (intervalo)
Uso típico	Smoke test, reprocessamento, execução avulsa	Rotina diária, semanal ou mensal
Escolha de Runner	Obrigatória	Manual (você escolhe) ou Smart (automático)
Encerramento	Some da fila após rodar	Fica ativa até ser desativada

5.2 Dois caminhos para criar uma Task

Há duas portas de entrada, e vale conhecer as duas porque poupam cliques:

1. **Pelo módulo Tasks** — acesse a aba **Tasks** no menu lateral e clique no botão roxo **New Task +** no canto inferior direito. Aqui você escolhe a automação manualmente no popup.
2. **Pela automação** — na tabela do módulo **Automations**, clique no ícone de três pontos da linha do RPA desejado e escolha **New Task**. O popup abre com aquela automação já pré-selecionada. É o atalho recomendado quando você já está olhando o RPA.

Dica

Prefira o caminho “pela automação” quando estiver validando um deploy recém-feito: você acabou de subir o `.zip`, está na linha do RPA, e o menu de três pontos leva direto ao **New Task** com o nome certo já preenchido — menos chance de disparar a task na automação errada.

5.3 Preenchendo o popup New Task

Ao acionar **New Task +**, o orquestrador abre um popup pedindo quatro informações. Preencha todos os campos antes de confirmar.

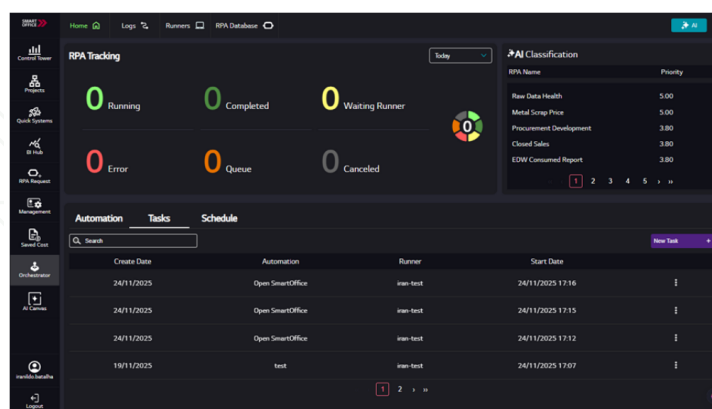


Figura 16: Popup de criação de Task, com os campos Automation, Date, Runner e Priority.

Campo	O que preencher e por quê
Automation	Selecione qual automação terá a execução. É o RPA cadastrado no módulo Automations; se não aparecer na lista, a automação ainda não foi criada.
Date	Ao clicar, abre um calendário para escolher data e hora da execução. Se você deixar em branco , o campo é preenchido automaticamente com a data e hora atuais — ou seja, a task tenta rodar imediatamente.
Runner	Selecione o Runner, isto é, em qual máquina real a automação vai rodar. É esse Runner que precisa estar iniciado para consumir a task.
Priority	Grau de prioridade da execução: 1 é o mais urgente e 5 o menos urgente . Quando um Runner tem várias tasks aguardando, ele as processa por ordem de prioridade.

Com os campos preenchidos, clique em **Create Task**. O sistema retorna uma mensagem informando se a criação foi bem-sucedida ou se houve algum problema. Em caso de sucesso, a nova task aparece na tabela da tela principal.

Informação

O que acontece nos bastidores ao criar a task: o Smart Office (o control tower) registra a ordem e a coloca na fila do Runner escolhido, associada ao ambiente virtual (**venv**) provisionado no servidor. Nada roda enquanto o Runner — a máquina real — não estiver iniciado. É o Runner que “puxa” as tasks alocadas para ele e executa o RPA dentro daquele ambiente, seguindo a ordem de prioridade.

5.4 O ciclo de vida de uma Task

Entender por quais estados a task passa evita o susto de “criei e nada aconteceu”. Uma task recém-criada fica **Waiting Runner** até que a máquina designada seja iniciada; só então entra em execução.

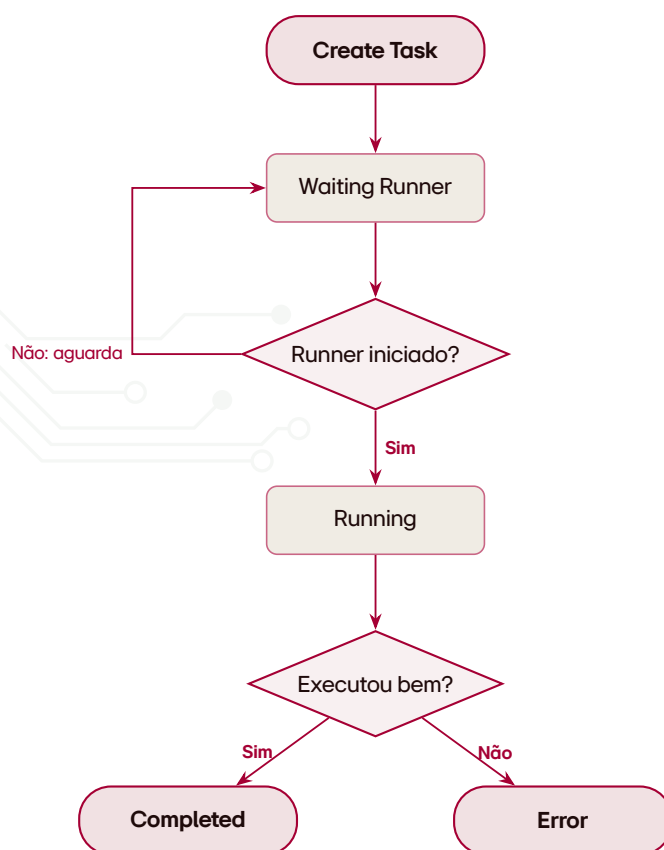


Figura 17: Ciclo de vida de uma Task (execução única)

Erro comum

Criei a task, preenchi tudo, mas ela ficou parada e o RPA nunca rodou.

Solução

Quase sempre a task está em **Waiting Runner** porque o Runner escolhido **não foi iniciado**. Vá ao cliente do Runner na máquina de destino e clique em **Start Runner**; assim que ele sobe, as tasks alocadas são enviadas e executadas por ordem de prioridade. Confirme também que o Runner selecionado no popup é mesmo o que está ligado — selecionar a máquina errada deixa a task esperando indefinidamente.

5.5 A tabela de Tasks

Na tela principal do módulo, todas as tasks criadas aparecem em uma tabela de acompanhamento.

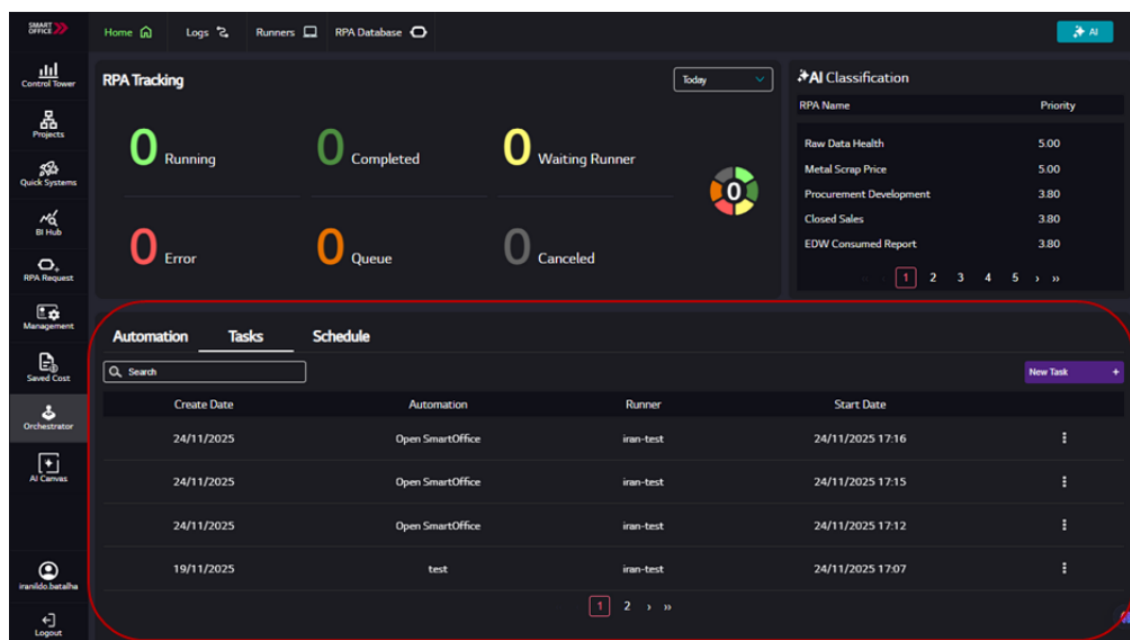


Figura 18: Tabela de Tasks, com as colunas de acompanhamento de cada ordem de execução.

Coluna	Significado
Create Date	Data em que a task foi criada (o momento em que você clicou em Create Task).
Automation	Nome da automação que será executada durante a task.
Runner	Nome do Runner (máquina) que executará a task.
Start Date	Data e hora agendadas para a execução — o que você escolheu no calendário do campo Date.

Atenção

Não confunda **Create Date** com **Start Date**. A primeira é quando a ordem *nasceu*; a segunda é quando ela *vai rodar*. Se você deixou o campo Date em branco na criação, as duas ficarão praticamente iguais, porque a task foi agendada para o instante atual.

5.6 Ações sobre uma Task

Cada linha da tabela tem um ícone de **três pontos** que abre o menu de ações da task.

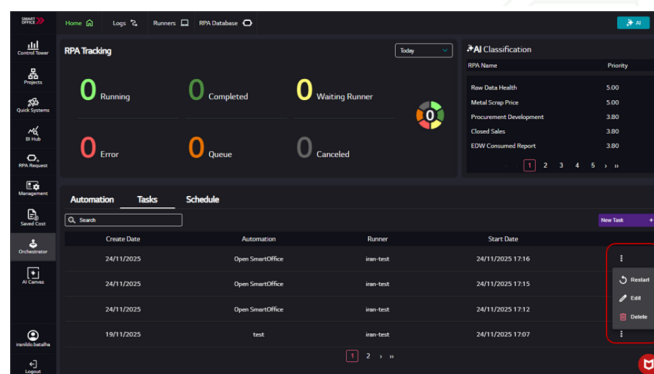


Figura 19: Menu de três pontos com as ações Restart, Edit e Delete de uma Task.

- **Restart** — abre um popup para **reiniciar** a execução da task. É aqui que você pode **escolher**

outro Runner para rodar a automação. Útil quando o Runner original está offline ou a execução falhou por um problema pontual da máquina: você redireciona a mesma task para um Runner saudável sem precisar recriá-la do zero.

- **Edit** — abre um popup para alterar os dados da task: **data e hora** da execução e o **Runner**. Há um comportamento que precisa de atenção (veja o alerta abaixo).
- **Delete** — remove a task quando ela não é mais necessária. Use para limpar ordens criadas por engano ou que ficaram obsoletas antes de rodar.

Atenção

Editar para uma data futura cria uma task NOVA. Ao usar **Edit**, se a data e hora que você informar forem **posteriores ao momento atual**, o orquestrador **não** altera a task existente: ele **cria uma nova task** com os dados editados. O resultado é que você pode acabar com duas ordens — a original e a editada. Se a intenção era só remarcar, confira a tabela depois e, se necessário, apague a task antiga com **Delete** para não executar o RPA em duplicidade.

Dica

Precisa apenas trocar a máquina de execução de uma task que ainda vai rodar? Prefira **Restart** (ou **Edit** mantendo a mesma data/hora) em vez de editar para um horário futuro. Assim você evita o efeito colateral de gerar uma segunda task.

5.7 Busca de Tasks

Acima da tabela há uma barra de busca. O critério de filtragem é o **nome da automação**: digite o nome do RPA desejado e a tabela é filtrada, mostrando apenas as tasks daquela automação. É o jeito rápido de localizar as execuções de um processo específico quando a tabela cresce.

Dica

A busca usa o nome da automação, então vale a pena manter a convenção de nomenclatura da LG (por exemplo `SCM_InquiryEndingOnhand_AUTO`). Nomes padronizados com departamento e processo tornam o filtro muito mais previsível: basta digitar o prefixo do departamento para reunir todas as tasks daquela área.

fim do capítulo ■

6 Schedules — Execução Recorrente

Nos capítulos anteriores vimos que uma **Task** representa uma única ordem de execução: você agenda um RPA para rodar em um momento específico e, terminada essa execução, a Task se encerra e não volta a rodar. Isso resolve bem os casos pontuais, mas boa parte das automações corporativas precisa rodar **de tempos em tempos** — o relatório que sai toda manhã, a conciliação que roda toda segunda-feira, o fechamento que acontece no primeiro dia de cada mês. Recriar uma Task manualmente a cada ocorrência seria trabalhoso e sujeito a esquecimento.

É exatamente esse o papel do módulo **Schedules**. Um **Schedule** é uma **execução recorrente**: em vez de disparar uma única vez, ele gera automaticamente uma nova execução a cada intervalo de tempo definido. Você configura o intervalo uma vez, e o Smart Office se encarrega de acionar o RPA repetidamente, sem intervenção humana, enquanto o agendamento permanecer ativo.

Informação

Task vs. Schedule em uma frase: a Task executa **uma vez** e morre; o Schedule executa **repetidamente**, criando novas execuções a cada intervalo. Use Task para uma reprocessagem avulsa; use Schedule para tudo que tem periodicidade fixa (diária, semanal, mensal).

Por baixo dos panos o comportamento é o mesmo dos demais disparos do orquestrador: quando o intervalo é atingido, o Smart Office coloca a automação na fila e um **Runner** (a máquina real) executa o RPA dentro da intranet da LG. O Schedule não roda nada por si só — ele apenas *dispara* a execução no intervalo certo. Sem um Runner disponível, a execução fica aguardando.

6.1 Criar Schedule — “New Schedule +”

Para criar um agendamento recorrente, acesse o módulo **Schedules** e clique no botão roxo no canto inferior direito com o texto **New Schedule +**.

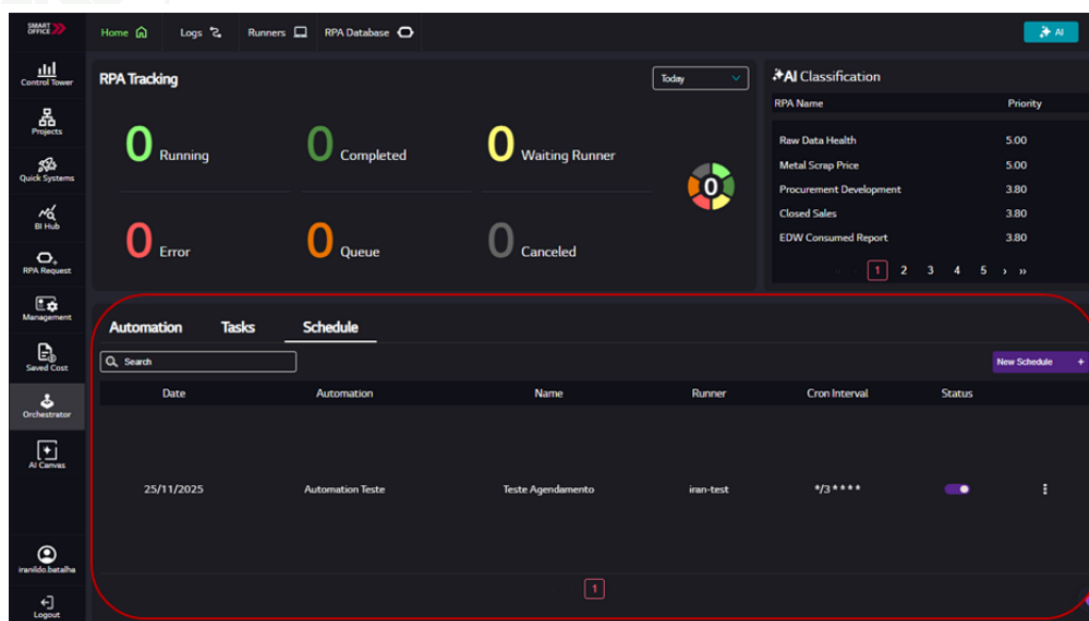


Figura 20: Tela do módulo Schedules com o botão “New Schedule +” e a tabela de agendamentos já cadastrados.

Ao clicar, o Smart Office abre um **popup** com os campos que definem o agendamento. Preencha-os

com atenção — especialmente o intervalo CRON, que é a peça central de um Schedule.

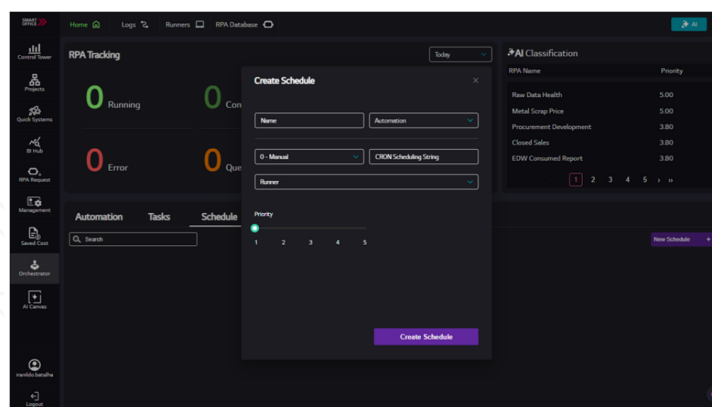


Figura 21: Popup de criação de Schedule com os campos Name, Automation, Type, Cron Scheduling String, Runner e Priority.

Os campos do popup são:

Campo	O que preencher / por quê
Name	Nome que identifica o Schedule. É por ele que você vai encontrar o agendamento na barra de pesquisa da tela. Escolha um nome descritivo — ele não é o nome da automação, e sim o rótulo deste agendamento específico.
Automation	Qual automação (RPA já cadastrado) será executada de forma recorrente. Só aparecem aqui as automações previamente criadas no módulo Automations.
Type	Define <i>como</i> o Runner é escolhido: Manual (você indica o Runner) ou Smart (o orquestrador escolhe automaticamente). Veja a distinção logo abaixo.
Cron Scheduling String	O intervalo de recorrência, obrigatoriamente no padrão CRON . Se a string não estiver nesse formato, o Smart Office recusa a criação do Schedule.
Runner	A máquina que executará o RPA. Este campo só é obrigatório — e só faz sentido — quando o Type é Manual . Com Type Smart , o Runner é resolvido automaticamente.
Priority	Grau de prioridade da execução, de 1 (mais alta) a 5 (mais baixa). Quando vários jobs disputam o mesmo Runner, os de menor número são executados primeiro.

Dica

Manual vs. Smart — qual escolher?

- **Manual:** você amarra o Schedule a um Runner específico. Use quando a automação depende daquela máquina em particular — por exemplo, porque só ela tem acesso a um sistema interno, uma VPN, um certificado ou um software instalado. Com Manual, o campo **Runner** passa a ser **obrigatório**.
- **Smart:** o orquestrador decide sozinho qual Runner disponível vai executar. Use quando qualquer máquina do pool serve, deixando o Smart Office balancear a carga. Aqui você **não precisa** informar o Runner.

Na dúvida, prefira **Smart** para automações genéricas e reserve **Manual** para os casos em que a máquina de execução realmente importa.

6.1.1 Entendendo o formato CRON

O campo **Cron Scheduling String** é o coração do Schedule, e é onde mais aparecem erros de preenchimento. O padrão **CRON** vem do mundo Unix e descreve “quando” uma tarefa deve rodar usando **cinco campos separados por espaço**, sempre nesta ordem:

Posição	Valores válidos	Significado
1 — Minuto	0–59	Minuto da hora em que dispara.
2 — Hora	0–23	Hora do dia (formato 24h).
3 — Dia do mês	1–31	Dia do mês.
4 — Mês	1–12	Mês do ano.
5 — Dia da semana	0–6	Dia da semana (0 = domingo, 1 = segunda, ..., 6 = sábado).

O caractere ***** (asterisco) significa “qualquer valor” naquele campo. Assim, ler uma string CRON é ler os cinco campos da esquerda para a direita: *no minuto X, na hora Y, nos dias/meses/dias-da-semana que casarem*. Alguns exemplos práticos:

String CRON	Quando executa
0 8 * * *	Todos os dias às 08:00 (diário).
0 8 * * 1	Toda segunda-feira às 08:00 (semanal).
0 8 1 * *	No dia 1 de cada mês às 08:00 (mensal).
30 18 * * 5	Toda sexta-feira às 18:30.
0 */2 * * *	A cada 2 horas, no minuto 0 (00:00, 02:00, 04:00...).
*/15 * * * *	A cada 15 minutos.

Atenção

A ordem dos campos é fixa: **minuto vem antes da hora**. Um erro clássico é escrever **8 0 * * *** querendo dizer “08:00” — isso agenda para **00:08** (minuto 8 da hora 0). Confira sempre os dois primeiros campos antes de salvar.

Depois de preencher todos os campos corretamente, clique no botão roxo **Create Schedule**. O Smart Office exibirá uma mensagem indicando se o agendamento foi criado com sucesso ou se houve algum erro (o motivo mais comum de erro é justamente uma string CRON malformada ou um **Type Manual** sem **Runner** informado).

Erro comum

Ao clicar em **Create Schedule**, o sistema recusa a criação e o **Schedule** não aparece na tabela.

Solução

Verifique nesta ordem: (1) a **Cron Scheduling String** tem exatamente cinco campos separados por espaço e está no padrão CRON — fora desse padrão o Smart Office não cria a **schedule**; (2) se o **Type** é **Manual**, o campo **Runner** está obrigatoriamente preenchido; (3) a automação foi selecionada em **Automation**. Corrigidos esses pontos, repita o **Create Schedule**.

6.2 A tabela de Schedules

De volta à tela inicial do módulo, todas as **schedules** cadastradas aparecem em uma tabela. Cada linha resume um agendamento recorrente:

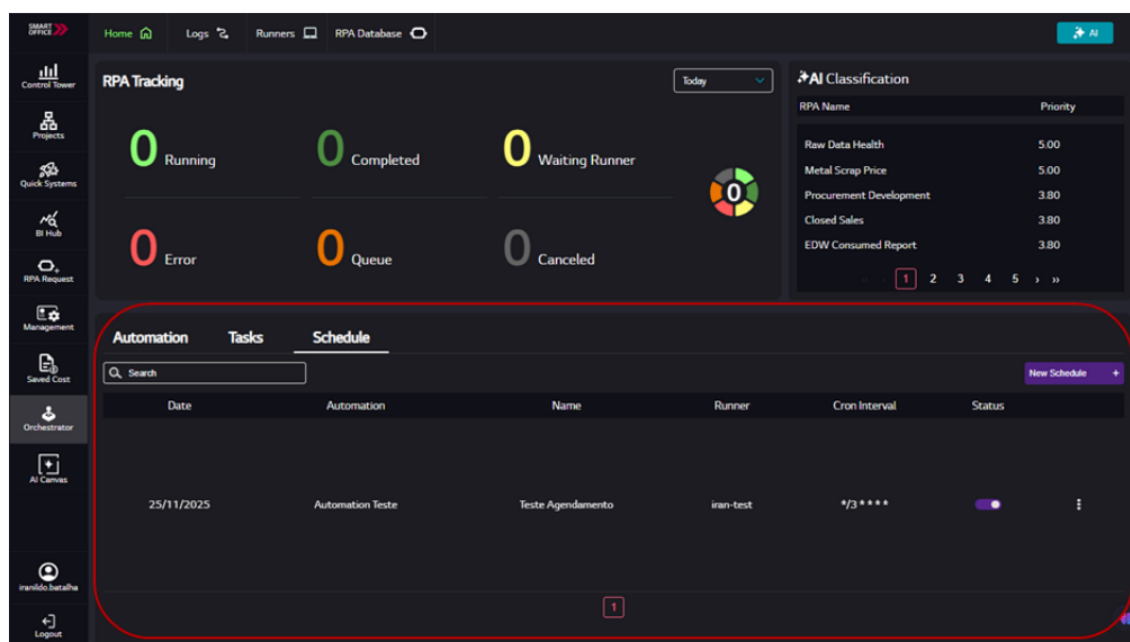


Figura 22: Tabela de Schedules com as colunas Date, Automation, Name, Runner, Cron Interval, Status e Edit.

Coluna	Descrição
Date	Data de criação da schedule.
Automation	Nome da automação que será executada por este agendamento.
Name	Nome dado à schedule no momento da criação (usado também na busca).
Runner	Nome da máquina que executará o agendamento.
Cron Interval	O intervalo de recorrência, apresentado no formato CRON.
Status	Indica se a schedule está ativo ou inativo .
Edit	Permite editar a schedule, alterando qualquer um dos campos do popup de criação.

Atenção

A coluna **Status** é decisiva: enquanto uma schedule está **inativa**, o Smart Office **não gera nenhuma execução** nos intervalos especificados — ela simplesmente não roda, mesmo com um CRON válido e um Runner definido. Se um processo recorrente “parou de rodar” sem explicação aparente, o primeiro lugar a checar é o Status desta tabela.

Dica

Precisou pausar temporariamente um processo recorrente (por exemplo, durante uma manutenção do sistema-alvo na intranet)? Em vez de deletar a schedule e ter de recriá-la depois com todo o CRON, use o **Edit** para deixá-la inativa e reative quando a manutenção terminar. Assim você preserva a configuração original.

O fluxo completo de vida de uma schedule — da criação à execução recorrente — pode ser visto assim:

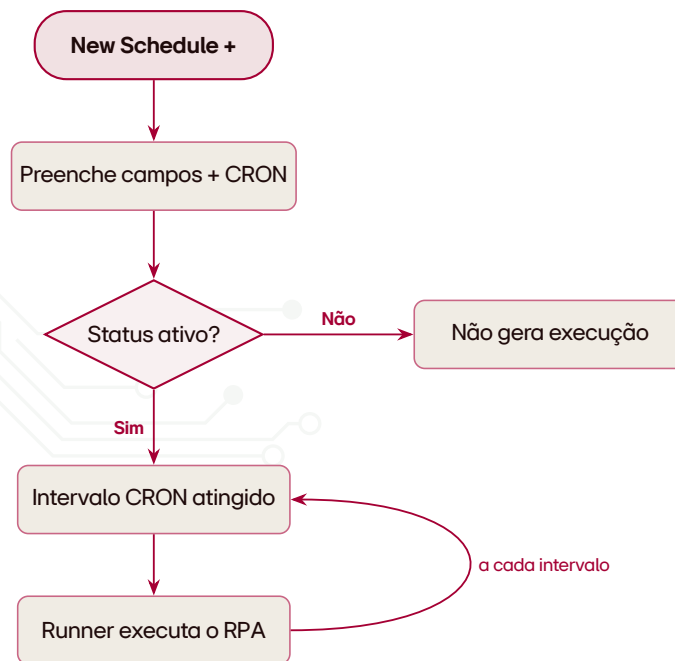


Figura 23: Fluxo de execução de uma Schedule recorrente

Note o laço à direita: diferentemente de uma Task, o Schedule **volta** ao ponto de disparo a cada intervalo, gerando uma nova execução no Runner — é essa repetição que caracteriza a execução recorrente.

Informação

Boa prática de nomenclatura. Ao nomear schedules, siga a convenção do padrão de desenvolvimento da LG: <DEPARTAMENTO>_<NOME_DO_PROCESSO>_SCHED (por exemplo, SCM_InquiryEndingOnhand_SCHED). Isso mantém os agendamentos rastreáveis e alinhados aos nomes usados no repositório GitLab e na automação correspondente.

fim do capítulo ■

7 Logs e Monitoramento

Um RPA orquestrado pelo Smart Office não roda diante dos seus olhos. Ele é executado por um **Runner** — uma máquina real — a partir do ambiente virtual provisionado no servidor pelo control tower. Como a execução é desassistida (ninguém fica olhando a tela do Runner enquanto o robô clica, digita e navega pela intranet da LG), surge uma pergunta óbvia: como saber se a automação rodou, travou ou falhou? É exatamente essa a função do módulo **Logs**.

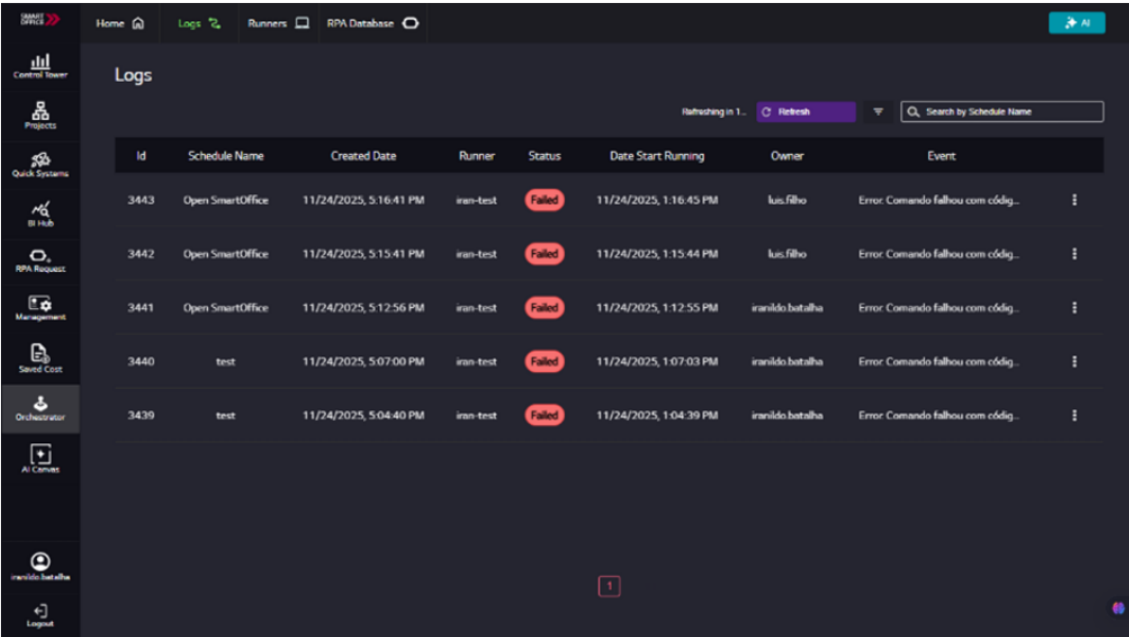
O módulo Logs é a janela de observabilidade do orquestrador. Ele reúne, em uma única tabela, o registro de **todas as execuções** disparadas na plataforma — tanto as **tasks** (execuções únicas) quanto os **schedules** (execuções recorrentes) — e permite acompanhar, em tempo real, se os RPAs estão se comportando adequadamente ou não.

Informação

Cada vez que uma **task** é disparada ou um **schedule** atinge seu horário de gatilho, o Smart Office gera uma **linha de log**. Não confunda esses estados de execução (Running, Failed, Success...) com os status de governança de uma solicitação: aqui falamos apenas do ciclo de vida operacional de uma execução no Runner.

7.1 A tabela de Logs

Ao abrir o módulo, você encontra uma tabela em que cada linha corresponde a uma execução. As colunas descrevem o que está rodando, onde e em que estado.



Id	Schedule Name	Created Date	Runner	Status	Date Start Running	Owner	Event
3443	Open SmartOffice	11/24/2025, 5:16:41 PM	iran-test	Failed	11/24/2025, 1:16:45 PM	luis.filho	Error Comando falhou com códig...
3442	Open SmartOffice	11/24/2025, 5:15:41 PM	iran-test	Failed	11/24/2025, 1:15:44 PM	luis.filho	Error Comando falhou com códig...
3441	Open SmartOffice	11/24/2025, 5:12:56 PM	iran-test	Failed	11/24/2025, 1:12:55 PM	iranildo batalha	Error Comando falhou com códig...
3440	test	11/24/2025, 5:07:00 PM	iran-test	Failed	11/24/2025, 1:07:03 PM	iranildo batalha	Error Comando falhou com códig...
3439	test	11/24/2025, 5:04:40 PM	iran-test	Failed	11/24/2025, 1:04:39 PM	iranildo batalha	Error Comando falhou com códig...

Figura 24: Módulo Logs: cada linha é uma execução (task ou schedule), com seu Runner, horário e status em tempo real.

Coluna	O que informa
Id	Identificador único do log gerado pela execução. Serve para localizar rapidamente uma task ou schedule específica e para referenciá-la em um chamado de suporte.
Schedule Name	Nome dado à task ou ao schedule no momento da criação. É também o campo pelo qual você faz a busca na tela.
Created Date	Data e hora em que a task/schedule foi <i>criada</i> (não confundir com o horário de início da execução).
Runner	Nome do Runner (a máquina real) designado para executar a automação.
Status	Estado atual da execução. Assume um dos seis valores detalhados na Seção seguinte.
Date Start Running	Data e hora em que a execução foi ou será iniciada pelo Runner.
Owner	Nome do usuário que criou a task/schedule — útil para saber a quem recorrer.
Event	Mensagem curta resumindo o que ocorreu ao término da execução. É a primeira pista de diagnóstico; o log completo fica na ação Information .

7.2 Os seis estados de execução (Status)

A coluna **Status** é o coração do monitoramento. Ela conta, sem que você precise abrir nada, em que ponto do ciclo de vida cada execução se encontra. Entender cada estado — e a ação correta para ele — evita que você espere por um resultado que nunca virá ou que interprete um simples enfileiramento como uma falha.

Status	Significado	Ação recomendada
Waiting Runner	A execução foi criada, mas ainda não há um Runner disponível para assumi-la. Em geral o Runner está desligado, ocupado ou não foi iniciado.	Verifique se o Runner está online e se alguém clicou em Start Runner na máquina. Se persistir, confira a designação do Runner.
In Queue	A execução já foi entregue ao Runner e aguarda na fila. O Runner processa por ordem de <i>prioridade</i> e chegada.	Aguarde. Se for urgente, confira a prioridade (1 = mais alta) e o volume da fila.
Running	A automação está executando <i>neste momento</i> no Runner.	Acompanhe por Information . Use Cancel Task apenas se detectar que travou ou saiu do esperado.
Canceled	A execução foi interrompida — normalmente por um usuário via Cancel Task .	Se o cancelamento foi engano, recrie a execução com uma nova task.
Failed	A automação terminou com erro. Algo lançou uma exceção não tratada ou o ambiente falhou.	Abra Information , leia o Event e o <i>stack trace</i> , diagnostique e corrija (ver Seção de diagnóstico).
Success	A execução terminou com êxito, do início ao fim.	Nenhuma. O resultado está pronto; se desejar, valide as evidências geradas pelo RPA.

Dica

Ao acompanhar uma janela de execução, leia a tabela pela severidade dos estados: **Waiting Runner** e **In Queue** são transitórios e esperados; **Running** pede atenção; e apenas **Failed** exige ação imediata. Isso evita alarme falso quando várias execuções estão simplesmente enfileiradas.

7.3 Ações disponíveis em cada log

Além de observar, cada linha da tabela oferece duas ações operacionais:

- **Information** — abre os *detalhes* da execução, incluindo os **logs gerados pela própria automação**. É aqui que aparecem as mensagens que o seu RPA registrou (o que ele imprimiu, os avisos, e o *stack trace* de qualquer erro). É a ferramenta primária de diagnóstico.

- **Cancel Task** — **interrompe** a execução de uma automação. Vale destacar: mesmo que a automação esteja *rodando neste exato momento*, é possível abortá-la em tempo real. Use com critério — um cancelamento no meio de uma escrita em planilha ou sistema pode deixar o processo em estado inconsistente.

Atenção

Cancel Task não faz “desfazer”. Ele interrompe o RPA no ponto em que ele está — não reverte cliques, gravações ou envios que a automação já executou na intranet. Antes de cancelar uma execução **Running**, avalie o que o robô já pode ter feito.

7.4 Diagnosticando uma falha (Status = Failed)

O fluxo de investigação de uma falha no Smart Office é sempre o mesmo e começa pela tabela de Logs. A regra de ouro: **Failed nunca é o fim da investigação — é o começo**. O **Event** dá a manchete; o **Information** dá a história completa.

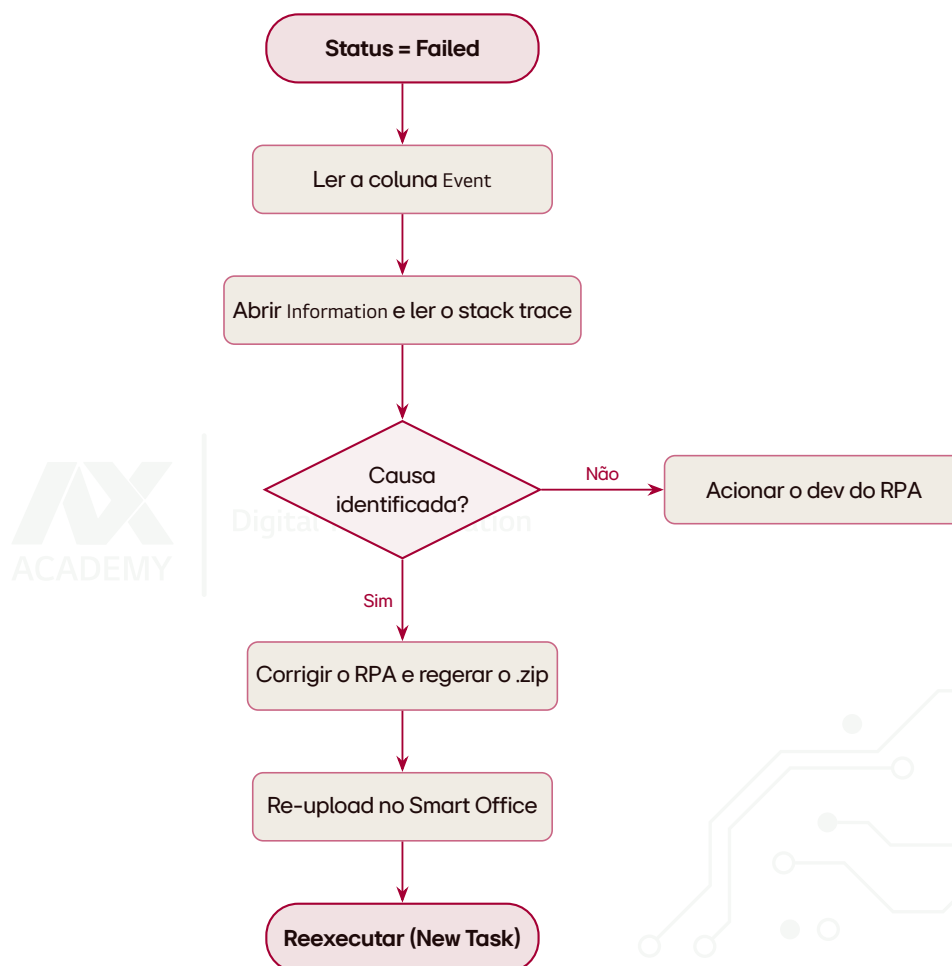


Figura 25: Fluxo de diagnóstico de uma falha (Status = Failed)

Como o Smart Office é **agnóstico** — ele orquestra, mas não escreve o RPA por você — a maioria das falhas se enquadra em três famílias. Reconhecê-las acelera o diagnóstico.

Erro comum

Information mostra **ModuleNotFoundError** ou **ImportError** logo no início da execução.

Solução

Falta uma dependência no ambiente virtual. Lembre que o deploy no Smart Office é o **upload de um .zip** contendo `bot.py` e `requirements.txt`; se a biblioteca não estiver listada, o `venv` provisionado não a terá. Adicione a dependência ao `requirements.txt`, regenere o .zip e faça o novo upload.

Erro comum

A linha fica presa em `Waiting Runner` e nunca sai desse estado.

Solução

Não é falha do código: é falta de Runner. A máquina designada está desligada ou o Runner Cliente não foi iniciado. Confira no módulo de Runners se ele está ativo e clique em `Start Runner` na máquina executora. Só então a execução avança para `In Queue`.

Erro comum

`Failed` com erro de autenticação/credencial ou de elemento não encontrado (ex.: `NoSuchElementException`).

Solução

Credencial expirada, tela do sistema-alvo mudou, ou espera insuficiente. Nunca “chumbe” senha no código — use o cofre de credenciais. Para telas, prefira esperas explícitas (`wait_for`) a `sleep` fixo, e seletores estáveis (Id, Name) a XPath frágil. Corrija no RPA, regere o .zip e reenvie.

7.5 Busca e atualização da tela

Em uma operação com dezenas de automações, encontrar a execução certa rápido é essencial. A tela de Logs oferece dois recursos para isso:

- **Busca por Schedule Name** — a barra de pesquisa acima da tabela filtra as execuções pelo nome da task/schedule. Como o nome segue a nomenclatura padrão (por exemplo, `SCM_InquiryEndingOnhand_SCHE`) buscar pelo departamento ou pelo nome do processo já reduz a lista ao que interessa.
- **Atualização da tabela** — para não depender de você recarregar a página manualmente, a tela se atualiza de duas formas: **manualmente**, ao clicar no botão de atualização, ou **automaticamente a cada 10 segundos**.

Dica

Ao monitorar uma janela crítica de execução (por exemplo, o fechamento de um relatório diário), deixe a tabela se atualizando sozinha a cada 10 segundos e mantenha a busca filtrada pelo `Schedule Name` do processo. Você vê o estado migrar de `In Queue` para `Running` e para `Success` sem tocar em nada.

Informação

O acompanhamento em tempo real do módulo Logs é o que alimenta a visão de **control tower** da operação: é a partir desses estados que a plataforma mede a saúde do parque de automações — meta de taxa de execução acima de **85%** — e sinaliza a classificação operacional (verde / amarelo / vermelho). Monitorar não é só reagir a falhas: é garantir que a operação

inteira permaneça no verde.

7.6 Logs só ajudam se o RPA os escrever bem

Há um detalhe que decide se o módulo Logs será útil ou frustrante: **a tela Information mostra apenas aquilo que o seu RPA registrou**. Se o robô não escreve nada, um **Failed** vira um beco sem saída — você sabe que falhou, mas não onde nem por quê. Por isso, a qualidade do monitoramento começa no desenvolvimento, com **logs estruturados** (assunto tratado em detalhe no capítulo de Boas Práticas de Desenvolvimento).

As diretrizes que tornam o **Information** realmente diagnosticável:

- Use os **níveis de log** corretos: **DEBUG**, **INFO**, **WARNING**, **ERROR**, **CRITICAL**.
- Registre sempre **a etapa** do processo, para localizar onde a falha ocorreu.
- Em erros, inclua o **stack trace** (`exc_info=True`), o timestamp e uma mensagem clara.
- Prefira um formato **estruturado** (chave=valor ou JSON), que facilita a leitura e a busca.

Log estruturado com contexto e stack trace

```

1 import logging
2 from botcity.web import WebBot
3
4 logging.basicConfig(
5     level=logging.INFO,
6     format="%(asctime)s | %(levelname)s | %(message)s",
7 )
8 log = logging.getLogger("SCM_InquiryEndingOnhand_BOT")
9
10 class Bot(WebBot):
11     def action(self, execution=None):
12         try:
13             log.info("etapa=login sistema=EP status=iniciando")
14             # ... automacao ...
15             log.info("etapa=extracao linhas=128 status=ok")
16         except Exception as e:
17             # exc_info=True imprime o stack trace no painel Information
18             log.error("etapa=extracao erro=%s", e, exc_info=True)
19             raise # relanca para que o Status vire Failed, e nao Success silencioso

```

Atenção

Nunca capture uma exceção, registre e siga adiante sem **raise**. Se o erro for engolido, a execução termina como **Success** mesmo tendo falhado — e o monitoramento passa a mentir. Uma falha honesta em **Failed**, com stack trace no **Information**, vale mais do que um **Success** enganoso.

O ciclo completo se fecha assim: um RPA bem instrumentado escreve logs estruturados; o Smart Office os expõe em tempo real na tabela de Logs; e você, pela combinação de **Status**, **Event** e **Information**, diagnostica, corrige e reimplanta — mantendo a operação previsível, auditável e sustentável.

fim do capítulo ■

8 Runners e RPA Database

Nos capítulos anteriores tratamos das entidades que *descrevem* o trabalho: a Automation cadastra o RPA, a Task dispara uma execução única e o Schedule cria execuções recorrentes. Nada disso, porém, roda no vácuo. No fim da cadeia existe uma **máquina real** que abre o navegador, digita nos campos e move o mouse dentro da intranet da LG. Essa máquina é o **Runner**. Este capítulo cobre os três módulos que dão visibilidade sobre a frota de execução e sobre o código publicado: **Runners Management** (inventário dos Runners), **View Runner** (saúde e capacidade de um Runner) e **RPA Database** (o repositório de código-fonte dos RPAs, integrado ao DX Lake).

Informação

Retomando a arquitetura dos capítulos iniciais: o Smart Office é um **control tower** que vive em um servidor e provisiona um ambiente virtual (venv). Ele não executa o RPA por conta própria — ele *aloca* a execução para um Runner. O Runner é uma máquina física com recursos finitos (núcleos de CPU, memória RAM, disco e versões de software instaladas). Entender esses limites é o que separa uma alocação bem-feita de uma task que trava na fila ou de um navegador que morre por falta de memória.

O diagrama abaixo situa os três módulos deste capítulo dentro do caminho de execução.

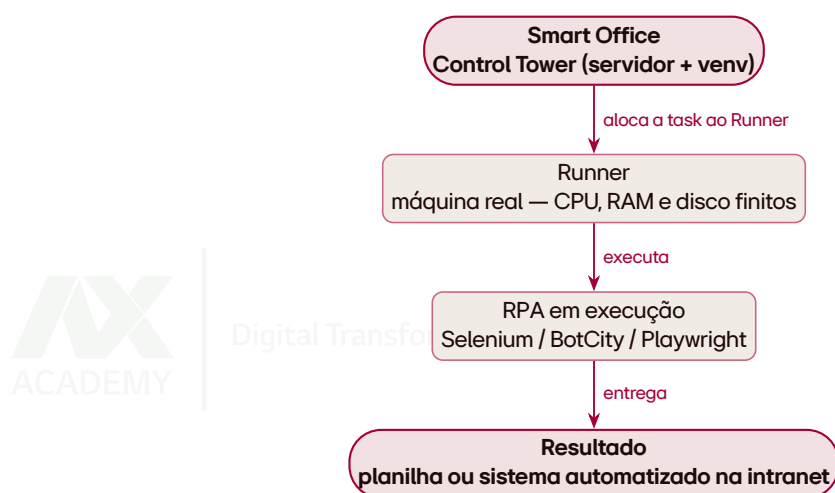


Figura 26: Do orquestrador ao resultado: o papel do Runner na cadeia de execução

8.1 Runners Management

O módulo **Runners Management** apresenta todos os Runners cadastrados na plataforma, ou seja, todas as máquinas disponíveis para executar automações. É a sua visão de inventário: antes de mandar uma task para um Runner ou de amarrar um Schedule a uma máquina, é aqui que você confere quais Runners existem e se estão vivos.

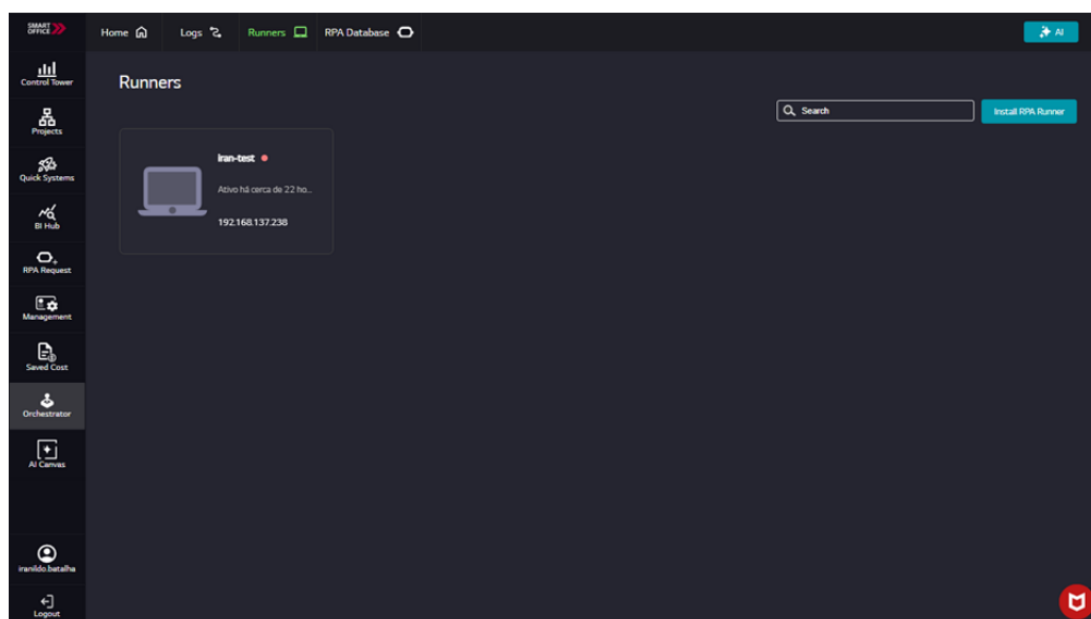


Figura 27: Tela de Runners Management: lista dos Runners cadastrados com nome, tempo desde a última ativação e endereço IP, além da barra de busca por nome.

Para cada Runner a lista traz as seguintes informações:

Campo	Descrição e por que observar
Nome do Runner	Identificador da máquina. Segue as convenções do padrão de nomenclatura: numérico (<i>dx-virtual-machine1</i>) ou por área (<i>scm-runner</i> , <i>fin-runner</i>). O nome é o que você seleciona depois no campo "Runner" ao criar Task ou Schedule.
Última ativação	Tempo decorrido desde a última vez que o Runner esteve ativo. É o sinal de vida da máquina: um Runner sem ativação recente provavelmente está parado, desligado ou com o cliente Runner fechado — e não vai puxar tasks da fila.
IP	Endereço do Runner na rede da LG. Serve para identificar fisicamente a máquina e para acesso remoto quando é preciso investigar uma execução.

A **barra de busca** filtra a lista pelo nome do Runner. Em operações com muitas máquinas — por exemplo, uma frota agrupada por departamento — a busca é o caminho rápido para localizar o Runner de um projeto específico.

Atenção

O campo de última ativação é o seu primeiro diagnóstico quando uma task não sai do estado *Waiting Runner*. Se você agendou uma execução para um Runner que está parado há muito tempo, a task fica esperando indefinidamente: enquanto o Runner escolhido não for iniciado, nada é executado. Confira aqui *antes* de suspeitar do código do RPA.

8.2 View Runner

Enquanto o Runners Management mostra a frota inteira de forma resumida, o **View Runner** abre a ficha completa de um Runner específico. É a tela que você usa para **dimensionar a execução**: decidir se aquela máquina aguenta o RPA que você pretende alocar nela.

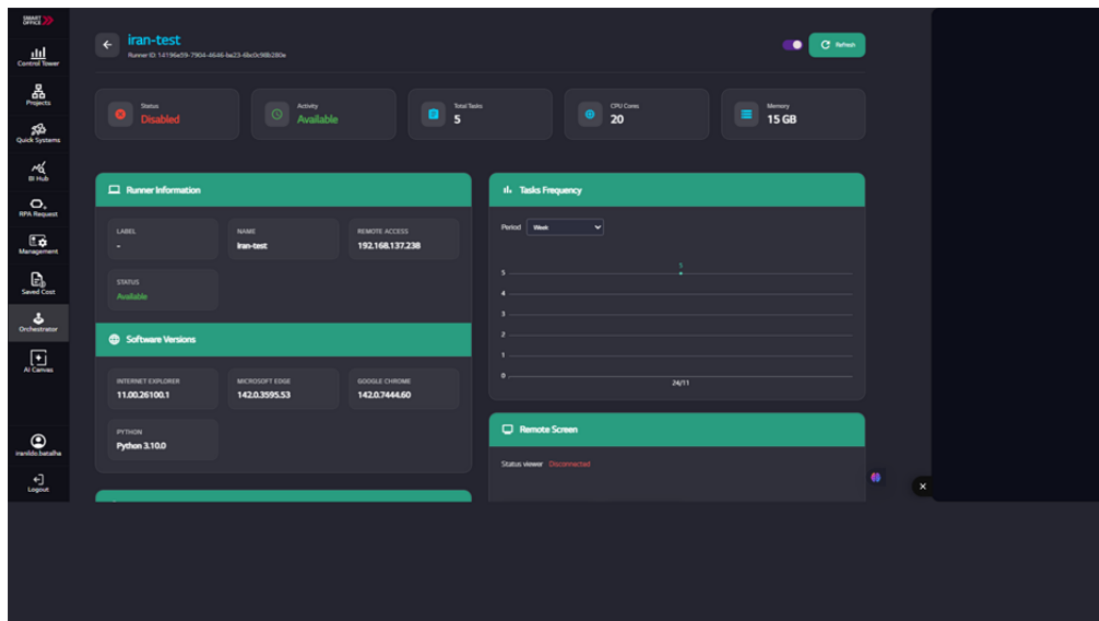


Figura 28: View Runner: visão detalhada de um Runner com status, atividade, total de tasks, recursos de hardware, informações de identificação, versões de software e acesso remoto à tela.

Os indicadores agrupam-se em três blocos de decisão: *estado operacional* (o Runner está pronto e o que já processou), *capacidade* (quanto hardware ele tem) e *compatibilidade* (quais versões de software ele oferece). A tabela a seguir explica cada um e por que ele pesa na hora de alocar uma execução.



Digital Transformation

Indicador	O que mostra	Por que importa para dimensionar
Status	Estado operacional do Runner: "Available" (disponível, pronto para receber e executar tasks) ou "Disabled" (indisponível/parado).	Alocar uma task a um Runner "Disabled" faz a execução travar na fila. Só um Runner "Available" consome o que foi agendado para ele.
Activity	O que o Runner está fazendo agora — executando uma automação ou ocioso.	Evita empilhar trabalho pesado em uma máquina que já está ocupada, o que serializaria as execuções e atrasaria tudo.
Total Tasks	Quantidade total de tasks já processadas pelo Runner.	Dá noção da carga histórica da máquina e ajuda a balancear a distribuição de automações entre vários Runners.
CPU Cores	Número de núcleos de processamento disponíveis.	RPAs que abrem navegador e processam dados em paralelo competem por núcleos; poucos núcleos significam concorrência e lentidão em execuções simultâneas.
Memory	Memória RAM disponível na máquina.	Navegadores controlados por Selenium, Playwright ou BotCity consomem muita RAM. Memória insuficiente derruba o navegador no meio da execução e gera falha.
Runner Information	Dados de identificação: Label, Name e Remote Access/IP.	Label e Name localizam a máquina certa; o Remote Access/IP permite acesso remoto para conferir se é o host físico esperado dentro da intranet.
Software Versions	Versões instaladas dos navegadores (Chrome, Edge, Firefox) e do Python.	O webdriver do RPA precisa casar com a versão do navegador do Runner, e o Python precisa ser compatível com o <code>requirements.txt</code> . Divergência aqui é causa clássica de falha no deploy e na execução.
Tasks Frequency	Histórico/frequência de execuções ao longo do tempo.	Revela janelas de pico da máquina, ajudando a planejar horários de Schedule que não colidam com outras cargas.
Remote Screen	Visualização remota da tela do Runner.	Permite depurar visualmente o que o RPA está fazendo na máquina real, sem precisar estar fisicamente diante dela.

Dica

Trate o View Runner como um checklist antes de alocar uma execução crítica: (1) Status em "Available"; (2) CPU e memória compatíveis com o peso do RPA; (3) versão de navegador e de Python alinhadas ao que o projeto declara no README. Essa conferência de trinta segundos evita a maioria das falhas de ambiente em produção.

Erro comum

Um RPA que rodava perfeitamente na máquina do desenvolvedor falha logo na abertura do navegador quando alocado a um Runner, com erro de webdriver incompatível.

Solução

Abra o View Runner do Runner alvo e confira o bloco **Software Versions**. O README do projeto declara a versão de navegador e do webdriver exigidos; se o Chrome do Runner for mais novo (ou mais antigo) que o webdriver empacotado no RPA, a sessão nem inicia. Alinhe a versão do

driver ao navegador do Runner — ou aloque a task a um Runner cuja versão de navegador seja compatível.

Erro comum

Uma automação pesada fica lenta, congela ou é encerrada de forma inesperada quando executada junto de outras no mesmo Runner.

Solução

Verifique **Memory** e **Activity** no View Runner. Se a máquina já está executando outra automação com navegador aberto, a RAM disponível pode não bastar para uma segunda sessão. Distribua as execuções em Runners diferentes ou ajuste o Schedule para que as cargas pesadas não se sobreponham na mesma janela.

8.3 RPA Database

O módulo **RPA Database** apresenta as automações e os seus respectivos arquivos de execução de forma direta, permitindo baixar o **código-fonte** de cada RPA. Isso é possível graças à integração com o **DX Lake**, o repositório que armazena os códigos-fonte dos RPAs publicados na plataforma. Em outras palavras, é a memória de código do orquestrador: o que foi efetivamente deployado fica guardado e recuperável aqui.

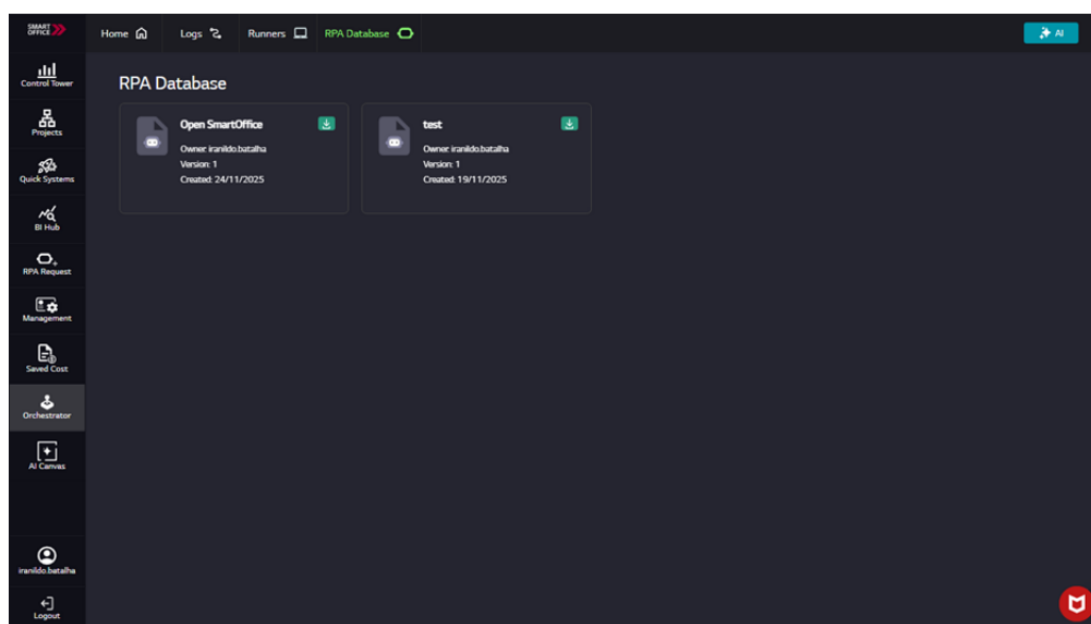


Figura 29: RPA Database: listagem das automações com nome, owner, versão e data de criação, e o ícone verde de download do código-fonte em .zip.

Para cada RPA, a tabela exibe as informações que permitem identificar e auditar a versão publicada:

Coluna	Descrição
Nome do RPA	Nome da automação, o mesmo identificador usado nos módulos de Automation e Tasks.
Owner	Quem criou/é responsável pela automação — a pessoa selecionada como dona no cadastro do RPA.
Version	Versão do código, calculada com base no número de uploads do arquivo da automação. Cada novo deploy do <code>bot.py</code> incrementa a versão.
Created	Data de criação/registro da automação.

O download é feito pelo **ícone verde de download**: ao clicar nele, o código-fonte do RPA é baixado como um arquivo `.zip`. Esse pacote é o mesmo artefato que foi publicado no orquestrador — com `bot.py` e `requirements.txt` na raiz, exatamente como a estrutura de deploy exige. Por isso o RPA Database serve a três usos práticos:

- **Auditoria**: conferir qual código está de fato em produção, associado a qual owner e a qual versão.
- **Recuperação**: baixar novamente a fonte de uma automação para inspecionar, corrigir ou reimplantar, sem depender apenas da máquina de quem desenvolveu.
- **Portabilidade**: obter o pacote pronto para rodar localmente ou para preparar uma nova versão.

Informação

O DX Lake é a fonte de armazenamento por trás do RPA Database. O GitLab continua sendo o versionamento oficial durante o desenvolvimento (branches, Merge Request, tags), enquanto o RPA Database reflete o que foi *publicado* no orquestrador via upload do `.zip`. São camadas complementares: o GitLab cobre a revisão do código; o RPA Database dá acesso rápido à fonte da versão que está de fato executando. Note que a publicação no orquestrador é sempre o **upload manual do zip** — o GitLab garante rastreabilidade e revisão, mas não empurra o bot automaticamente para o Smart Office.

Dica

Quando precisar diagnosticar um comportamento estranho em produção, baixe a versão exata pelo RPA Database em vez de assumir que o repositório local está idêntico ao que roda no Runner. A coluna “Version” confirma se o código que você está olhando é o mesmo que o orquestrador está executando.

Em conjunto, esses três módulos fecham a visão operacional do orquestrador: o Runners Management diz *onde* há máquina disponível, o View Runner diz *se aquela máquina aguenta* o trabalho e o RPA Database diz *qual código* está publicado nela. Como o Runner é uma máquina real de recursos finitos — e não um recurso infinito na nuvem —, dimensionar a execução com base nesses indicadores é parte essencial do modelo de operação esperado pela LG.

fim do capítulo ■

9 O Runner Client

Até aqui vimos como cadastrar automações, agendar *Tasks* e *Schedules* e acompanhar tudo pelos *Logs*. Mas há uma pergunta que ainda não foi respondida: *onde*, de fato, o RPA roda? O orquestrador é o cérebro da operação — ele guarda o código, decide o que executar e quando — porém ele não é a mão que aperta os botões nos sistemas da LG. Quem faz esse trabalho braçal é o **Runner**, e é dele que trata este capítulo.

9.1 O que é o Runner e onde ele se encaixa

O **Runner** é o componente responsável por executar os RPAs em uma **máquina real** — um computador dentro da intranet da LG. Ele é, em essência, um trabalhador que fica *aguardando* na escuta: assim que existe uma automação alocada para ele na fila, ele a puxa e executa. Enquanto não há trabalho, ele apenas espera.

É importante separar dois conceitos que compartilham o mesmo nome:

Conceito	O que é
Runner (registro)	O <i>cadastro lógico</i> dentro do orquestrador. É um nome que aparece em <i>Runners Management</i> e que fica disponível para seleção ao criar uma Task ou um Schedule. Sozinho, ele não executa nada.
Runner Client	A <i>aplicação instalada na máquina real</i> . É ela que dá vida ao registro: conecta a máquina ao orquestrador, recebe as automações alocadas e as executa localmente.

Ou seja, o **Runner Client é a ponte** entre o registro abstrato no orquestrador e o hardware que efetivamente roda o robô. Sem o cliente instalado e iniciado em uma máquina, o registro é apenas um nome vazio, e qualquer Task apontada para ele fica parada aguardando.

Informação

Vale relembrar a divisão de papéis do Smart Office. O orquestrador (o servidor) prepara o ambiente de execução — provisiona o ambiente virtual (**venv**) e as dependências declaradas no **requirements.txt** do RPA — e *aciona* o Runner. O Runner Client, na máquina real, é quem de fato abre o navegador, digita nos campos e automatiza a planilha ou o sistema-alvo, usando o framework que o RPA adotou (BotCity, Selenium ou Playwright). O Smart Office é agnóstico: ele orquestra qualquer um deles.

O ciclo de vida completo, do orquestrador até a execução na máquina, pode ser visualizado assim:

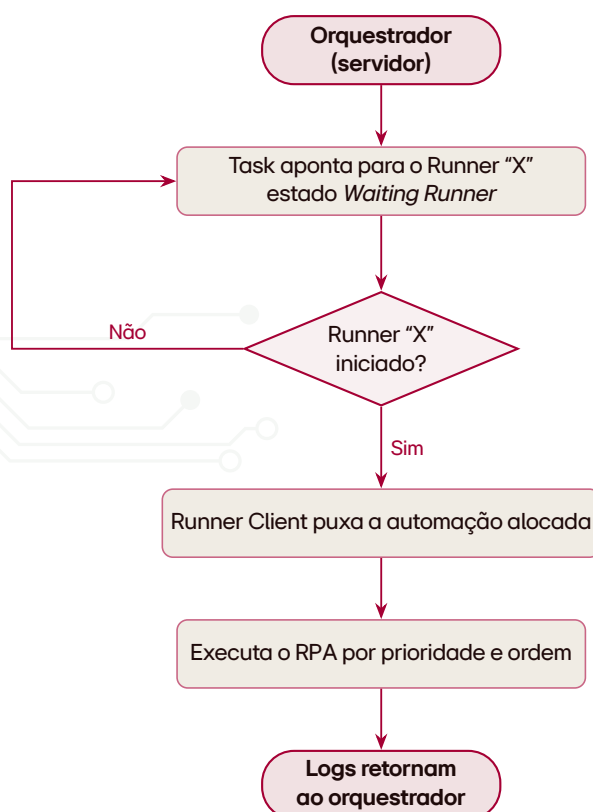


Figura 30: O Runner e sua posição na cadeia de execução

9.2 Instalando o Runner Client

Para transformar uma máquina em Runner, é preciso instalar nela o **Runner Client**. O instalador não é procurado em um site externo: ele é baixado a partir do próprio orquestrador, garantindo que a versão do cliente esteja alinhada à do Smart Office.

O passo a passo é direto:

1. Acesse o Smart Orchestrator e abra o menu **Runners**.
2. Clique no botão azul "Install RPA Runner" para baixar o instalador.
3. Execute o instalador *na máquina que você deseja transformar em Runner* — não na sua estação de trabalho de desenvolvimento, e sim naquela que deve ficar dedicada a executar as automações.

Atenção

A escolha da máquina não é um detalhe. Como o RPA vai automatizar sistemas e planilhas *dentro da intranet da LG*, a máquina onde o Runner Client roda precisa ter acesso de rede aos sistemas-alvo e às credenciais necessárias. Além disso, ela precisa permanecer **ligada e com o Runner iniciado** nos horários agendados — caso contrário, as Tasks e Schedules ficarão acumuladas na fila, esperando alguém que nunca chega.

9.3 Login na aplicação

Concluída a instalação, ao abrir a aplicação pela primeira vez é apresentada uma **tela inicial de login**. Aqui mora um detalhe que costuma gerar confusão: o login *não* usa o e-mail corporativo com que você acessa o portal Smart Office no navegador. A autenticação é feita com o **usuário e a senha**

do próprio computador onde o cliente está instalado.

Erro comum

O usuário tenta entrar no Runner Client com o e-mail e a senha corporativos da LGE (os mesmos do portal web) e o login é recusado.

Solução

Use as credenciais *da máquina* — o mesmo usuário e senha com que você faz logon no sistema operacional daquele computador. O Runner Client é uma aplicação local e se ancora na identidade da máquina, não na sessão web do orquestrador.

Após o login bem-sucedido, o cliente exibe uma tela de **control tower local**, que reúne os principais indicadores encontrados no Smart Office. Na prática, é um espelho enxuto do painel do orquestrador, agora sob a ótica daquela máquina específica — prático para verificar, sem sair do computador, se está tudo em ordem.

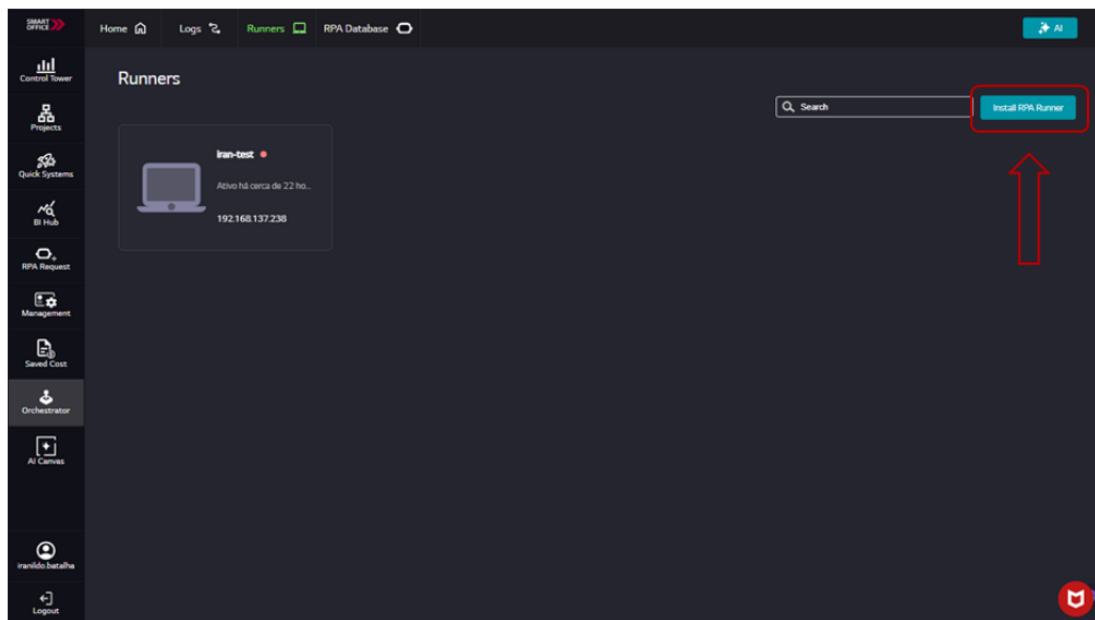


Figura 31: Tela inicial do Runner Client: um control tower local com os principais indicadores do Smart Office.

9.4 Funcionalidades e menu lateral

A navegação do Runner Client é feita por uma **barra de menu lateral**, que concentra as principais funcionalidades da aplicação. Os itens seguem a ordem natural de uso: primeiro o painel de indicadores (o control tower que acabamos de ver), depois a criação do Runner, em seguida a tela de acompanhamento da execução e, por fim, a tela de configuração de hosts. O logout fica isolado no canto inferior.



Figura 32: Menu lateral do Runner Client, com as funcionalidades disponíveis na aplicação.

Percorreremos cada uma dessas telas nas próximas seções, na mesma sequência em que aparecem no menu.

9.5 Criar o Runner na máquina

Acessando o **segundo item** do menu lateral, chega-se à tela de configuração e criação do Runner. É aqui que a máquina ganha uma identidade dentro do orquestrador.

O procedimento tem poucos passos:

1. Preencha o campo com o **nome do Runner**.
2. Clique no botão roxo “Create Runner”.
3. Os dados do Runner recém-criado são exibidos na tela.
4. Se precisar ajustar alguma informação, clique no **ícone de lápis** para editar.

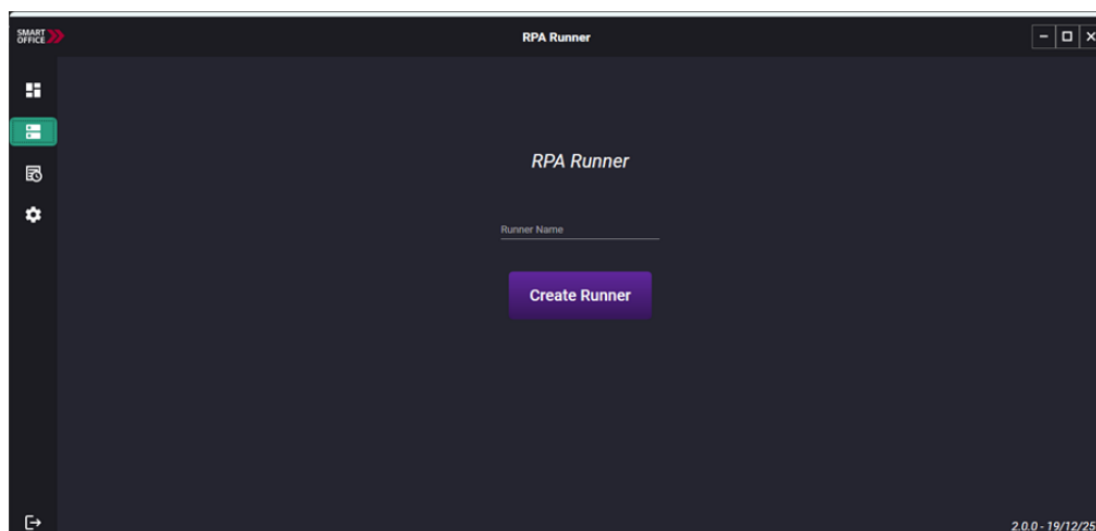


Figura 33: Tela de criação do Runner: informe o nome e clique em “Create Runner”.

O nome escolhido aqui é o que **vincula esta máquina** ao registro que aparecerá na lista de Runners do orquestrador — e, consequentemente, ao Runner que os desenvolvedores selecionam ao montar uma Task ou um Schedule. Por isso, ele não deve ser inventado ao acaso.

Dica

Siga a convenção de nomenclatura do padrão do Smart Office. Runners costumam adotar um de dois formatos, sempre em *kebab-case* (minúsculas e hífens):

- Numérico (padrão DX): `dx-virtual-machine1`, `dx-virtual-machine2...`
- Por projeto/departamento: `scm-runner`, `vtc-runner`, `fin-runner`.

Quando o volume de automações de uma área for alto, agrupe os Runners por departamento e mantenha registrado quem é o *owner* técnico de cada máquina.

Atenção

O nome cadastrado no cliente e o Runner selecionado nas Tasks/Schedules do orquestrador precisam ser **o mesmo**. É esse casamento de nomes que faz a automação chegar à máquina certa. Um nome trocado significa uma Task que nunca sai do estado *Waiting Runner*, porque está esperando por um Runner que, naquela máquina, não existe.

9.6 Acompanhar e iniciar a execução

O **terceiro ícone** do menu abre a tela de acompanhamento — o posto de comando do dia a dia. Ela é dividida em **dois espaços**: um mostra as *automações executadas* e o outro exibe os *logs* da automação em tempo real, permitindo verificar rapidamente o que está acontecendo naquela máquina.

Para colocar o Runner em operação, clique no botão roxo **“Start Runner”**. A partir desse momento, todas as automações alocadas para aquele Runner são enviadas a ele e passam a ser executadas — cada uma respeitando a **ordem de prioridade e de execução**.

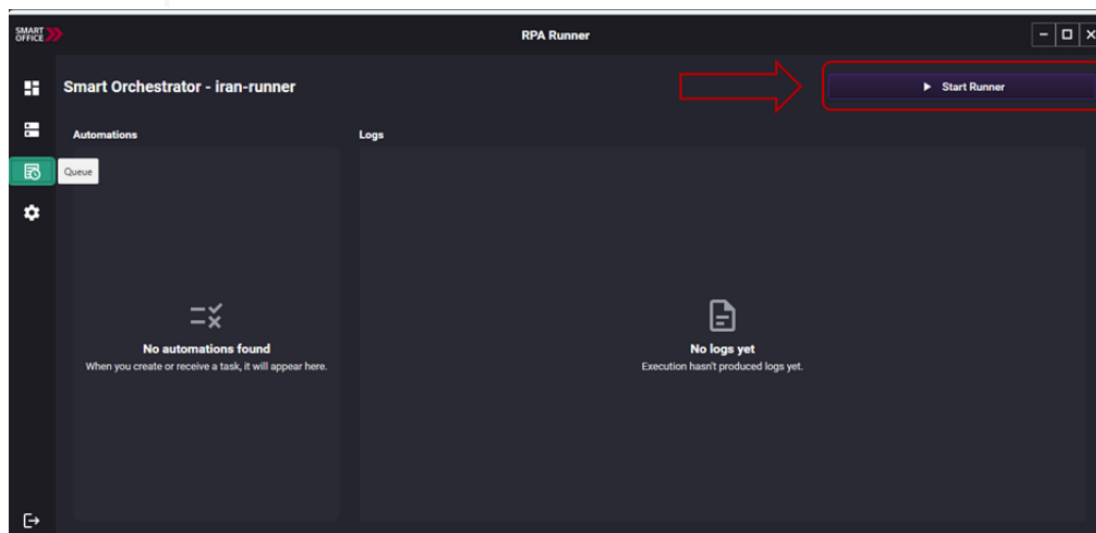


Figura 34: Tela de acompanhamento: automações executadas, logs em tempo real e o botão “Start Runner”.

A ordem importa. Ao criar uma Task ou Schedule, define-se uma *prioridade* de 1 (mais urgente) a 5 (menos urgente). Quando o Runner é iniciado e encontra várias automações na fila, ele as processa respeitando essa prioridade e, dentro do mesmo nível, a ordem de chegada. Assim, um fechamento crítico com prioridade 1 não fica atrás de um relatório rotineiro com prioridade 5.

Erro comum

Uma Task foi agendada corretamente no orquestrador, mas permanece indefinidamente no estado *Waiting Runner* e nunca executa.

Solução

Esse estado significa, quase sempre, que o Runner escolhido para a Task *não foi iniciado*. Abra o Runner Client na máquina correspondente e clique em “Start Runner”. Enquanto o Runner não for iniciado, a Task fica retida na fila — o orquestrador tem a ordem, mas não há trabalhador escutando para cumpri-la.

Para interromper o Runner, clique em “**Stop Runner**”. Isso desabilita a máquina como executora: novas automações deixam de ser puxadas até que ela seja reiniciada.

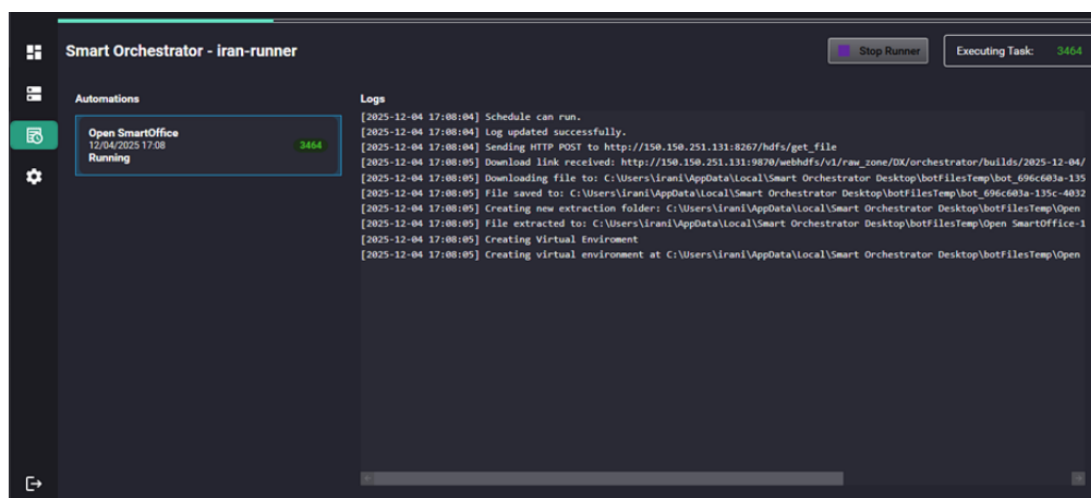


Figura 35: Botão “Stop Runner”, usado para desabilitar a execução na máquina.

Atenção

Parar o Runner (ou desligar/hibernar a máquina) não cancela as Tasks agendadas — apenas impede que elas rodem no horário. Elas se acumulam aguardando. Para operação contínua de processos agendados, a máquina precisa ficar ligada e o Runner iniciado nas janelas de execução previstas.

9.7 Configuração de hosts e validação

A última funcionalidade, acessada pelo **último ícone** do menu lateral, é a tela de **configuração de hosts / URL**. Ela permite *consultar* as informações de configuração que ligam aquela máquina ao restante do ecossistema, servindo como um ponto de checagem para validar se tudo está apontando para o lugar certo.

Os dados apresentados nessa tela são:

Campo	Para que serve
URL DX Lake	Endereço do DX Lake, onde ficam armazenados os códigos-fonte dos RPAs. É a origem dos arquivos que a máquina precisa para executar as automações.
URL SmartOffice	Endereço do portal Smart Office ao qual esta instalação está associada.
Hostname	Identificação da própria máquina na rede — útil para confirmar que o cliente está instalado no host esperado.
URL Smart Orchestrator	Endereço do orquestrador que aciona este Runner. É por esse canal que as automações alocadas chegam à máquina.

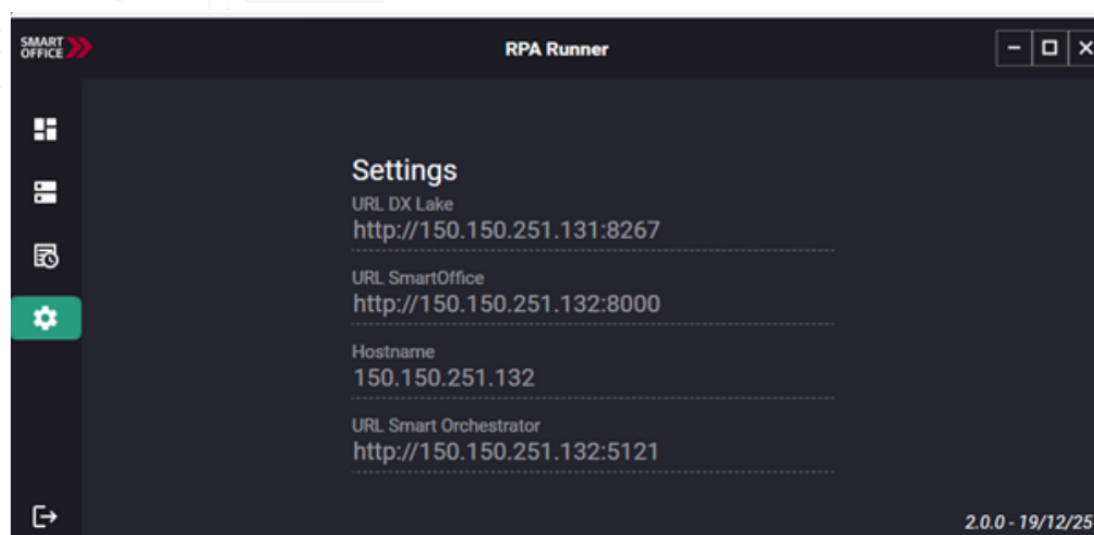


Figura 36: Tela de configuração de hosts/URL, usada para validar se a configuração do Runner está correta.

Erro comum

O Runner aparece criado e iniciado, mas não recebe as automações alocadas, ou não parece conectado ao orquestrador.

Solução

Comece por esta tela. Confira se a *URL Smart Orchestrator* e a *URL SmartOffice* apontam para o ambiente correto e se o *Hostname* corresponde à máquina esperada. Um endereço apontando para o ambiente errado é uma causa comum de um Runner que “sobe” localmente sem nunca conversar com o orquestrador.

9.8 Encerrando a sessão (Logout)

Quando quiser encerrar a sessão na aplicação, clique no **ícone de logout** localizado no canto inferior esquerdo. A autenticação é imediatamente encerrada e o usuário deixa de estar logado no Runner Client.

Dica

Não confunda *logout* com *Stop Runner*. O “Stop Runner” desabilita a execução das automações, mas mantém você autenticado; o *logout* encerra a sua sessão na aplicação. Para uma máquina que deve continuar executando processos agendados, o esperado é justamente o contrário: manter o Runner iniciado e a sessão ativa, deixando o computador disponível para receber a próxima automação da fila.

fim do capítulo ■



Digital Transformation

10 Arquitetura de Projeto e Boas Práticas (The DX Way)

Esta é a abertura da Parte II do manual, dedicada ao **modelo de desenvolvimento** esperado pela LG. Até aqui você viu *como* o Smart Office orquestra a execução: ele provisiona um ambiente virtual (venv) em um servidor e aciona um *runner* — uma máquina real — para rodar o RPA dentro da intranet. Agora o foco muda para o outro lado da mesa: *como* o código que chega ao orquestrador deve ser escrito, organizado e empacotado.

O conjunto de convenções que rege esse trabalho é chamado internamente de **The DX Way**. Ele integra Clean Code, arquitetura modular, governança de qualidade e o padrão GitLab em um único modelo profissional. O objetivo é direto: garantir que toda automação seja **modular, escalável e rastreável**, para que qualquer pessoa do time — e não só o autor original — consiga entender, manter e evoluir o robô.

Informação

O Smart Office é **agnóstico**: ele não impõe um framework de automação. O RPA em si pode ser escrito com BotCity, Selenium ou Playwright — o orquestrador executa qualquer um deles. As boas práticas deste capítulo, portanto, valem para o desenvolvimento em Python de modo geral; os exemplos usam BotCity e Selenium por serem os mais comuns na LG.

10.1 Fluxo de Desenvolvimento Padrão

Todo projeto percorre um caminho previsível, do template ao orquestrador. Entender esse fluxo evita o erro mais comum de quem vem de outras stacks: achar que “dar merge” publica o robô em produção. **Não publica**. No Smart Office o versionamento (GitLab) e a publicação no orquestrador (upload do .zip) são etapas separadas.

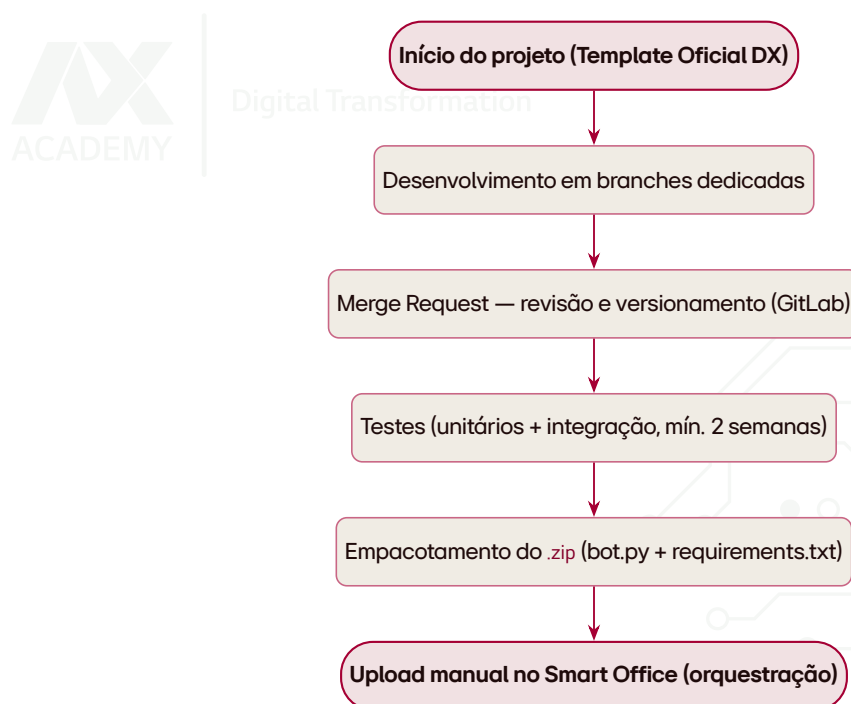


Figura 37: Fluxo de desenvolvimento padrão de um RPA

Atenção

O último passo é **upload manual do .zip** no Smart Office. O GitLab cobre versionamento, nomenclatura e revisão (Merge Request); ele *não* publica o bot no orquestrador. O Smart Office não é compatível com GitHub Actions nem com pipelines de CI/CD genéricos como forma de deploy — o caminho de publicação é sempre o upload do pacote.

O artefato final que o Smart Office consome é um **.zip** contendo, no mínimo, o **bot.py** (ponto de entrada) e o **requirements.txt** (dependências). A partir desse pacote o orquestrador cria o venv no servidor e despacha a execução ao runner. Por isso a estrutura de projeto a seguir foi pensada para empacotar de forma limpa.

10.2 Estrutura de Projeto

Nenhum projeto começa de uma pasta vazia. Todo RPA deve ser iniciado a partir do **Template Oficial DX**, gerado via **cookiecutter**. Isso garante que dois desenvolvedores diferentes, em processos diferentes, produzam repositórios com a mesma anatomia — o que torna a revisão, o onboarding e a manutenção muito mais baratos.

Estrutura de projeto padrão (Template DX)

```
RPA0123_Processamento_Faturas_TAX/
|-- app/                # Componentes principais da automacao
|-- modules/            # Subprocessos / etapas isoladas da automacao
|-- utils/              # Funcoes auxiliares e constantes (DRY)
|-- resources/          # Ativos visuais (imagens p/ reconhecimento de tela)
|-- tests/              # Testes unitarios e de integracao
|-- config/             # Configuracoes e parametros por ambiente
|-- docs/               # Documentacao e fluxogramas (.drawio)
|-- ci_cd/              # Scripts de pipeline (update_bot.py) -> ver nota
|-- bot.py              # Ponto de entrada -- contem o metodo action()
|-- requirements.txt     # Dependencias Python (pip install -r)
|-- .gitlab-ci.yml       # Pipeline GitLab (build/test) -> ver nota
```

A tabela abaixo resume a responsabilidade de cada diretório. A regra mental é simples: cada pasta responde a uma pergunta clara sobre “onde vive” cada tipo de código.

Diretório / Arquivo	Responsabilidade
app/	Componentes principais que compõem o esqueleto da automação.
modules/	Módulos reutilizáveis; a lógica do processo dividida em subetapas independentes.
utils/	Constantes e funções auxiliares genéricas. Concentra o que seria repetido (princípio DRY).
resources/	Recursos visuais: imagens usadas para reconhecimento de tela, templates.
tests/	Testes automatizados, unitários e de integração.
config/	Parâmetros que variam entre ambientes (URLs, timeouts, caminhos).
docs/	Documentação técnica e fluxogramas (diagramas .drawio , relatórios).
bot.py	Ponto de entrada e lógica principal do robô; contém o método action() .
requirements.txt	Lista de dependências instaláveis via pip install -r requirements.txt .

Atenção

Os itens **ci_cd/** e **.gitlab-ci.yml** **existem no template** (o **ci_cd/** traz o script **update_bot.py** e o **.yml** descreve os estágios **build / test / deploy**). Porém, para o **Smart Office**, o estágio de “deploy” *não* publica o bot automaticamente no orquestrador. A publicação é o **upload manual do .zip**. Use o pipeline GitLab, se quiser, para **build** e **test**; a subida ao Smart Office continua sendo manual.

Informação

Algumas variantes do template usam `main.py` como ponto de entrada em vez de `bot.py`. Para o Smart Office, o que importa é que o `.zip` enviado contenha um ponto de entrada executável e o `requirements.txt`. Ao longo deste manual adotamos `bot.py` por ser o padrão do ciclo de vida BotCity.

10.3 Princípios Arquiteturais

Antes de olhar linha a linha, quatro princípios governam *como* o código é fatiado. Eles não são teoria acadêmica: cada um resolve uma dor concreta de manutenção de robôs.

Princípio	O que exige	Por que importa
SRP (Single Responsibility)	Cada módulo tem uma única responsabilidade.	Facilita manutenção e testes: você conserta uma coisa sem quebrar outra.
SoC (Separation of Concerns)	Separar a lógica de negócio da mecânica de automação (cliques, esperas).	Promove clareza e reuso; a regra de negócio não fica presa a um seletor de tela.
DRY (Don't Repeat Yourself)	Centralizar funções comuns em <code>utils/</code> ou <code>modules/</code> .	Reduz redundância e erros: corrige-se em um lugar só.
Clean Code	Código legível, com <i>docstrings</i> e <i>type hints</i> .	Melhora colaboração e revisões; reduz a dependência do desenvolvedor original.

O impacto operacional é medível. Código limpo reduz o **MTTR** (*Mean Time To Repair*) — o tempo médio para consertar um robô que falhou — e aumenta a disponibilidade da automação em produção.

10.4 Boas Práticas de Codificação Python**10.4.1 Conformidade com a PEP 8**

A **PEP 8** é o guia de estilo oficial do Python, e sua observância é obrigatória. Ela não é sobre gosto pessoal; é sobre remover o atrito de leitura para que revisões de código fluam.

Ponto-chave	Diretriz
Comprimento da linha	Limitar todas as linhas a no máximo 79 caracteres .
Nomenclatura	Variáveis e funções em <code>snake_case</code> ; classes em <code>CapWords</code> (CamelCase); constantes em <code>ALL_CAPS</code> .
Espaços em branco	4 espaços por nível de indentação; evitar espaços supérfluos (antes de vírgulas, parênteses).
Importações	Uma por linha, agrupadas na ordem: (1) biblioteca padrão, (2) bibliotecas de terceiros, (3) módulos locais.

Dica

Não confie na disciplina manual para PEP 8: automatize. Use um **linter** (`Flake8` ou `Pylint`) para apontar violações e um **formatador** (`Black`) para corrigi-las sozinho. Rodá-los localmente antes do commit — e opcionalmente no pipeline GitLab — garante conformidade sem esforço humano.

10.4.2 Docstrings no Padrão Google Style

Toda função, método e classe **DEVE** ter docstring descrevendo propósito, parâmetros e retorno. O padrão adotado é o **Google Style**, por ser legível e consistente. Uma docstring bem escrita é a dife-

rença entre reusar uma função com confiança e ter que ler todo o corpo dela para entender o que faz.

Docstring no padrão Google Style

```

1 def extrair_dados_tabela(driver: WebDriver, seletor: str) -> list:
2     """
3     Extrai dados de uma tabela HTML utilizando o seletor CSS fornecido.
4
5     Args:
6         driver: Instancia do WebDriver do Selenium.
7         seletor: Seletor CSS da tabela a ser extraída.
8
9     Returns:
10        Uma lista de listas, onde cada lista interna representa
11        uma linha da tabela.
12
13    Raises:
14        NoSuchElementException: Se o seletor da tabela não for encontrado.
15    """
16    # Logica de extracao...
17    pass

```

Repare no bloco **Raises**: documentar as exceções que a função pode lançar é o que permite a quem chama tratá-las corretamente — assunto da próxima seção.

10.5 Boas Práticas Específicas de RPA

Aqui mora a diferença entre um robô que roda uma vez na sua máquina e um robô que roda 300 vezes por mês sem babá. RPA interage com sistemas externos — páginas web lentas, telas que mudam, redes instáveis — e precisa ser **resiliente** a tudo isso.

10.5.1 Tratamento de Exceções Robusto

- **try-except em toda interação externa**: envolva em **try-except** *qualquer* interação com a UI ou com sistemas externos (APIs, bancos de dados). Se pode falhar por causa do mundo lá fora, tem que estar protegido.
- **Exceções específicas, não genéricas**: capture a exceção exata (ex.: `NoSuchElementException` do Selenium) em vez de `Exception`. Capturar tudo esconde bugs reais e trata sintomas diferentes como se fossem iguais.
- **Retries com backoff exponencial**: para falhas temporárias (lentidão de rede, página ainda carregando), tente de novo com espera crescente entre as tentativas (2s, 4s, 8s...). Isso dá tempo ao sistema de se recuperar sem martelá-lo.

Tratamento de exceções: retry e espera explícita

```

1 import time
2 import logging
3
4 from selenium.webdriver.common.by import By
5 from selenium.webdriver.support.ui import WebDriverWait
6 from selenium.webdriver.support import expected_conditions as EC
7 from selenium.common.exceptions import NoSuchElementException, TimeoutException
8
9 logger = logging.getLogger(__name__)
10

```

```

11
12 def clicar_com_retry(driver, locator, tentativas=3):
13     """Aguarda o elemento ficar clicavel e clica, com retry e backoff."""
14     for tentativa in range(1, tentativas + 1):
15         try:
16             elemento = WebDriverWait(driver, 15).until(
17                 EC.element_to_be_clickable(locator)
18             )
19             elemento.click()
20             return
21         except (TimeoutException, NoSuchElementException) as exc:
22             # backoff: espera ENTRE tentativas, nao para aguardar a UI
23             espera = 2 ** tentativa
24             logger.warning(
25                 "Falha ao clicar em %s (tentativa %d/%d): %s. "
26                 "Nova tentativa em %ds.",
27                 locator, tentativa, tentativas, exc, espera,
28             )
29             time.sleep(espera)
30             driver.save_screenshot("erro_clique.png")
31             raise RuntimeError(
32                 f"Elemento {locator} indisponivel apos {tentativas} tentativas"
33             )

```

10.5.2 Esperas Explícitas: nunca `time.sleep()` fixo

Atenção

NUNCA use `time.sleep()` fixo para *aguardar um elemento na tela*. Um `sleep(5)` ou é longo demais (desperdiça tempo em toda execução) ou curto demais (falha quando a página demora um pouco mais). Use **esperas explícitas** — `WebDriverWait` no Selenium ou o `wait_for` do BotCity — que aguardam a *condição exata* do elemento (visível, clicável, presente) e seguem em frente assim que ela é satisfeita.

Erro comum

Um robô “funciona na minha máquina” mas falha aleatoriamente em produção, sempre em telas diferentes. O log mostra `NoSuchElementException` logo após um `time.sleep(3)`.

Solução

O `sleep` fixo não acompanha a variação de latência do ambiente de produção. Troque cada espera fixa por `WebDriverWait(driver, timeout).until(EC.presence_of_element_located(locator))`. O robô passa a esperar exatamente o necessário e nem um segundo a mais.

Observação: usar `sleep` como *backoff entre tentativas de retry* (como no exemplo acima) é legítimo — ali você não está esperando a UI, e sim dando folga ao sistema entre reintentos.

10.5.3 Seletores Robustos

Priorize seletores que **não mudam com o layout**: ID, Name e atributos `data-*`. Evite XPath longos ou seletores CSS complexos que dependem da posição do elemento na árvore — eles quebram assim que o front-end muda uma `div`. Um bom seletor é aquele que continua válido depois de um redesign de tela.

10.5.4 Credenciais e Configuração Externa

Atenção

Credenciais (usuário e senha) **NÃO DEVEM** estar no código-fonte — nem em texto, nem “escondidas” em variáveis, nem commitadas por engano. Use o **Vault / Credentials do framework** de automação (por exemplo, o Credentials do BotCity) ou um gerenciador de credenciais corporativo. Segredo em código vaza no primeiro `git push` e é praticamente impossível de remover do histórico depois.

Parâmetros que variam entre ambientes (URLs, timeouts, caminhos) devem ser lidos de **variáveis de ambiente** ou de arquivos em `config/` — não fixados no meio da lógica. Assim o mesmo bot roda em homologação e em produção só trocando a configuração.

10.5.5 Logs Estruturados

- Use os **níveis apropriados**: `DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`.
- Prefira logs em formato **estruturado** (JSON), que ferramentas de monitoramento conseguem ingerir e filtrar automaticamente.
- Todo log de erro **DEVE** conter: *timestamp*, ID do processo, etapa em que falhou, mensagem de erro e *stack trace*. Sem isso, depurar uma falha que aconteceu de madrugada vira adivinhação.

10.5.6 Screenshots e Fechamento de Recursos

Dica

Capture screenshot **apenas no momento de uma exceção não tratada**, para auxiliar a depuração. Evite *prints* a cada etapa: eles degradam a performance e enchem o armazenamento sem agregar informação.

Todo recurso aberto — conexão de banco, arquivo, sessão de navegador — **DEVE** ser liberado ao final, tanto em sucesso quanto em falha. Para arquivos e conexões, use o gerenciador de contexto (`with open(...)` as `f:`). Para o navegador, encerre sempre: `driver.quit()` no Selenium, `web_bot.stop_browser()` no BotCity. Um robô que “esquece” de fechar o navegador acumula processos zumbis no runner até derrubá-lo.

10.6 Ciclo de Vida do Bot (BotCity)

Quando a automação usa o framework BotCity, ela é estruturada em torno de **métodos de ciclo de vida** bem definidos. Respeitá-los mantém o código previsível e separa claramente “montar o cenário” de “executar o processo”.

Método	Papel
<code>on_start</code>	Inicialização: abrir o navegador, ler configuração, autenticar.
<code>action</code>	Toda a lógica de negócio do processo vive aqui.
<code>not_found</code>	Chamado quando um elemento esperado não é encontrado na tela — bom ponto para screenshot e erro tratado.
<code>on_finish</code>	Finalização: fechar navegador e sessões, liberar recursos — executado sempre, com sucesso ou falha.

Ciclo de vida de um bot BotCity

```
1 from botcity.web import WebBot
```



```

2
3
4 class Bot(WebBot):
5
6     def on_start(self):
7         # Inicializacao: configurar navegador, ler config, autenticar.
8         self.headless = False
9
10    def action(self, execution=None):
11        # A LOGICA DE NEGOCIO vive aqui -- e somente aqui.
12        self.processar_faturas()
13
14    def not_found(self, label):
15        # Elemento esperado nao apareceu na tela.
16        self.save_screenshot("nao_encontrado.png")
17        raise RuntimeError(f"Elemento nao encontrado: {label}")
18
19    def on_finish(self):
20        # Liberacao de recursos -- roda em sucesso OU falha.
21        self.stop_browser()

```

Atenção

Não misture inicialização e negócio. A regra é dura: lógica de negócio **só** no `action`; abrir recursos no `on_start`; fechá-los no `on_finish`. Isso garante que o navegador seja encerrado mesmo quando o processo estoura no meio.

O BotCity encapsula módulos de automação reutilizáveis — aproveite isso para criar funções novas a partir das existentes, reforçando a modularidade (SRP e DRY) em vez de reescrever a mesma manipulação de tela em cada bot.

10.7 Testes e Qualidade

Qualidade não é etapa final; é verificada durante todo o desenvolvimento e, obrigatoriamente, antes do deploy. A fase de testes de um RPA tem duração **mínima de 2 semanas** no modelo LG.

Tipo de teste	O que valida
Unitário	Funções isoladas, especialmente as de <code>utils/</code> e <code>modules/</code> .
Integração	A comunicação entre módulos, com ambiente de QA e dados mascarados.
Ponta a ponta	Simulam a execução completa do processo, do início ao fim.

- Nomeie arquivos e funções de teste com o **prefixo test_** — é como o descobridor de testes os encontra.
- Cada módulo principal de lógica de negócio deve ter **pelo menos um teste unitário**.
- Use **mocks** para simular dependências externas (APIs, UI) nos testes unitários. Assim você testa *a sua lógica*, e não a rede ou o sistema de terceiros — o teste fica rápido e determinístico.

A execução padrão da suíte é feita pela linha de comando:

Executando a suíte de testes

```
python -m unittest discover tests/
```

Dica

Rode a suíte de testes **antes de empacotar o .zip**. Como o Smart Office recebe o pacote por upload manual, não há um portão automático que barre um bot quebrado — o teste local (ou o estágio *test* do pipeline GitLab) é a sua rede de segurança antes da publicação no orquestrador.

fim do capítulo ■



Digital Transformation

11 GitLab: Versionamento e Merge Request

O GitLab é a **única** plataforma de versionamento integrada ao fluxo de trabalho do Smart Office. Todo o código de uma automação — o `bot.py`, seus módulos e o `requirements.txt` — vive, é versionado e é revisado no GitLab, seguindo um conjunto de convenções que o time chama internamente de “The DX Way”. Este capítulo descreve essas convenções na ordem em que elas aparecem na vida de um projeto: como **nomear** o repositório, como organizar o trabalho em **branches** (GitFlow), como escrever mensagens de **commit** e, por fim, como submeter e revisar um **Merge Request** (MR).

Antes de tudo, vale separar dois papéis que costumam ser confundidos. O GitLab cuida do ciclo de vida do *código-fonte*: versionamento, histórico, rastreabilidade e revisão por pares. O Smart Office cuida da *publicação* e da *execução* do bot. A ponte entre os dois não é automática — e entender isso evita a maior fonte de mal-entendidos deste capítulo.

Informação

O Smart Office integra-se **exclusivamente ao GitLab**. Ele **não** usa GitHub Actions e **não** é acionado por pipelines de CI/CD genéricos para publicar o bot no orquestrador. O GitLab cobre **versionamento, nomenclatura e revisão** (o Merge Request); a **publicação** no Smart Office é sempre o **upload manual do arquivo .zip** (contendo `bot.py` e `requirements.txt`). Resumindo: *GitLab* = onde o código vive e é aprovado; *Smart Office* = onde o bot é enviado, provisionado (venv no servidor) e executado (no runner). Sempre que a documentação-fonte falar em “pipeline de deploy para o orquestrador”, leia como “empacotar o zip e subir manualmente no Smart Office”.

11.1 Nomenclatura de Repositórios

Um nome de repositório é a primeira informação que qualquer pessoa vê — e, num ecossistema com dezenas de automações, ele precisa dizer sozinho *o que é o projeto, de qual processo ele trata e de qual área ele veio*. Por isso o nome não é livre: segue um formato obrigatório por tipo de projeto. O (ID) costuma ser o número do processo ou da issue de origem, o que permite rastrear o repositório de volta à solicitação que o gerou.

Formato de nome de repositório por tipo de projeto

```
# RPA (robôs de automação)
RPA<ID>_<Nome_Do_Processo>_<DEPARTMENT>

# Quick Systems (soluções rápidas / mini-aplicações)
QS_<Nome_da_Solucao>_<DEPARTMENT>

# APIs / Serviços de back-end
API_<NomeServico>_<ENV/DEPT>
```

Tipo de projeto	Exemplo real
RPA (automação)	RPA0123_Processamento_Faturas_TAX
Quick System	QS_InvoicePreview_FIN
API / Serviço back-end	API_MetadataService_CORE

Note o critério por trás de cada prefixo: o ID numérico só existe no padrão RPA porque robôs nascem de uma solicitação numerada; Quick Systems e APIs recebem um nome legível no lugar do número. O sufixo final é sempre o **departamento** (ou, no caso de APIs, o ambiente/departamento), escrito em maiúsculas.

Dica

Para os **grupos** (namespaces) do GitLab, use **kebab-case** e mantenha uma hierarquia clara — por exemplo, o grupo **rpa-core** para os repositórios oficiais do time e **smart-office** para o que pertence ao produto. É no nível do grupo que se configuram **branch protection** e **merge rules**, aplicando de uma vez as regras de proteção a todos os repositórios abaixo.

Atenção

O nome do repositório é praticamente **imutável na prática**: renomear depois quebra clones, remotes e referências já espalhadas pela equipe. Acerte o formato *na criação*. Todo projeto novo deve nascer do *template oficial* (via **cookiecutter**), que já entrega a estrutura padrão de pastas — não crie repositório vazio “na mão”.

11.2 GitFlow: Fluxo de Versionamento

O modelo de ramificação adotado é baseado no **Git Flow**. A ideia central é simples: nunca se trabalha diretamente sobre o código de produção. Existem duas branches *permanentes* que servem de referência, e todo o trabalho do dia a dia acontece em branches *temporárias* que nascem, cumprem seu papel e são integradas de volta.

11.2.1 Branches Principais

Branch	Função	Observações
main	Produção	Código estável, liberado e validado. Commits diretos são proibidos.
develop	Integração	Branch principal de desenvolvimento; integra novas features antes de virar release.

O **main** representa exatamente o que está (ou pode ir) para produção. O **develop** é o ponto de encontro: cada nova funcionalidade é integrada nele primeiro, onde convive e é testada junto com o resto antes de ser promovida.

11.2.2 Branches de Trabalho

Toda tarefa começa criando uma branch dedicada a partir do **develop**. O nome segue um formato que amarra a branch à sua *issue*, para que o histórico conte a própria história do que foi feito:

Formato das branches de trabalho

```
<tipo>/<id_issue>-<descricao - curta>
```

Exemplo:

```
feature/456-adiciona-login-ui
```

Prefixo	Quando usar	Exemplo
feature/	Nova funcionalidade	feature/456-adiciona-login-ui
bugfix/	Correção não urgente	bugfix/123-ajusta-timeout-api
hotfix/	Correção crítica em produção	hotfix/789-erro-gerp-producao
release/	Preparação de versão	release/v1.0.0
docs/	Atualização de documentação	docs/melhora-readme

A diferença entre **bugfix/** e **hotfix/** é o *ponto de partida* e a *urgência*. Um **bugfix/** corrige algo ainda não crítico e segue o caminho normal via **develop**. Um **hotfix/** nasce para apagar um incêndio em produção: ele parte do **main**, corrige, volta para o **main** — e precisa também ser trazido de volta ao **develop**, senão a mesma falha reaparece na próxima release.

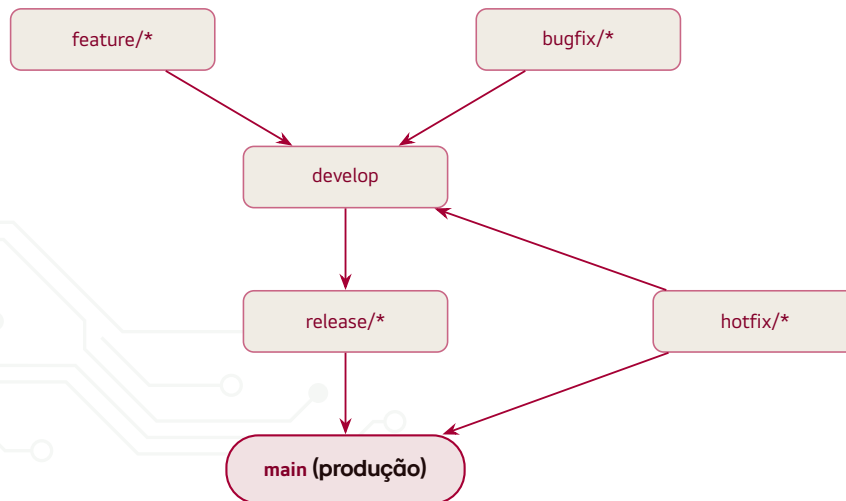


Figura 38: Fluxo de versionamento GitFlow: das branches de trabalho à main

Atenção

main é produção. **Nenhum commit vai direto para o main** (nem para o **develop**): toda mudança entra por uma branch de trabalho e só é integrada via Merge Request aprovado. A **branch protection** configurada no grupo faz isso valer na prática — um **push** direto será **rejeitado pelo servidor**.

Erro comum

Ao tentar subir uma correção rápida direto na branch protegida, o Git recusa:

remote: You are not allowed to push code to protected branches on this project.

Solução

Isso é o esperado, não um bug de permissão. Crie uma branch de trabalho a partir do **develop** e abra um MR:

Criar a branch e abrir o Merge Request

```

git checkout develop
git pull
git checkout -b feature/456-adiciona-login-ui
# ... implementa, testa ...
git push -u origin feature/456-adiciona-login-ui
# abrir o Merge Request pela interface do GitLab
  
```

11.3 Commits Semânticos

Uma mensagem de commit não é um bilhete para você mesmo daqui a cinco minutos — é a documentação que a equipe (e o revisor do MR) vai ler para entender *por que* cada linha mudou. Por isso o formato é padronizado (*Semantic Commits*): um (**tipo**), um (**escopo**) opcional entre parênteses e uma descrição breve no modo imperativo.

Formato dos commits semânticos

```
<tipo>(<escopo>): <descricao breve>
```

Exemplo:

```
feat(login): adiciona autenticao via OAuth
```

Tipo	Propósito	Exemplo
feat	Nova funcionalidade	feat(core): adiciona módulo de autenticação
fix	Correção de bug	fix(api): corrige erro no endpoint
refactor	Refatoração sem mudança de comportamento	refactor(utils): padroniza funções de log
docs	Documentação	docs(readme): adiciona instruções de setup
test	Testes	test(api): adiciona testes unitários
style	Formatação / estilo	style: corrige indentação conforme PEP 8
chore	Tarefas administrativas	chore: atualiza versão do cookiecutter

A distinção entre **refactor** e **fix** importa na hora de ler o histórico: **fix** muda comportamento (corrige algo errado); **refactor** melhora o código *sem* mudar o que ele faz. Já **style** é apenas cosmético (indentação, espaços) — nunca esconda uma correção de lógica sob um commit **style**, pois o revisor tende a passar os olhos rápido por eles.

Dica

Três regras que fazem a diferença numa revisão:

- **Modo imperativo**: escreva “adiciona”, “corrige”, “remove” — e não “adicionado” ou “adicionando”. Leia como um comando ao repositório.
- **No máximo 72 caracteres** na primeira linha: mensagens longas são cortadas na visualização do GitLab.
- **Referencie a issue** (ex.: #456). Assim o commit aparece automaticamente vinculado à issue, fechando o ciclo de rastreabilidade.

Exemplo: commit e envio da branch

```
git add .
git commit -m "feat(login): adiciona autenticao via OAuth (#456)"
git push
```

11.4 Merge Request (MR)

O Merge Request é o **principal checkpoint de qualidade** do código. É o momento em que uma branch de trabalho pede para ser integrada ao **develop** (ou ao **main**) e, antes disso, passa por revisão humana. Nenhum código chega a produção sem cruzar essa etapa.

11.4.1 Requisitos Obrigatórios

- **Revisão** por pelo menos um membro **sênior** — código não se auto-aprova.
- **Descrição detalhada** do MR: qual o propósito, o que mudou e qual o impacto. O revisor precisa entender a intenção sem abrir cada arquivo.
- O autor deve se definir como **Assignee** e adicionar um **Reviewer**. Assignee = quem responde pelo MR; Reviewer = quem aprova.
- Código em conformidade com **PEP 8** e com os padrões do The DX Way.

11.4.2 Critérios de Qualidade

- Conformidade com **PEP 8** e princípios de *Clean Code*.
- **Docstrings** no padrão Google Style obrigatórias em classes e funções.
- Cobertura mínima de testes exigida para a integração.

- Logs **estruturados**, com contexto e *timestamps*.

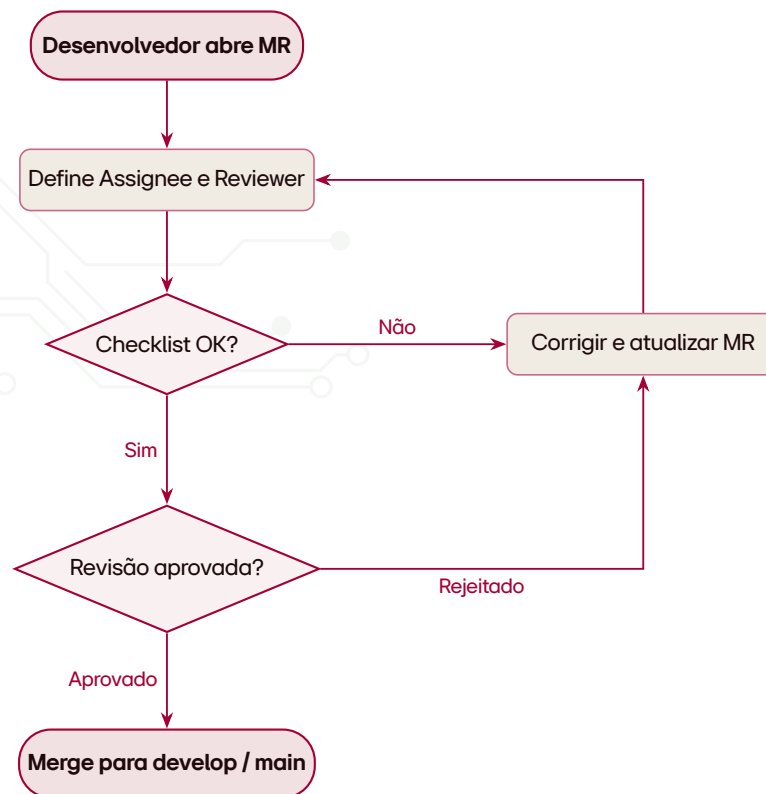


Figura 39: Fluxo do Merge Request: da abertura à revisão e ao merge

O fluxo tem duas portas de saída para o “não”: se o *checklist* do MR ainda não está completo (descrição, Reviewer, PEP 8), o autor corrige e reenvia; se a revisão do sênior reprova, o caminho é o mesmo — ajustar e voltar. Só quando ambas as portas dizem “sim” o merge acontece. Esse loop de correção é normal e saudável: um MR raramente é aprovado de primeira, e é exatamente aí que a qualidade é construída.

Atenção

O merge aprovado integra o código no GitLab — ele **não** publica o bot. A documentação-fonte descreve um estágio de “deploy” em pipeline que enviaria o robô ao orquestrador; **no Smart Office isso não existe**. Depois do merge, o pacote `.zip` (com `bot.py` e `requirements.txt`) é enviado ao Smart Office por **upload manual**. O GitLab garante que o que será empacotado passou por revisão; o Smart Office recebe o zip, provisiona o ambiente virtual (venv) no servidor e aciona o runner para executar. São etapas separadas, feitas por pessoas — não por um gatilho de CI/CD.

Dica

Uma metáfora que o time usa para fixar o fluxo: o `main` é a *pista de pouso* onde só aterrissa voo aprovado; o `develop` é o *hangar* onde as features são montadas; cada `feature/` ou `bugfix/` é um *voo de carga* com destino e manifesto (a issue) definidos; o commit é o *registro de bordo*; e o Merge Request é a *inspeção de segurança* — só depois dela o código “decola” para produção.

12 Nomenclatura e Agendamento de Automações

Uma automação que roda hoje sem problemas pode virar um pesadelo de manutenção daqui a seis meses se ninguém conseguir dizer, olhando para o painel do orquestrador, *qual* código está executando, *de onde* ele veio e *quando* ele foi disparado. É exatamente esse elo de rastreabilidade que a nomenclatura operacional resolve: ela é a ponte que amarra o **versionamento no GitLab** (onde o código vive e evolui) ao **Smart Office** (onde o código é orquestrado e executado). Este capítulo trata de duas coisas práticas e complementares: como *nomear* os objetos da orquestração e como *agendar* sua execução de forma padronizada.

Informação

O Smart Office é **agnóstico** quanto ao framework de automação. O código do robô pode ter sido escrito com BotCity, Selenium ou Playwright — o orquestrador não se importa. O que ele padroniza e controla é o ciclo de vida operacional: o nome, a versão, a máquina que executa e o horário. A convenção descrita aqui vale para qualquer um desses frameworks.

12.1 As quatro entidades da orquestração

Antes de nomear, é preciso entender *o que* se está nomeando. A operação no Smart Office se organiza em quatro entidades que representam a mesma automação em estágios diferentes do seu ciclo de execução:

Entidade	O que é / responsabilidade
Bot	O código da automação propriamente dito (Python, usando BotCity, Selenium ou Playwright). É quem executa as tarefas automatizadas.
Automation	O registro lógico dentro do Smart Office que vincula um Bot a um ou mais Runners. Controla deploy, versão e status.
Runner	A máquina real que executa o RPA, acionada pelo Smart Office a partir do ambiente virtual (venv) provisionado no servidor. Roda os jobs sob agendamento.
Scheduler	O módulo de agendamento que define frequência e horário de execução. Automatiza execuções periódicas via expressão cron.

A relação entre elas é cíclica: o Bot é publicado como uma Automation, que é executada por um Runner, sob o comando de um Scheduler que reaciona a Automation periodicamente.

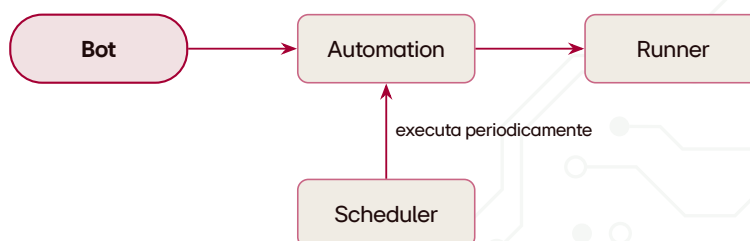


Figura 40: As quatro entidades da orquestração e seus identificadores

12.2 Convenção de nomes: departamento, processo e tipo

Bots, Automations e Schedulers seguem uma única convenção obrigatória:

<DEPARTAMENTO>_<NOME_DO_PROCESSO>_<TIPO>

Elemento	Descrição	Exemplo
<DEPARTAMENTO>	Abreviação da área de negócio ou projeto.	SCM, VTC, FIN, LOG
<NOME_DO_PROCESSO>	Nome curto e legível que descreve a automação.	InquiryEndingOnhand
<TIPO>	Identificador do objeto no ciclo de vida.	BOT, AUTO, SCHED

A regra de ouro está no <TIPO>: os três objetos de uma mesma automação **compartilham o identificador base** e mudam apenas o sufixo. Assim, um único processo produz três nomes irmãos:

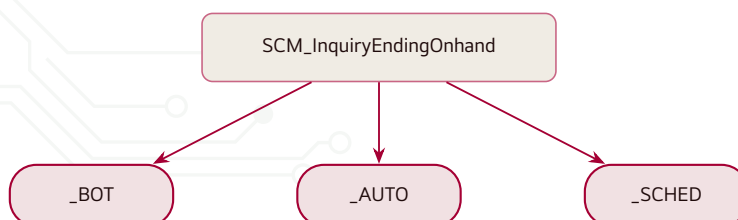


Figura 41: Convenção de nomes: departamento, processo e tipo

Dica

O motivo prático de compartilhar a base é a **rastreabilidade instantânea**. Quando um Schedule dispara às 8h e o log aparece com erro, o operador lê `SCM_InquiryEndingOnhand_SCHED`, deduz na hora qual é a Automation (`..._AUTO`) e qual é o Bot (`..._BOT`), sem precisar consultar tabela de-para nenhuma. Um único nome carrega toda a linhagem.

Erro comum

Nomear os objetos com bases divergentes — por exemplo, Bot chamado `SCM_Onhand_BOT` mas Schedule chamado `SCM_InquiryEndingOnhand_SCHED`. Resultado: o agendamento vira “órfão” aos olhos de quem opera, ninguém conecta um ao outro, e a investigação de uma falha leva o dobro do tempo.

Solução

Defina o identificador base *uma vez*, ao criar o Bot, e reaproveite-o literalmente ao criar a Automation e o Schedule. Copie e cole a base; só troque o sufixo. Trate a base como um nome próprio imutável da automação.

12.3 Nomenclatura de Runners

O Runner é a máquina que executa. Diferente dos objetos anteriores, ele não pertence a um único processo — ele hospeda vários. Por isso segue uma convenção própria, em dois padrões possíveis:

Tipo de Runner	Padrão	Exemplo
Númerico (default DX)	<code>dx-virtual-machine<N></code>	<code>dx-virtual-machine1</code>
Por projeto / departamento	<code><DEPARTAMENTO>-runner</code>	<code>scm-runner, vtc-runner, fin-runner</code>

Diretrizes obrigatórias para nomes de Runner:

- Use nomes curtos e descritivos em **kebab-case** (tudo minúsculo, palavras separadas por hífen). Nada de espaços, acentos ou maiúsculas.
- Agrupe Runners *por departamento* quando o volume de automações for alto — isso isola cargas, facilita capacity planning e evita que uma automação pesada de um setor derrube as de outro.

- Cada Runner deve ter um **owner técnico documentado**, ou seja, uma pessoa responsável por manter aquela máquina saudável e sincronizada.

Atenção

O padrão numérico `dx-virtual-machine<N>` é o default e serve à maioria dos casos. Migre para o padrão por departamento (`<dept>-runner`) apenas quando houver justificativa de volume ou isolamento — proliferar Runners sem critério fragmenta a operação e dilui a responsabilidade sobre cada máquina.

12.4 Deploy e versionamento operacional

Toda automação publicada precisa estar amarrada a uma versão rastreável do código. Cada uma das quatro entidades tem uma origem e uma regra de vínculo:

Elemento	Fonte	Regra
Bot	Repositório GitLab (branch <code>main</code>)	Deve estar associado a uma <i>tag</i> de versão semântica <code>vX.Y.Z</code> .
Automation Runner	Smart Office (Orquestrador) Máquina designada	Deve referenciar o <i>build</i> mais recente e ativo. Deve ter o ambiente sincronizado com as dependências Python do projeto.
Schedule	Scheduler do Smart Office	Deve seguir um cronograma (cron) validado e documentado.

A tag `vX.Y.Z` na `main` não é decoração: é o que permite responder “qual versão está em produção?” e, em caso de regressão, voltar a um estado conhecido. Versão semântica — *major.minor.patch* — comunica a natureza de cada mudança sem precisar ler o diff.

Atenção

Como o código chega ao orquestrador. No Smart Office o deploy é o **upload manual de um arquivo .zip** contendo o `bot.py` e o `requirements.txt` na raiz. O GitLab cobre *versionamento, nomenclatura e revisão* (o fluxo de Merge Request do “The DX Way”), mas **não publica** o bot automaticamente. Não existe pipeline de CI/CD nem GitHub Actions empurrando o código para o orquestrador: quem sobe o zip é uma pessoa, pela interface. Trate o GitLab como a fonte da verdade do código e o Smart Office como o destino manual da publicação.

Dica

Amarre os dois mundos pela versão: gere a tag `vX.Y.Z` no GitLab *antes* de empacotar o zip, e use exatamente essa versão como referência ao subir a Automation. Assim o “build ativo” no Smart Office sempre aponta para um commit identificável no repositório.

O requisito “ambiente sincronizado” no Runner conecta-se diretamente ao `requirements.txt`: o Smart Office provisiona um ambiente virtual (venv) a partir desse arquivo e o Runner executa dentro dele. Se o `requirements.txt` estiver incompleto, a automação sobe mas quebra em runtime.

Erro comum

O bot roda na máquina do desenvolvedor mas falha no Runner com `ModuleNotFoundError`. Quase sempre é uma dependência que estava instalada localmente e não foi declarada no `requirements.txt`.

Solução

Congele o ambiente antes de empacotar: gere o `requirements.txt` a partir de um `venv` limpo (ex.: `pip freeze > requirements.txt`) e valide que o zip instala do zero. Nunca dependa de pacotes “que já estavam na máquina”.

12.5 Política de Schedulers (agendamentos)

O agendamento no Smart Office usa expressões **cron**, o padrão de mercado para tarefas periódicas. Uma expressão cron tem cinco campos, lidos da esquerda para a direita:

Anatomia da string CRON

```
campo:   minuto  hora  dia-do-mes  mes  dia-da-semana
faixa:   0-59    0-23  1-31          1-12  0-6 (0 = domingo)

0 8 * * * -> todo dia, as 08:00
0 8 * * 1 -> toda segunda-feira, as 08:00
0 8 1 * * -> dia 1 de cada mes, as 08:00
(o asterisco * significa "qualquer valor")
```

A política operacional padroniza três frequências recomendadas:

Tipo de Processo	Cron	Nome do Scheduler	Observações
Diário	0 8 * * *	<DEPT>_<PROC>_SCHED	Executa no início do expediente.
Semanal	0 8 * * 1	<DEPT>_<PROC>_SCHED	Executa às segundas-feiras.
Mensal	0 8 1 * *	<DEPT>_<PROC>_SCHED	Fechamento de relatórios mensais.

O horário **08:00** é intencional: roda o processo logo no início do expediente, de modo que os dados estejam prontos quando as pessoas chegam, e sobra a manhã inteira para reagir caso a execução falhe.

Erro comum

Dois erros clássicos de cron: (1) inverter os campos de hora e minuto — `8 0 * * *` agenda para 00:08 da manhã, não 08:00; (2) empilhar várias automações pesadas no mesmo minuto `0 8 * *`, sobrecarregando o Runner e formando fila.

Solução

Leia sempre a expressão em voz alta pela ordem “minuto, hora, dia, mês, dia-da-semana”. Para processos concorrentes no mesmo Runner, escalone os minutos (ex.: `0 8, 10 8, 20 8`) em vez de dispará-los todos às 8h em ponto.

12.6 Boas práticas de governança operacional

Cinco práticas sustentam a operação a longo prazo:

1. **Padronizar nomes e tags** entre Bots, Automations e Schedulers — a base compartilhada e a versão `vX.Y.Z` devem ser consistentes de ponta a ponta.
2. **Documentar dependências** no `requirements.txt`, sempre gerado de um ambiente limpo.
3. **Centralizar logs** via Smart Office ou Data Lake DX, para que a investigação de falhas parta sempre de um único lugar.

4. **Registrar falhas com contexto completo:** cada erro deve carregar *stack trace*, *runner*, *timestamp* e *ID da execução*. Sem esses quatro dados, reproduzir e diagnosticar um erro intermitente vira loteria.
5. **Realizar backup periódico** das automations, armazenando-o no GitLab (ex.: em `backups/automations`).

Dica

Log sem *ID de execução* e *timestamp* é quase inútil quando o mesmo bot roda dezenas de vezes por dia: não há como saber a qual execução aquele erro pertence. Faça do quarteto *stack trace* + *runner* + *timestamp* + *ID* o formato mínimo de todo log de erro.

12.7 Monitoramento operacional

O Smart Office resume a saúde de cada automação em três estados, no padrão de semáforo. Cada estado tem uma ação associada:

Status	Descrição	Ação
• Running	Execução normal.	Nenhuma ação necessária.
• Warning	Execução instável.	Revisar os logs.
• Error	RPA com falha.	Acionar manutenção.

Informação

O estado **Warning** é o mais valioso e o mais ignorado. Ele sinaliza degradação *antes* da falha — lentidão crescente, retries frequentes, elemento de tela que às vezes não aparece. Tratar um Warning cedo costuma ser a diferença entre um ajuste de cinco minutos e um Error em produção no fechamento do mês.

12.8 Papéis e permissões no orquestrador

O Smart Office define três níveis técnicos de permissão. Trata-se de *controle de acesso à plataforma*, não de cargo ou hierarquia de time:

Papel	Permissões principais
Admin	Gerenciamento da organização, dos usuários e das permissões.
Developer	Deploy de bots, configuração de schedulers e leitura de logs.
Operator	Acesso apenas de leitura a dashboards e logs.

Atenção

Aplique o **princípio do menor privilégio**. Quem só acompanha a operação deve ser **Operator**; a capacidade de fazer deploy e mexer em schedulers (**Developer**) fica com quem desenvolve; e o **Admin**, que altera usuários e permissões, deve ser restrito ao mínimo de pessoas. Conceder Admin “por conveniência” amplia a superfície de erro sobre toda a infraestrutura de orquestração.

fim do capítulo ■

13 Validação e Deploy em Produção

Este capítulo encerra a Parte II do manual. Até aqui você aprendeu a solicitar, mapear, arquitetar e codificar um RPA seguindo os padrões da LG, além de conhecer as entidades operacionais do orquestrador — **Runner**, **Tasks** e **Logs**. Falta o último passo: colocar o robô em **produção** de forma controlada. O procedimento a seguir é obrigatório e existe por um motivo simples — um RPA que funciona na máquina do desenvolvedor não é, por si só, um RPA pronto para produção. Ambiente, credenciais, conectividade e agendamento mudam quando o robô sai da estação de desenvolvimento e passa a ser executado por um **Runner** acionado pelo Smart Office dentro da intranet.

Todo o fluxo de deploy é conduzido por **seis etapas encadeadas**, cada uma com um responsável claro e uma condição objetiva para avançar. O revisor (um desenvolvedor Pleno ou Sênior) é a figura central: ele aprova o código, publica no orquestrador, valida e agenda. Nada avança se a etapa anterior não tiver sido concluída com sucesso.

Informação

O Smart Office é um **control tower** (orquestrador), não um framework de automação. Ele provisiona um ambiente virtual (**venv**) em um servidor a partir do **requirements.txt** e aciona um **Runner** — uma máquina real — para executar o RPA. O robô em si continua usando os frameworks de mercado (BotCity, Selenium, Playwright); o Smart Office apenas orquestra. Por isso o **artefato de deploy** é um pacote **.zip** enxuto contendo, no mínimo, o **bot.py** e o **requirements.txt**.

13.1 Etapa 1 — Gatilho: merge na branch **main**

O procedimento de deploy **não começa por decisão do desenvolvedor**, e sim por um evento técnico bem definido: o **merge do código na branch **main**** do GitLab, após a aprovação do Merge Request (MR).

Lembre-se de que commits diretos na **main** são proibidos (padrão GitLab / The DX Way). O caminho é sempre: trabalho em uma branch dedicada (**feature/**, **bugfix/**, **hotfix/**) → MR para integração → aprovação → merge. Quando esse merge para a **main** acontece, ele sinaliza que **o código está pronto para a produção** e dispara o ciclo de validação e deploy descrito neste capítulo.

Atenção

A **main** representa produção. Um merge nela é uma promessa de estabilidade. Não use a **main** como área de rascunho: se o código ainda precisa de ajustes, ele pertence a **develop** ou a uma branch de trabalho, jamais à **main**.

13.2 Etapa 2 — Revisão final (Code Review)

Antes de qualquer publicação, o MR para a **main** passa por uma **revisão final** feita por um desenvolvedor de senioridade superior.

13.2.1 Responsabilidade do revisor

- **Revisor designado**: deve ser um desenvolvedor **Pleno ou Sênior**, distinto do autor do código. A revisão por um par mais experiente é o principal checkpoint de qualidade antes da produção.
- **Foco da revisão**: não é “o código roda?” (isso os testes já responderam), e sim três eixos de risco:
 1. **Governança** — aderência aos padrões de arquitetura, nomenclatura, estrutura de pas-

tas e documentação (README.md atualizado) vistos nos capítulos anteriores.

2. **Tratamento de exceções** — o robô captura exceções específicas, faz `retry` com backoff, encerra o navegador em caso de falha e registra logs estruturados? Em produção, resiliência vale mais do que o caminho feliz.
3. **Aderência ao negócio** — a solução realmente resolve o processo mapeado, com as regras de negócio corretas?

13.2.2 Decisão de aprovação

O revisor toma uma decisão binária e a registra no MR:

Resultado	Ação imediata	Responsável
Aprovado	O revisor avança para o Deploy em produção (Etapa 3).	Revisor (Pleno/Sênior)
Rejeitado	O MR é devolvido ao desenvolvedor original com as correções detalhadas; após o ajuste, um novo MR reinicia o ciclo.	Revisor (Pleno/Sênior)

O diagrama a seguir reimagina o fluxo do Merge Request até a liberação para deploy, incluindo os laços de retorno quando falta revisor ou quando há correções pendentes:



Digital Transformation

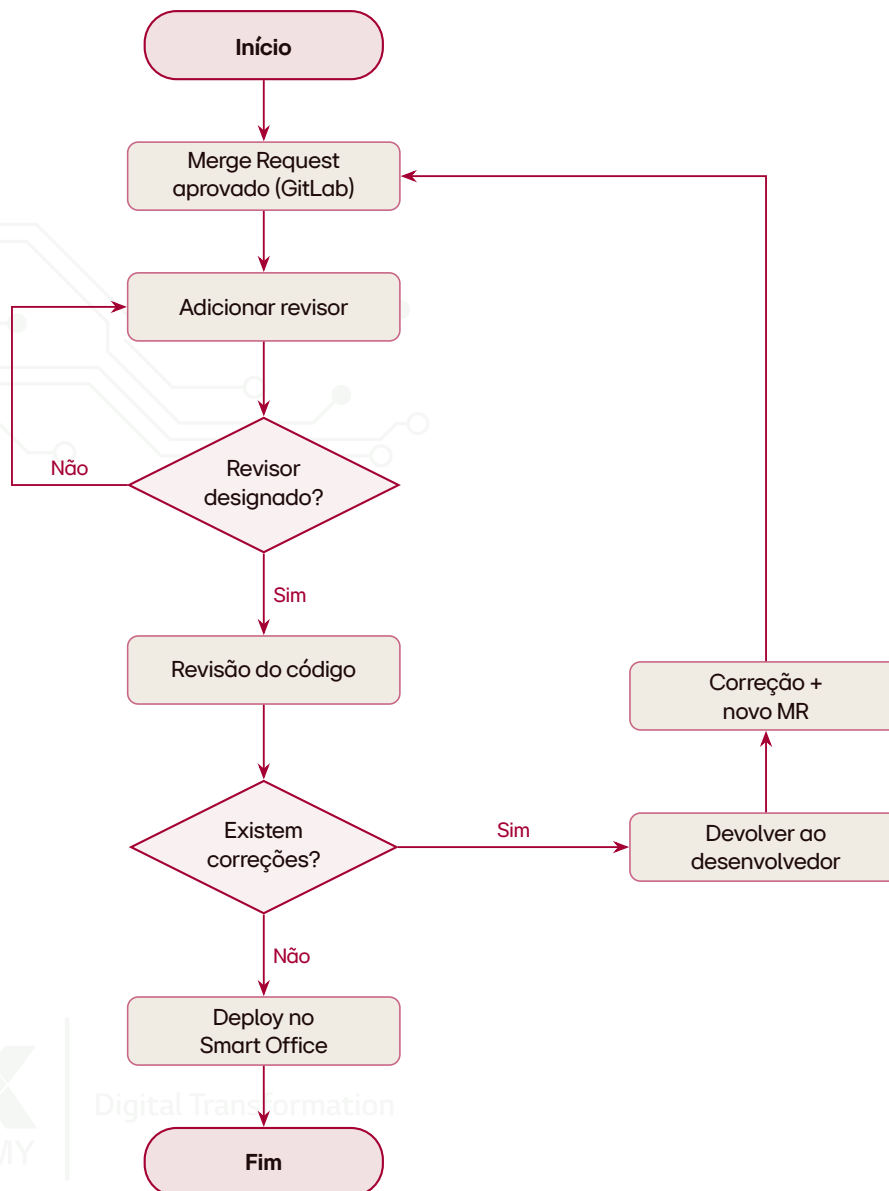


Figura 42: Fluxo de revisão de código e deploy em produção

13.3 Etapa 3 — Deploy em produção

Aprovado o código, **o próprio revisor** assume a responsabilidade pela implantação. Isso é intencional: quem aprovou responde pela publicação, evitando o clássico “passei adiante e não sei o que aconteceu”.

13.3.1 Como o deploy realmente acontece no Smart Office

Aqui vem a correção mais importante deste capítulo em relação à documentação genérica de orquestração:

Atenção

No Smart Office, o deploy em produção é o **upload manual do pacote .zip** da automação para o orquestrador. **Não** há publicação automática por pipeline de CI/CD nem por GitHub Actions. O GitLab (versionamento, nomenclatura, MR, revisão — “The DX Way”) cobre tudo *até* a aprovação do código; ele **não** publica o bot no orquestrador. Sempre que uma referência antiga mencionar

“deploy via `.gitlab-ci.yml`” ou “BotCity Orchestrator” como destino, entenda: aqui o destino é o **Smart Office** e a publicação é feita **à mão**, subindo o zip.

Na prática, o revisor empacota o projeto e faz o upload pela interface do orquestrador. Um zip de deploy típico contém o ponto de entrada e as dependências — o Smart Office usa o `requirements.txt` para provisionar o `venv` no servidor:

Estrutura do pacote de deploy

```

automacao.zip |——
bot.py         # ponto de entrada (método action) |——
requirements.txt # dependências -> venv provisionado no servidor |——
modules/       # subetapas do processo |——
utils/         # funções auxiliares e constantes |——
resources/     # ativos visuais (imagens de reconhecimento)
  
```

Para gerar o artefato antes do upload, basta empacotar o projeto:

Empacotar o projeto para o upload manual

```

# empacota o projeto para o deploy manual no Smart Office
zip -r automacao.zip bot.py requirements.txt modules/ utils/ resources/
  
```

Dica

Antes de subir o zip, confira se o `requirements.txt` está com as versões **fixadas** (`pip freeze`). O `venv` de produção é montado a partir dele: uma dependência sem versão pode instalar uma release nova e quebrar o robô só em produção — justamente onde é mais caro descobrir.

Erro comum

Subir um zip que inclui `.env`, credenciais ou a pasta `venv/` local. Credenciais no pacote são um risco de segurança grave; a `venv/` local infla o artefato e nem sequer é usada — o servidor recria o ambiente.

Solução

Mantenha credenciais fora do código (gerenciador de credenciais / variáveis de ambiente de produção) e empacote apenas `bot.py`, `requirements.txt` e os diretórios de código (`modules/`, `utils/`, `resources/`). Um `.gitignore` bem configurado evita empacotar o que não deve ir.

13.4 Etapa 4 — Validação pós-deploy (Smoke Test)

Deploy feito **não** significa robô validado. Imediatamente após a publicação, é **obrigatório** um **smoke test**: uma verificação mínima de que o robô acorda e respira no ambiente de produção. O nome vem da engenharia — “ligar o aparelho e ver se sai fumaça”.

13.4.1 Como executar o smoke test

O teste é feito **enviando manualmente Tasks** para a fila de execução do robô no orquestrador — exatamente o mesmo mecanismo apresentado no **módulo Tasks (Capítulo 05)**. A diferença está no critério de escolha dessas tarefas:

- **Escopo mínimo e não crítico**: envie poucas Tasks, escolhidas para exercitar apenas o essencial — **conectividade, acesso às credenciais de produção** e a **execução inicial do fluxo**

principal. Não é o momento de testar todos os casos de borda.

- **Dados que não impactam produção:** use dados de teste que *simulem* um cenário real, mas que **não causem efeito colateral** nos sistemas reais — nada de aprovar uma fatura de verdade, disparar um e-mail para um cliente ou gravar um registro definitivo. O objetivo é validar o caminho, não movimentar o negócio.

Como o Smart Office aciona um **Runner** (máquina real) para rodar a Task, o smoke test valida também a ponte servidor → Runner → sistema-alvo dentro da intranet — pontos que simplesmente não existem na máquina do desenvolvedor.

13.4.2 Onde olhar o resultado

O veredito do smoke test se lê nos **Logs (Capítulo 07)**. Acompanhe a execução da Task de teste pela tela de logs do orquestrador: mensagens **INFO** de progresso, ausência de **ERROR/CRITICAL** e a conclusão da Task com status de sucesso indicam que o robô está funcional. Se houver **stack trace** ou screenshot de exceção, a validação falhou.

13.4.3 Tratamento de falhas

Resultado	Ação imediata	Responsável
Sucesso	O revisor avança para o Agendamento (Etapa 5).	Revisor (Pleno/Sênior)
Falha	O revisor NÃO agenda o robô. O desenvolvedor original é acionado imediatamente para diagnóstico e correção; a solução volta pelo fluxo (correção → novo MR → novo deploy).	Revisor aciona o desenvolvedor original

Atenção

Um smoke test que falha **bloqueia o agendamento**. Nunca “agende assim mesmo, depois eu vejo”: um robô agendado com falha começa a rodar no horário do negócio, sem ninguém olhando, e transforma um problema conhecido em um incidente de produção. Sem sucesso no smoke test, o robô não é ativado — ponto.

Erros mais comuns nesta etapa concentram-se em duas causas, ambas invisíveis em desenvolvimento:

Erro comum

A Task de teste falha logo no login do sistema-alvo (autenticação recusada, “credenciais inválidas”).

Solução

Quase sempre é diferença entre credenciais de desenvolvimento e de **produção**. Confirme que o robô lê as credenciais do ambiente de produção (gerenciador de credenciais / variáveis do Runner) e não valores fixos do ambiente local.

Erro comum

A Task fica presa ou estoura **timeout** ao acessar um sistema interno que “funcionava na minha máquina”.

Solução

Verifique a conectividade a partir do **Runner**: rede, VPN, firewall e URLs internas podem diferir da estação do desenvolvedor. As telas de configuração do cliente do orquestrador (URLs internas e hostname do Runner) ajudam a validar se o Runner enxerga os endereços corretos.

O diagrama abaixo reimagina o fluxo do smoke test, do deploy até a decisão de agendar ou devolver para correção:

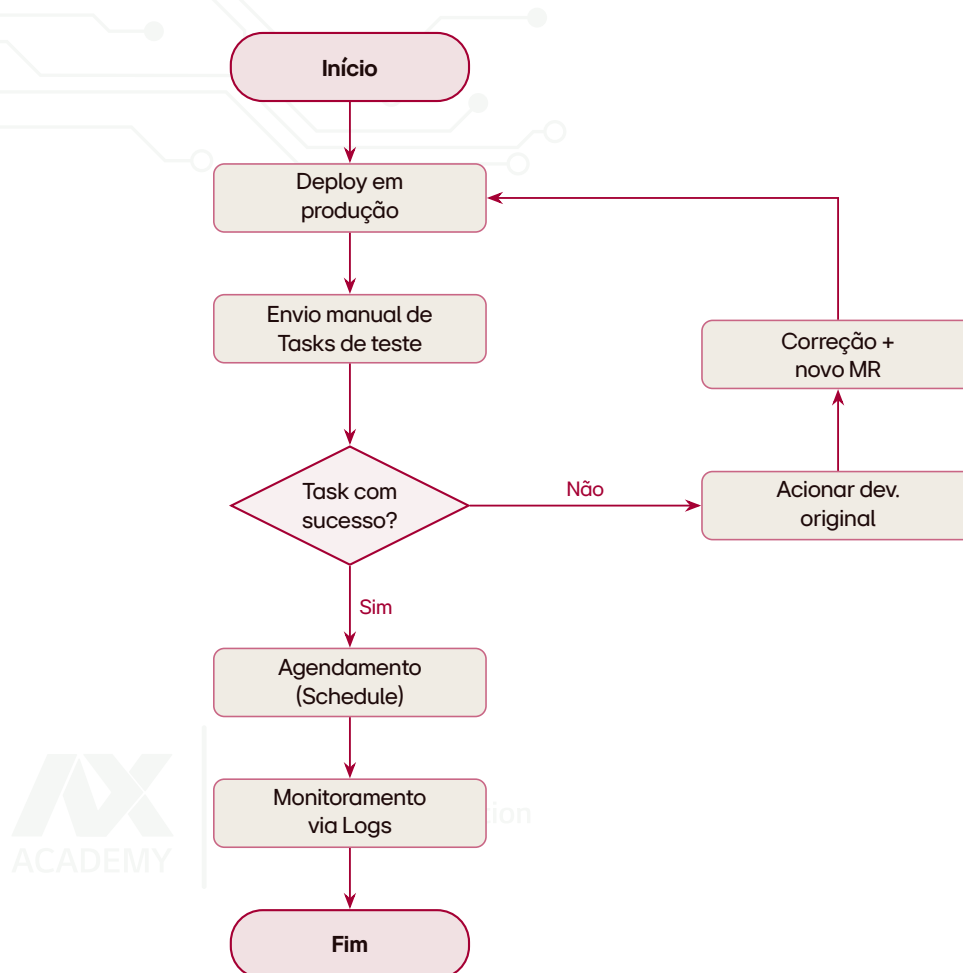


Figura 43: Fluxo de validação pós-deploy (smoke test)

13.5 Etapa 5 — Agendamento (Schedule)

Somente após o smoke test bem-sucedido a automação pode ser ativada para execução contínua, criando-se o **Schedule** no orquestrador.

- **Base do agendamento:** horários e frequência devem seguir o que foi **solicitado pelos stakeholders** do processo (diário, semanal, mensal ou sob demanda). O robô existe para servir o negócio — é o negócio que define quando ele roda.
- **Capacidade do ambiente:** o revisor deve garantir que o agendamento respeite a capacidade de processamento — número de **slots** disponíveis e janelas de execução. Empilhar muitas automações pesadas no mesmo horário e no mesmo Runner cria filas e atrasos.
- **Registro da ativação:** registre data e hora do deploy e do agendamento na documentação do projeto (README.md ou wiki do GitLab) e, se aplicável, no controle de mudanças. Esse registro

é o que torna a operação **auditável**.

Dica

Ao definir a janela de execução, prefira horários de menor concorrência (por exemplo, madrugada) para processos pesados e evite fazer todos os robôs iniciarem no mesmo minuto “redondo”. Distribuir os gatilhos reduz picos de carga no Runner e facilita a leitura dos Logs.

13.6 Etapa 6 — Monitoramento contínuo

Deploy e agendamento não encerram o ciclo de vida do robô — iniciam sua vida em produção. A partir daqui, a **Operação/Suporte** acompanha as execuções pelos **Logs (Capítulo 07)**: sucessos, falhas, tempo de execução e mensagens de erro. É o monitoramento que sustenta a classificação operacional do control tower e permite reagir a falhas antes que o negócio sinta o impacto.

Informação

O smoke test valida o robô *uma vez*, no momento do deploy. O monitoramento valida o robô *sempre*. Sistemas-alvo mudam de layout, credenciais expiram e a rede oscila — por isso o acompanhamento contínuo dos Logs é parte permanente da operação, não uma etapa que termina.

13.7 Fluxo de trabalho resumido

A tabela a seguir consolida as seis etapas, com o responsável e a condição objetiva para avançar de cada uma para a próxima. Use-a como **checklist** sempre que colocar um RPA em produção:

Etapa	Descrição	Responsável	Condição de avanço
1. Merge na main	Código finalizado e aprovado no MR, pronto para produção.	Desenvolvedor original	MR aprovado e merge concluído
2. Revisão final	Code Review focado em governança, tratamento de exceções e aderência ao negócio.	Revisor (Pleno/Sênior)	Aprovação do revisor
3. Deploy em produção	Upload manual do .zip da automação no orquestrador Smart Office.	Revisor (Pleno/Sênior)	Deploy concluído com sucesso
4. Validação (Smoke Test)	Envio manual de Tasks mínimas e não críticas para testar conectividade, credenciais e fluxo principal.	Revisor (Pleno/Sênior)	Tasks de teste executadas com sucesso
5. Agendamento	Configuração do Schedule conforme a demanda do negócio e a capacidade do ambiente.	Revisor (Pleno/Sênior)	Schedule configurado e robô ativo
6. Monitoramento	Acompanhamento contínuo da execução em produção via Logs.	Operação / Suporte	Processo em execução estável

Dica

Este é o ponto em que as duas partes do manual se encontram. A **Parte I (operação)** ensinou o que o Smart Office é e como ele orquestra — Runner, Tasks, Schedule e Logs. A **Parte II (desenvolvimento)** ensinou a construir um RPA que merece produção — arquitetura modular, Clean Code, tratamento de exceções, padrão GitLab e documentação. O deploy é a costura entre

elas: um bom código (Parte II) só entrega valor quando é publicado, validado e monitorado corretamente no orquestrador (Parte I). Domine as duas metades e você fecha o ciclo completo — da linha de código à execução confiável em produção.

fim do capítulo ■



Digital Transformation

Glossário

Automation

Registro lógico do orquestrador que identifica um RPA e vincula o seu `bot.py` a um ou mais Runners. Controla versão, deploy e status da automação.

Bot

O código da automação propriamente dito, empacotado em um `.zip` com `bot.py` e `requirements.txt` na raiz. É o que o Runner executa.

BotCity

Framework de RPA em Python usado para escrever bots. O Smart Office orquestra bots feitos em BotCity, mas não depende dele — ver **agnóstico**.

Control Tower

Torre de controle: painel único de orquestração e monitoramento de todos os RPAs. É o papel que o Smart Office desempenha.

CRON

Notação para expressar horários e periodicidade de execução (minuto, hora, dia do mês, mês, dia da semana). Usada nas Schedules.

DX Lake

Data lake da LG onde ficam armazenados os códigos-fonte dos RPAs. O módulo RPA Database do Smart Office se integra a ele para permitir o *download* do código.

FTE

Full-Time Equivalent: tempo gasto para executar um processo manualmente. Métrica usada para dimensionar o ganho da automação.

GitFlow

Modelo de versionamento com *branches* de papéis fixos (`main`, `develop`, `feature/`, `bugfix/`, `hotfix/`, `release/`).

GitLab

Plataforma de versionamento adotada pela LG (o “The DX Way”). É a única integração de versionamento do Smart Office — não há GitHub Actions nem *pipeline* de CI/CD.

Merge Request (MR)

Pedido de integração de código no GitLab; principal ponto de controle de qualidade, com revisão obrigatória por um desenvolvedor sênior.

Orquestrador

Sistema que agenda, dispara, distribui e monitora a execução de automações. No Smart Office, corresponde ao módulo *Orchestrator*.

Playwright

Framework de automação de navegador. Como BotCity e Selenium, pode ser orquestrado pelo Smart Office.

RPA

Robotic Process Automation: automação de tarefas repetitivas por meio de um robô de software que opera sistemas e planilhas como um usuário.

Runner

A **máquina real** (física ou virtual) onde o RPA de fato roda. Fica aguardando as automações da fila e as executa por ordem de prioridade.

Runner Client

Aplicação instalada na máquina para transformá-la em um Runner e vinculá-la ao orquestrador.

Schedule

Execução **recorrente** de uma automação, definida por uma *string* CRON. Diferente da Task, repete-se em intervalos.

Selenium

Framework de automação de navegador em Python, muito usado nos RPAs. Orquestrável pelo Smart Office.

Smoke Test

Teste mínimo pós-deploy que valida conectividade, credenciais e o fluxo principal antes de o RPA ser agendado em produção.

Task

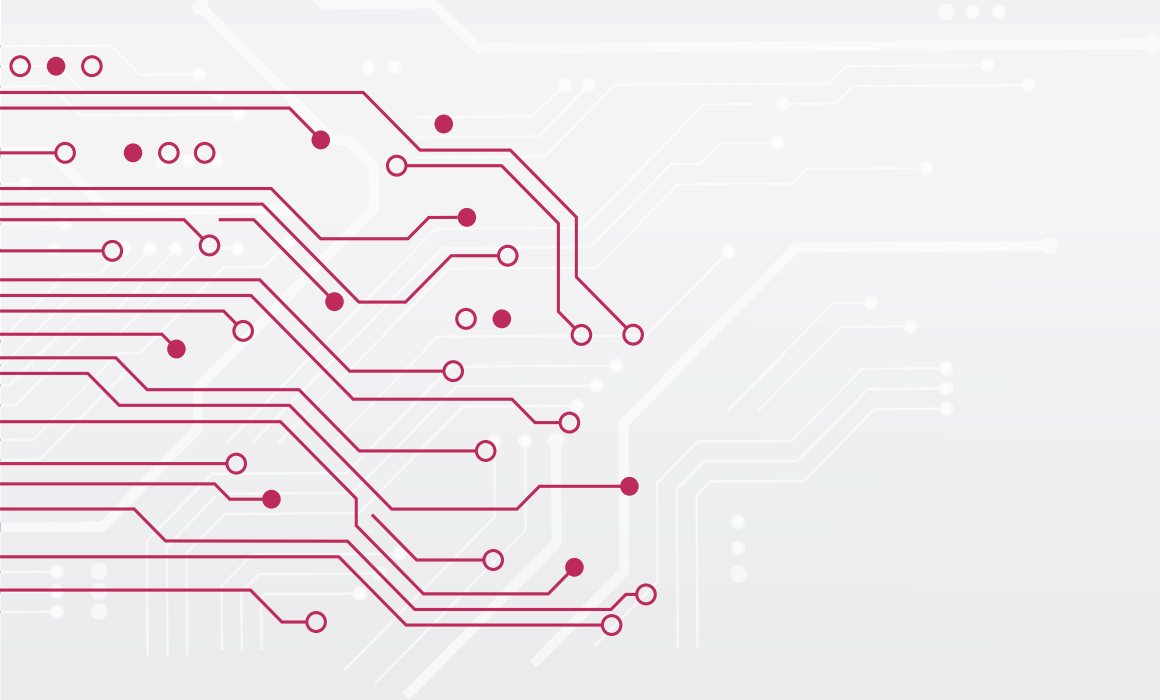
Ordem de execução **única** de uma automação para um momento agendado. Ao terminar, não se repete.

venv

Ambiente virtual Python provisionado pelo servidor para isolar as dependências (**requirements.txt**) de cada automação antes de acioná-la no Runner.



Digital Transformation



Digital Transformation